

PHAR520: Applied Omics Science for Drug Discovery and Development

ModuleII: 2025_RNA-Seq-Analysis

Contents

RNA-seq analysis workshop	13
1 Differential gene expression (DGE) analysis overview	14
1.1 RNA-seq Introduction: MOV10-FMRP Study	15
1.2 Setting up	17
1.3 [1] “data.frame”	18
1.4 [1] “data.frame”	18
1.5 [1] “Irrel_kd_1” “Irrel_kd_2” “Irrel_kd_3” “Mov10_kd_2” “Mov10_kd_3”	19
1.6 [6] “Mov10_oe_1” “Mov10_oe_2” “Mov10_oe_3”	19
1.7 [1] “Mov10_kd_2” “Mov10_kd_3” “Mov10_oe_1” “Mov10_oe_2” “Mov10_oe_3”	19
1.8 [6] “Irrel_kd_1” “Irrel_kd_2” “Irrel_kd_3”	19
1.9 Differential gene expression analysis overview	19
2 Count normalization	28
2.1 Normalization	28
2.2 Count normalization of Mov10	34
2.3 [1] TRUE	34
2.4 [1] FALSE	34
2.5 [1] TRUE	34
2.6 Irrel_kd_1 Irrel_kd_2 Irrel_kd_3 Mov10_kd_2 Mov10_kd_3	36
2.7 1/2-SBSRNA4 45 31 39 57 41	36
2.8 A1BG 77 58 40 71 40	36
2.9 A1BG-AS1 213 172 126 256 177	36

<i>CONTENTS</i>	3
2.10 A1CF 0 0 0 0 1	36
2.11 A2LD1 91 80 50 146 81	36
2.12 A2M 9 8 4 10 9	36
2.13 [1] “design” “dispersionFunction” “rowRanges”	36
2.14 [4] “colData” “assays” “NAMES”	36
2.15 [7] “elementMetadata” “metadata”	36
2.16 Irrel_kd_1 Irrel_kd_2 Irrel_kd_3 Mov10_kd_2 Mov10_kd_3 Mov10_oe_1 Mov10_oe_2	37
2.17 1.1224020 0.9625632 0.7477715 1.5646728 0.9351760 1.2016082 1.1205912	37
2.18 Mov10_oe_3	37
2.19 0.6534987	37
3 Quality Control (QC)	38
3.1 Sample-level QC	38
3.2 Gene-level QC	47
3.3 Mov10 quality assessment and exploratory analysis using DESeq2	47
3.4 Irrel_kd_1 Irrel_kd_2 Irrel_kd_3 Mov10_kd_2 Mov10_kd_3 Mov10_oe_1	51
3.5 Irrel_kd_1 1.0000000 0.9999614 0.9999532 0.9997202 0.9997748 0.9996700	51
3.6 Irrel_kd_2 0.9999614 1.0000000 0.9999544 0.9996918 0.9997568 0.9996984	51
3.7 Irrel_kd_3 0.9999532 0.9999544 1.0000000 0.9996816 0.9997574 0.9997067	51
3.8 Mov10_kd_2 0.9997202 0.9996918 0.9996816 1.0000000 0.9999492 0.9994868	51
3.9 Mov10_kd_3 0.9997748 0.9997568 0.9997574 0.9999492 1.0000000 0.9996154	51
3.10 Mov10_oe_1 0.9996700 0.9996984 0.9997067 0.9994868 0.9996154 1.0000000	51
3.11 Mov10_oe_2 Mov10_oe_3	51
3.12 Irrel_kd_1 0.9996599 0.9995804	51
3.13 Irrel_kd_2 0.9996825 0.9996227	51
3.14 Irrel_kd_3 0.9997090 0.9996026	51
3.15 Mov10_kd_2 0.9994565 0.9993869	51
3.16 Mov10_kd_3 0.9995905 0.9995235	51
3.17 Mov10_oe_1 0.9999505 0.9999196	51
4 Differential expression analysis with DESeq2	55

4.1	Running DESeq2	56
4.2	DESeq2 differential gene expression analysis workflow	59
4.3	Irrel_kd_1 Irrel_kd_2 Irrel_kd_3 Mov10_kd_2 Mov10_kd_3 Mov10_oe_1 Mov10_oe_2	60
4.4	1.1224020 0.9625632 0.7477715 1.5646728 0.9351760 1.2016082 1.1205912	60
4.5	Mov10_oe_3	60
4.6	0.6534987	60
4.7	Irrel_kd_1 Irrel_kd_2 Irrel_kd_3 Mov10_kd_2 Mov10_kd_3 Mov10_oe_1 Mov10_oe_2	61
4.8	22687366 19381680 14962754 32826936 19360003 23447317 21713289	61
4.9	Mov10_oe_3	61
4.10	12737889	61
4.11	Irrel_kd_1 Irrel_kd_2 Irrel_kd_3 Mov10_kd_2 Mov10_kd_3 Mov10_oe_1 Mov10_oe_2	61
4.12	20213226 20135487 20009794 20980064 20701989 19513279 19376636	61
4.13	Mov10_oe_3	61
4.14	19491836	61
5	Negative Binomial model fitting	66
5.1	Generalized Linear Model fit for each gene	66
5.2	Shrunken log2 foldchanges (LFC)	67
5.3	Hypothesis testing using the Wald test	68
5.4	Creating contrasts	68
5.5	[1] “Intercept”	69
5.6	[2] “samplotype_MOV10_knockdown_vs_control”	69
5.7	[3] “samplotype_MOV10_overexpression_vs_control”	69
5.8	[1] “DESeqResults”	71
5.9	attr(“package”)	71
5.10	[1] “DESeq2”	71
5.11	DataFrame with 5 rows and 2 columns	72
5.12	type description	72
5.13		72
5.14	baseMean intermediate mean of normalized c.. . . .	72

5.15	log2FoldChange results log2 fold change (MA..	72
5.16	lfcSE results posterior SD: sample..	72
5.17	pvalue results Wald test p-value: s..	72
5.18	padj results BH adjusted p-values	72
5.19	log2 fold change (MAP): samplotype MOV10_overexpression vs control	73
5.20	Wald test p-value: samplotype MOV10 overexpression vs control	73
5.21	DataFrame with 6 rows and 5 columns	73
5.22	baseMean log2FoldChange lfcSE pvalue padj	73
5.23		73
5.24	1/2-SBSRNA4 45.652040 0.16820235 0.209100 0.1610752 0.2750250	73
5.25	A1BG 61.093102 0.13645896 0.176779 0.2401909 0.3716156	73
5.26	A1BG-AS1 175.665807 -0.04046122 0.116928 0.6789312 0.7840468	73
5.27	A1CF 0.237692 0.00626747 0.210057 0.7945932 NA	73
5.28	A2LD1 89.617985 0.29017719 0.195395 0.0333343 0.0774672	73
5.29	A2M 5.860084 -0.09619722 0.233561 0.1057823 0.1976308	73
5.30	[1] "baseMean" "log2FoldChange" "lfcSE" "pvalue"	74
5.31	[5] "padj"	74
5.32	[1] 7555	75
5.33	[1] 6464	75
5.34	[1] "Intercept"	76
5.35	[2] "samplotype_MOV10_knockdown_vs_control"	76
5.36	[3] "samplotype_MOV10_overexpression_vs_control"	76
6	Visualizing RNA-seq results	78
6.1	[1] TRUE	79
6.2	# A tibble: 6 x 3	85
6.3	gene samplename normalized_counts	85
6.4		85
6.5	1 ADAMTS1 Irrel_kd_1 6718.	85
6.6	2 ADAMTS1 Irrel_kd_2 6455.	85

6.7	3 ADAMTS1 Irrel_kd_3 6802.	85
6.8	4 ADAMTS1 Mov10_kd_2 6869.	85
6.9	5 ADAMTS1 Mov10_kd_3 8731.	85
6.10	6 ADAMTS1 Mov10_oe_1 13252.	85
6.11	[1] “gene” “baseMean” “log2FoldChange” “lfcSE”	88
6.12	[5] “pvalue” “padj” “threshold_OE”	88
6.13	[1] “gene” “baseMean” “log2FoldChange” “lfcSE”	88
6.14	[5] “pvalue” “padj”	88
6.15		88
6.16	FALSE TRUE	88
6.17	18793 955	88
6.18	[1] “MOV10” “H1F0” “HIST1H1C” “TXNIP” “NEAT1” “KLF10”	90
6.19	[7] “INSIG1” “NR1D1” “WDFY1” “HSPA1A” “ ” “ ”	90
6.20	[13] “ ” “ ” “ ” “ ” “ ” “ ”	90
6.21	[19] “ ” “ ”	90
7	Summary of DGE workflow	91
7.1	R version 4.5.1 (2025-06-13)	95
7.2	Platform: x86_64-apple-darwin20	95
7.3	Running under: macOS Sequoia 15.5	95
7.4		95
7.5	Matrix products: default	95
7.6	BLAS: /Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/lib/libRblas.0.dylib	95
7.7	LAPACK: /Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/lib/libRlapack.dylib; LAPACK version 3.12.1	95
7.8		95
7.9	locale:	95
7.10	[1] en_CA.UTF-8/en_CA.UTF-8/en_CA.UTF-8/C/en_CA.UTF-8/en_CA.UTF-8	95
7.11		95
7.12	time zone: America/Los_Angeles	95
7.13	tzcode source: internal	95

7.14		95
7.15	attached base packages:	95
7.16	[1] stats4 stats graphics grDevices utils datasets methods	95
7.17	[8] base	95
7.18		95
7.19	other attached packages:	95
7.20	[1] ggrepel_0.9.6 pheatmap_1.0.13	95
7.21	[3] DESeq2_1.48.2 SummarizedExperiment_1.38.1	95
7.22	[5] Biobase_2.68.0 MatrixGenerics_1.20.0	95
7.23	[7] matrixStats_1.5.0 GenomicRanges_1.60.0	95
7.24	[9] GenomeInfoDb_1.44.3 IRanges_2.42.0	95
7.25	[11] S4Vectors_0.46.0 BiocGenerics_0.54.1	95
7.26	[13] generics_0.1.4 RColorBrewer_1.1-3	95
7.27	[15] lubridate_1.9.4 forcats_1.0.1	95
7.28	[17] stringr_1.5.2 dplyr_1.1.4	95
7.29	[19] purrr_1.1.0 readr_2.1.5	95
7.30	[21] tidyr_1.3.1 tibble_3.3.0	95
7.31	[23] ggplot2_4.0.0 tidyverse_2.0.0	95
7.32		95
7.33	loaded via a namespace (and not attached):	95
7.34	[1] gtable_0.3.6 xfun_0.53 lattice_0.22-7	95
7.35	[4] numDeriv_2016.8-1.1 tzdb_0.5.0 vctrs_0.6.5	95
7.36	[7] tools_4.5.1 parallel_4.5.1 fansi_1.0.6	95
7.37	[10] pkgconfig_2.0.3 Matrix_1.7-3 S7_0.2.0	95
7.38	[13] lifecycle_1.0.4 GenomeInfoDbData_1.2.14 compiler_4.5.1	95
7.39	[16] farver_2.1.2 codetools_0.2-20 htmltools_0.5.8.1	95
7.40	[19] yaml_2.3.10 pillar_1.11.1 crayon_1.5.3	95
7.41	[22] MASS_7.3-65 BiocParallel_1.42.2 DelayedArray_0.34.1	95
7.42	[25] emdbook_1.3.14 abind_1.4-8 bdsmatrix_1.3-7	95
7.43	[28] locfit_1.5-9.12 tidyselect_1.2.1 digest_0.6.37	95

7.44	[31] mvtnorm_1.3-3 stringi_1.8.7 bookdown_0.45	95
7.45	[34] labeling_0.4.3 fastmap_1.2.0 grid_4.5.1	95
7.46	[37] cli_3.6.5 SparseArray_1.8.1 magrittr_2.0.4	95
7.47	[40] S4Arrays_1.8.1 utf8_1.2.6 withr_3.0.2	95
7.48	[43] scales_1.4.0 UCSC.utils_1.4.0 timechange_0.3.0	95
7.49	[46] rmarkdown_2.30 XVector_0.48.0 httr_1.4.7	95
7.50	[49] hms_1.1.4 coda_0.19-4.1 evaluate_1.0.5	95
7.51	[52] knitr_1.50 bbmle_1.0.25.1 rlang_1.1.6	95
7.52	[55] Rcpp_1.1.0 glue_1.8.0 apeglm_1.30.0	95
7.53	[58] rstudioapi_0.17.1 jsonlite_2.0.0 plyr_1.8.9	95
7.54	[61] R6_2.6.1	95
8	Functional analysis of RNA-seq data	96
8.1	Understanding Gene Ontology (GO) Enrichment	96
8.2	Perform the hypergeometric test: $P(X \geq q)$	100
8.3	[1] 2.551199e-06	101
8.4	Over-Representation Analysis (ORA)	102
8.5	[1] “gene_symbol” “ncbi_gene” “ensembl_gene”	107
8.6	[4] “db_gene_symbol” “db_ncbi_gene” “db_ensembl_gene”	107
8.7	[7] “source_gene” “gs_id” “gs_name”	107
8.8	[10] “gs_collection” “gs_subcollection” “gs_collection_name”	107
8.9	[13] “gs_description” “gs_source_species” “gs_pmid”	107
8.10	[16] “gs_geoid” “gs_exact_source” “gs_url”	107
8.11	[19] “db_version” “db_target_species”	107
8.12	# A tibble: 6 x 2	108
8.13	gene_symbol gs_id	108
8.14		108
8.15	1 AASDHPPT M29088	108
8.16	2 ALDH1L1 M29088	108
8.17	3 ALDH1L2 M29088	108

8.18	4 MTHFD1 M29088	108
8.19	5 MTHFD1L M29088	108
8.20	6 MTHFD2L M29088	108
8.21	[1] 617648 20	108
8.22	\$GOBP_10_FORMYLTETRAHYDROFOLATE_METABOLIC_PROCESS	109
8.23	[1] "AASDHPPT" "ALDH1L1" "ALDH1L2" "MTHFD1" "MTHFD1L" "MTHFD2L"	109
8.24		109
8.25	\$GOBP_2_OXOGLUTARATE_METABOLIC_PROCESS	109
8.26	[1] "AADAT" "ADHFE1" "COL6A1" "D2HGDH" "DLD" "DLST" "GOT1" "GOT2"	109
8.27	[9] "GPT2" "IDH1" "IDH2" "KGD4" "KYAT3" "L2HGDH" "OGDH" "OGDHL"	109
8.28	[17] "PHYH" "TAT"	109
8.29	\$10 FORMYLTETRAHYDROFOLATE METABOLIC PROCESS	110
8.30	[1] "AASDHPPT" "ALDH1L1" "ALDH1L2" "MTHFD1" "MTHFD1L" "MTHFD2L"	110
8.31		110
8.32	\$2 OXOGLUTARATE METABOLIC PROCESS	110
8.33	[1] "AADAT" "ADHFE1" "COL6A1" "D2HGDH" "DLD" "DLST" "GOT1" "GOT2"	110
8.34	[9] "GPT2" "IDH1" "IDH2" "KGD4" "KYAT3" "L2HGDH" "OGDH" "OGDHL"	110
8.35	[17] "PHYH" "TAT"	110
8.36	# A tibble: 6 x 2	111
8.37	gs_name gs_id	111
8.38		111
8.39	1 GOBP_10_FORMYLTETRAHYDROFOLATE_METABOLIC_PROCESS M29088	111
8.40	2 GOBP_2FE_2S_CLUSTER_ASSEMBLY M23654	111
8.41	3 GOBP_2_OXOGLUTARATE_METABOLIC_PROCESS M15857	111
8.42	4 GOBP_3_UTR_MEDIATED_MRNA_DESTABILIZATION M24263	111
8.43	5 GOBP_3_UTR_MEDIATED_MRNA_STABILIZATION M11060	111
8.44	6 GOBP_4FE_4S_CLUSTER_ASSEMBLY M45874	111
8.45	# A tibble: 6 x 2	112
8.46	term GOID	112
8.47		112

8.48	1	10	FORMYLTETRAHYDROFOLATE METABOLIC PROCESS M29088	112
8.49	2	2FE 2S	CLUSTER ASSEMBLY M23654	112
8.50	3	2	OXOGLUTARATE METABOLIC PROCESS M15857	112
8.51	4	3	UTR MEDIATED MRNA DESTABILIZATION M24263	112
8.52	5	3	UTR MEDIATED MRNA STABILIZATION M11060	112
8.53	6	4FE 4S	CLUSTER ASSEMBLY M45874	112
8.54	#	A	tibble: 6 x 2	113
8.55	gs_name	gene_symbol		113
8.56				113
8.57	1	GOBP_10_FORMYLTETRAHYDROFOLATE_METABOLIC_PROCESS	AASDHPPT	113
8.58	2	GOBP_10_FORMYLTETRAHYDROFOLATE_METABOLIC_PROCESS	ALDH1L1	113
8.59	3	GOBP_10_FORMYLTETRAHYDROFOLATE_METABOLIC_PROCESS	ALDH1L2	113
8.60	4	GOBP_10_FORMYLTETRAHYDROFOLATE_METABOLIC_PROCESS	MTHFD1	113
8.61	5	GOBP_10_FORMYLTETRAHYDROFOLATE_METABOLIC_PROCESS	MTHFD1L	113
8.62	6	GOBP_10_FORMYLTETRAHYDROFOLATE_METABOLIC_PROCESS	MTHFD2L	113
8.63	#	A	tibble: 6 x 2	114
8.64	term	gene		114
8.65				114
8.66	1	10	FORMYLTETRAHYDROFOLATE METABOLIC PROCESS AASDHPPT	114
8.67	2	10	FORMYLTETRAHYDROFOLATE METABOLIC PROCESS ALDH1L1	114
8.68	3	10	FORMYLTETRAHYDROFOLATE METABOLIC PROCESS ALDH1L2	114
8.69	4	10	FORMYLTETRAHYDROFOLATE METABOLIC PROCESS MTHFD1	114
8.70	5	10	FORMYLTETRAHYDROFOLATE METABOLIC PROCESS MTHFD1L	114
8.71	6	10	FORMYLTETRAHYDROFOLATE METABOLIC PROCESS MTHFD2L	114
8.72	gene	score	index	115
8.73	1	AASDHPPT 0.5082527	1	115
8.74	2	ALDH1L1 0.1969996	1	115
8.75	3	ALDH1L2 0.1870385	1	115
8.76	4	MTHFD1 0.1142226	1	115
8.77	5	MTHFD1L 0.7391241	1	115

8.78	6	MTHFD2L	0.8296142	0	115
8.79	[1]	18000	3		115
8.80	[1]	18000			115
8.81	[1]	5037			116
8.82					116
8.83	0	1			116
8.84	12963	5037			116
8.85	[1]	“source” “target” “overlapCoeff” “width” “label”			116
8.86	[1]	“filename” “GOID” “term”			117
8.87	[4]	“nGenes” “nQuery” “nOverlap”			117
8.88	[7]	“querySetFraction” “geneSetFraction” “foldEnrichment”			117
8.89	[10]	“P” “FDR” “overlapGenes”			117
8.90	[13]	“maxOverlapGeneScore” “cluster” “id”			117
8.91	[16]	“size” “formattedLabel”			117
8.92		GOID term nGenes nQuery P			117
8.93	1	M16023 RNA LOCALIZATION	204 4998 4.42e-15		117
8.94	2	M16816 REGULATION OF RNA SPLICING	189 5150 1.29e-14		117
8.95	3	M24840 VESICLE MEDIATED TRANSPORT IN SYNAPSE	260 5117 2.32e-13		117
8.96	4	M15413 NUCLEAR EXPORT	172 5112 1.40e-12		117
8.97	5	M15304 STEM CELL DIFFERENTIATION	265 5003 1.85e-11		117
8.98	6	M23968 ESTABLISHMENT OF RNA LOCALIZATION	165 5115 1.57e-11		117
8.99	[1]	59 5			118
8.100	[1]	57 17			118
8.101					118
8.102	===	Top enricher results	===		118
8.103	Description				120
8.104	RNA LOCALIZATION	RNA LOCALIZATION			120
8.105	REGULATION OF RNA SPLICING	REGULATION OF RNA SPLICING			120
8.106	VESICLE MEDIATED TRANSPORT IN SYNAPSE	VESICLE MEDIATED TRANSPORT IN SYNAPSE			120
8.107	NUCLEAR EXPORT	NUCLEAR EXPORT			120

8.108	ESTABLISHMENT OF RNA LOCALIZATION ESTABLISHMENT OF RNA LOCALIZATION	120
8.109	STEM CELL DIFFERENTIATION STEM CELL DIFFERENTIATION	120
8.110	Count pvalue qvalue	120
8.111	RNA LOCALIZATION 110 4.417655e-15 1.053727e-11	120
8.112	REGULATION OF RNA SPLICING 103 1.293937e-14 1.543190e-11	120
8.113	VESICLE MEDIATED TRANSPORT IN SYNAPSE 128 2.315881e-13 1.841329e-10	120
8.114	NUCLEAR EXPORT 92 1.404827e-12 8.377206e-10	120
8.115	ESTABLISHMENT OF RNA LOCALIZATION 87 1.573398e-11 7.345982e-09	120
8.116	STEM CELL DIFFERENTIATION 125 1.847842e-11 7.345982e-09	120
8.117		121
8.118	=== Top runGORESP results ===	121
8.119	term nOverlap P FDR	121
8.1201	RNA LOCALIZATION 110 4.42e-15 1.48e-11	121
8.1212	REGULATION OF RNA SPLICING 103 1.29e-14 2.17e-11	121
8.1223	VESICLE MEDIATED TRANSPORT IN SYNAPSE 128 2.32e-13 2.59e-10	121
8.1234	NUCLEAR EXPORT 92 1.40e-12 1.18e-09	121
8.1245	STEM CELL DIFFERENTIATION 125 1.85e-11 1.03e-08	121
8.1256	ESTABLISHMENT OF RNA LOCALIZATION 87 1.57e-11 1.03e-08	121
8.126[1]	65 12	121
8.127[1]	57 17	122
8.128	Overlapping terms: 57	122
8.129	Overlap percentage: 87.7 %	122
8.130	Gene Set Enrichment Analysis (GSEA) - Optional	123

RNA-seq analysis workshop

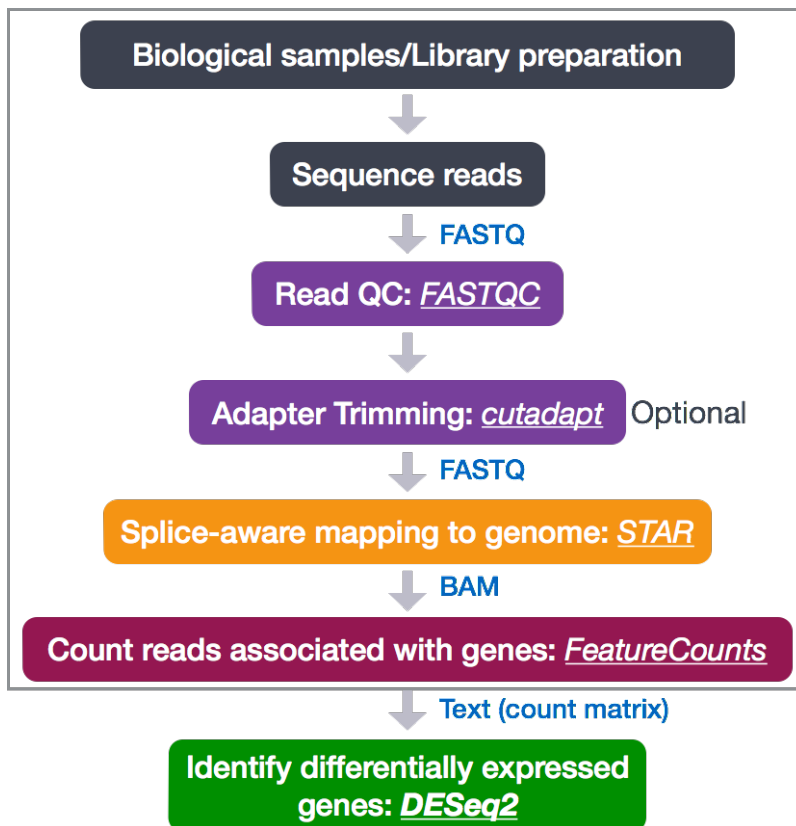
The published version for this module can be found on my bookdown site [2025_RNA-Seq-Analysis](#).

Chapter 1

Differential gene expression (DGE) analysis overview

The goal of RNA-seq is to perform differential expression testing to determine which genes are expressed at different levels between conditions. These genes can offer biological insight into the processes affected by the condition(s) of interest.

To determine the expression levels of genes, our RNA-seq workflow follows the steps detailed in the image below inside the box. All steps are performed on the command line (Linux/Unix) through the generation of read counts per gene as discussed in Corey's lectures. The differential expression analysis and any downstream functional analysis are generally performed in R using R packages specifically designed for the complex statistical analyses required to determine whether genes are differentially expressed starting from the count matrices.



In the next few lessons, we will walk you through an **end-to-end gene-level RNA-seq differential expression workflow** using various R packages. We will start with the count matrix, perform exploratory data analysis for quality assessment and to explore the relationship between samples, perform differential expression analysis, and visually explore the results prior to performing downstream functional analysis.

1.1 RNA-seq Introduction: MOV10-FMRP Study

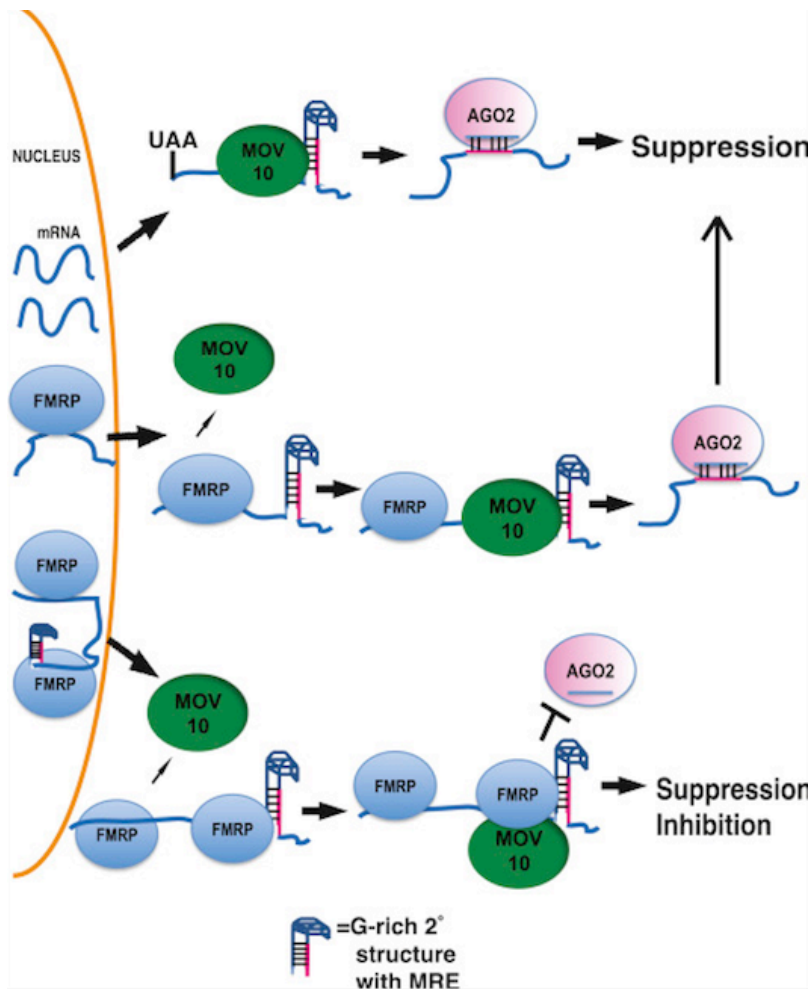
- **Reference:** Kenny PJ et al, Cell Rep 2014

1.1.1 Background: MOV10-FMRP Biology

What is the function of FMRP and MOV10

FMRP is the Fragile X Messenger Ribonucleoprotein. Loss of **FMRP** causes **Fragile X Syndrome (FXS)**, characterized by intellectual disability and autism spectrum disorder. **FMRP** regulates translation of a diverse and functionally important set of genes that are crucial for brain development, synaptic function, and neuronal plasticity by binding to their mRNAs and interacting with components of the microRNA (miRNA) pathway.

MOV10 usually facilitates translation suppression through the miRNA pathway. **MOV10-FMRP** interactions can either enhance this suppression or, when the **MOV10-FMRP** complex binds in proximity to **miRNA response elements (MREs)**, can sterically block **AGO2** access and translation is no longer suppressed.



Model for **MOV10-FMRP** Cooperation in miRNA-Mediated Translation Regulation

Top: **MOV10** helicase unwinds G-rich structures in 3' UTRs, exposing **miRNA response elements (MREs)** for **AGO2/RNA-induced silencing complex (RISC)** binding and gene silencing by translational repression.

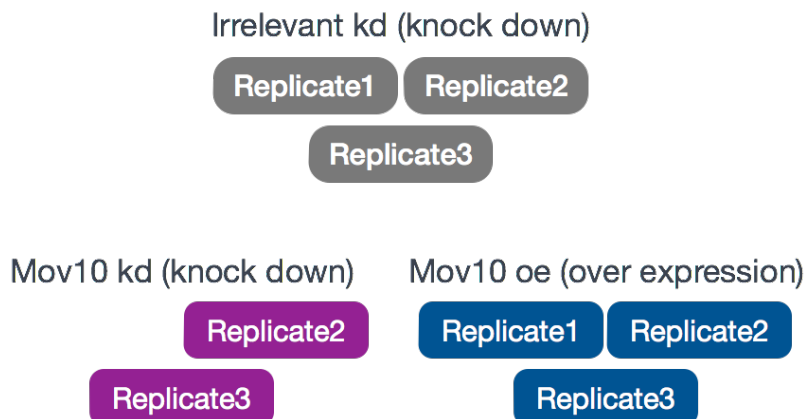
Middle: **FMRP** recruits **MOV10** to remodel RNA structure, facilitating **AGO2/RISC** access to **MREs** and translational repression.

Bottom: **FMRP-MOV10** complex binds in proximity to **MREs**, sterically blocking **AGO2/RISC** access and preventing miRNA-mediated suppression of translation.

1.1.2 Review of the dataset

Experimental design:

- **Cell type:** HEK293F cells
- **Conditions:** Cells were either transfected with a **MOV10** transgene, or siRNA to knock down **Mov10** expression, or non-specific (irrelevant) siRNA. This resulted in 3 conditions **Mov10 oe** (over expression), **Mov10 kd** (knock down) and **Irrelevant kd**, respectively.
- **Replicates:** 2-3 biological replicates per condition (shown below)



Research objectives:

The aim of the RNA-seq part of the study was to identify **MOV10** target genes and determine which overlap with **FMRP** targets, suggesting regulation by the **MOV10-FMRP** complex. Our analysis will include:

- Differential gene expression analysis (**MOV10** KD vs. control, **MOV10** OE vs. control)
- Identifying direct **MOV10** targets (genes with opposite responses to KD vs. OE)
- Functional enrichment of **MOV10**-responsive genes

Links to raw data [GEO]: <https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE51443> “Gene Expression Omnibus”
[SRA]: <https://trace.ncbi.nlm.nih.gov/Traces/sra/?study=SRP031507> “Sequence Read Archive”

1.2 Setting up

Make your personal codebook from the template

- In the **Files** pane, navigate to ‘09-DGE_codebook_template.Rmd’
- Check the box next to that file → click **More** → **Copy To...**
- In the dialog, choose the **students_notes/** folder and set the new name to: `<lastname>_09-DGE_codebook.Rmd` (for example, ‘garcia_09-DGE_codebook.Rmd’) → **OK**.
- Click the new file in **students_notes/** to open it. This is now **your** working copy for your homework. This will open in the script editor in the top left hand corner. This is where we will be running all commands required for the analysis, and like last time this is where you will enter the code/answers for your homework assignment.

From now on, **work only inside students_notes/**. Anything you create or edit there is yours and won’t interfere with course updates.

1.2.1 Loading libraries

For this analysis we will be using several R packages, some which have been installed from CRAN and others from Bioconductor. To use these packages (and the functions contained within them), we need to **load the libraries**.

```
## Setup
### Bioconductor and CRAN libraries used
library(tidyverse)
library(RColorBrewer)
library(DESeq2)
library(pheatmap)
library(ggplot2)
library(ggrepel)
```

1.2.2 Loading data

To load the data into our current environment, we will be using the `read.table` function. We need to provide the path to each file and also specify arguments to let R know that we have a header (`header = T`) and the first column is our row names (`row.names = 1`) and the `sep = "\t"` for tab.

```
## Load in data
data <- read.table("data/Mov10_full_counts.txt",
                  header=T, row.names=1, sep = "\t")

meta <- read.table("data/Mov10_full_meta.txt",
                  header=T, row.names=1, , sep = "\t")
```

Use `class()` to inspect our data and make sure we are working with data frames:

```
### Check classes of the data we just brought in
class(meta)
```

1.3 [1] “data.frame”

```
class(data)
```

1.4 [1] “data.frame”

1.4.1 Viewing data

Make sure your datasets contain the expected samples / information before proceeding to perform any type of analysis.

```
rownames(meta)
```

1.5 [1] "Irrel_kd_1" "Irrel_kd_2" "Irrel_kd_3" "Mov10_kd_2" "Mov10_kd_3"

1.6 [6] "Mov10_oe_1" "Mov10_oe_2" "Mov10_oe_3"

```
colnames(data)
```

1.7 [1] "Mov10_kd_2" "Mov10_kd_3" "Mov10_oe_1" "Mov10_oe_2"
"Mov10_oe_3"

1.8 [6] "Irrel_kd_1" "Irrel_kd_2" "Irrel_kd_3"

You'll notice the `colnames` of the data are the sample names, and the `rownames` of the metadata are the sample names except in a different order. This is important as we will be merging the metadata with the data based on the sample names.

1.9 Differential gene expression analysis overview

So what does this count data actually represent? The count data used for differential expression analysis represents the number of sequence reads that originated from a particular gene. The higher the number of counts, the more reads associated with that gene, and the assumption that there was a higher level of expression of that gene in the sample.

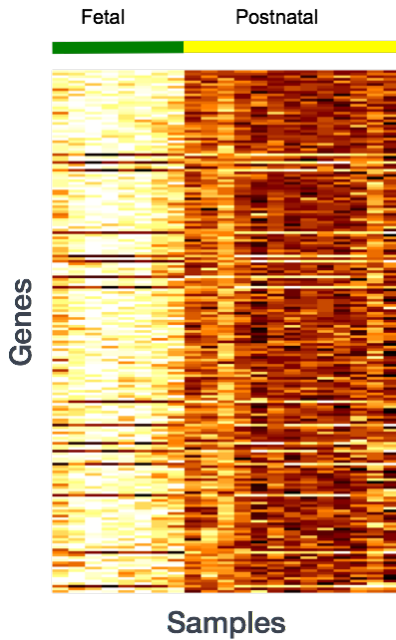
**samples: want to see if differences across
condition are significant
(w.r.t. biological and technical variation)**

features (e.g. genes)

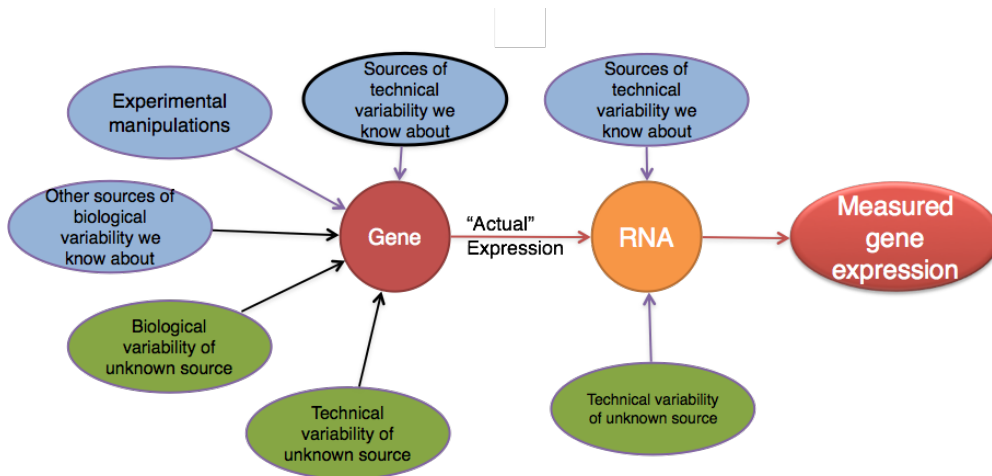
	SRR1039508	SRR1039509	SRR1039512	SRR1039513	SRR1039516
ENSG000000000003	679	448	873	408	1138
ENSG000000000005	0	0	0	0	0
ENSG000000000419	467	515	621	365	587
ENSG000000000457	260	211	263	164	245
ENSG000000000460	60	55	40	35	78

With differential expression analysis, we are looking for genes that change in expression between two or more groups (defined in the metadata) - case vs. control - correlation of expression with some variable or clinical outcome

Why does it not work to identify differentially expressed gene by ranking the genes by how different they are between the two groups (based on fold change values)?

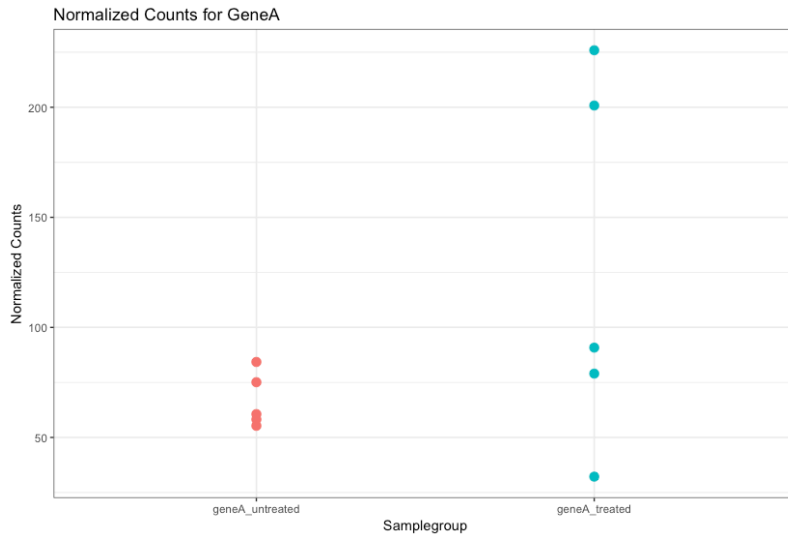


More often than not, there is much more going on with your data than what you are anticipating. Genes that vary in expression level between samples is a consequence of not only the experimental variables of interest but also due to extraneous sources. The goal of differential expression analysis is to determine the relative role of these effects, and to separate the “interesting” from the “uninteresting”.



Courtesy of Paul Pavlidis, UBC

The “uninteresting” presents as sources of variation in your data, and so even though the mean expression levels between sample groups may appear to be quite different, it is possible that the difference is not actually significant. This is illustrated for ‘GeneA’ expression between ‘untreated’ and ‘treated’ groups in the figure below. The mean expression level of geneA for the ‘treated’ group is twice as large as for the ‘untreated’ group, but the variation between replicates indicates that this may not be a significant difference. **We need to take into account the variation in the data (and where it might be coming from) when determining whether genes are differentially expressed.**

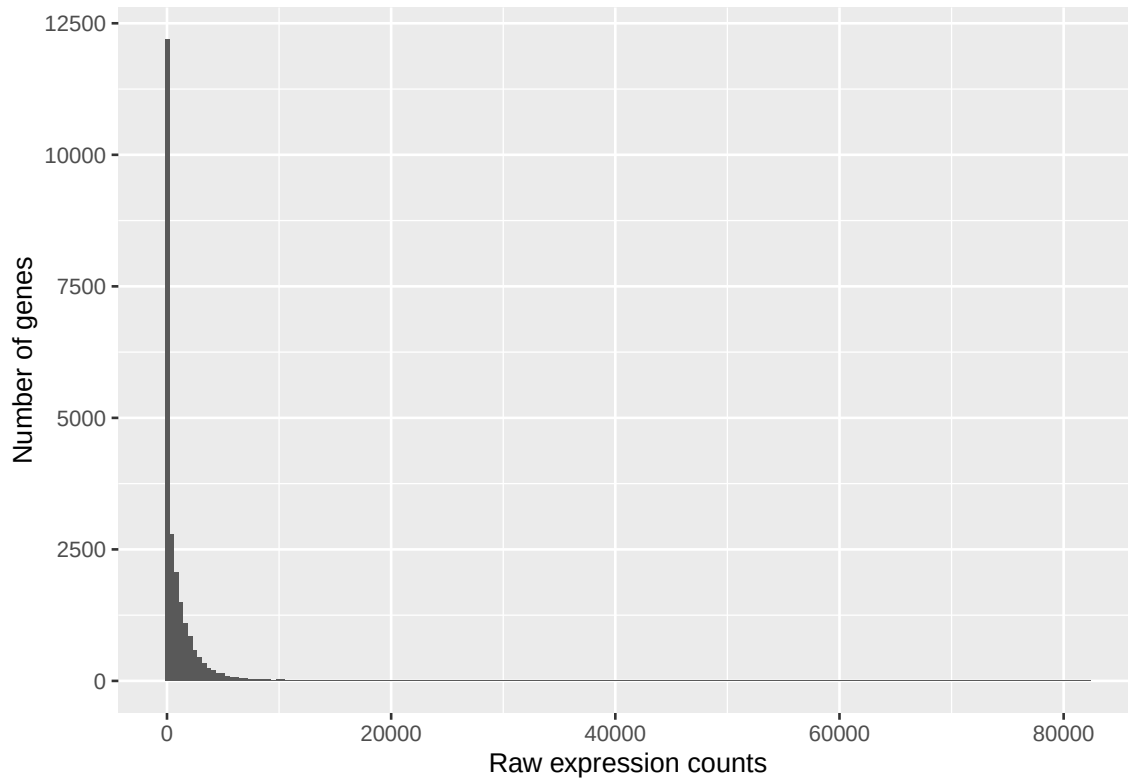


The goal of differential expression analysis is to determine, for each gene, whether the differences in expression (counts) **between groups** is significant given the amount of variation observed **within groups** (replicates). To test for significance, we need an appropriate statistical model that accurately performs normalization (to account for differences in sequencing depth, etc.) and variance modeling (to account for few numbers of replicates and large dynamic expression range).

1.9.1 RNA-seq count distribution

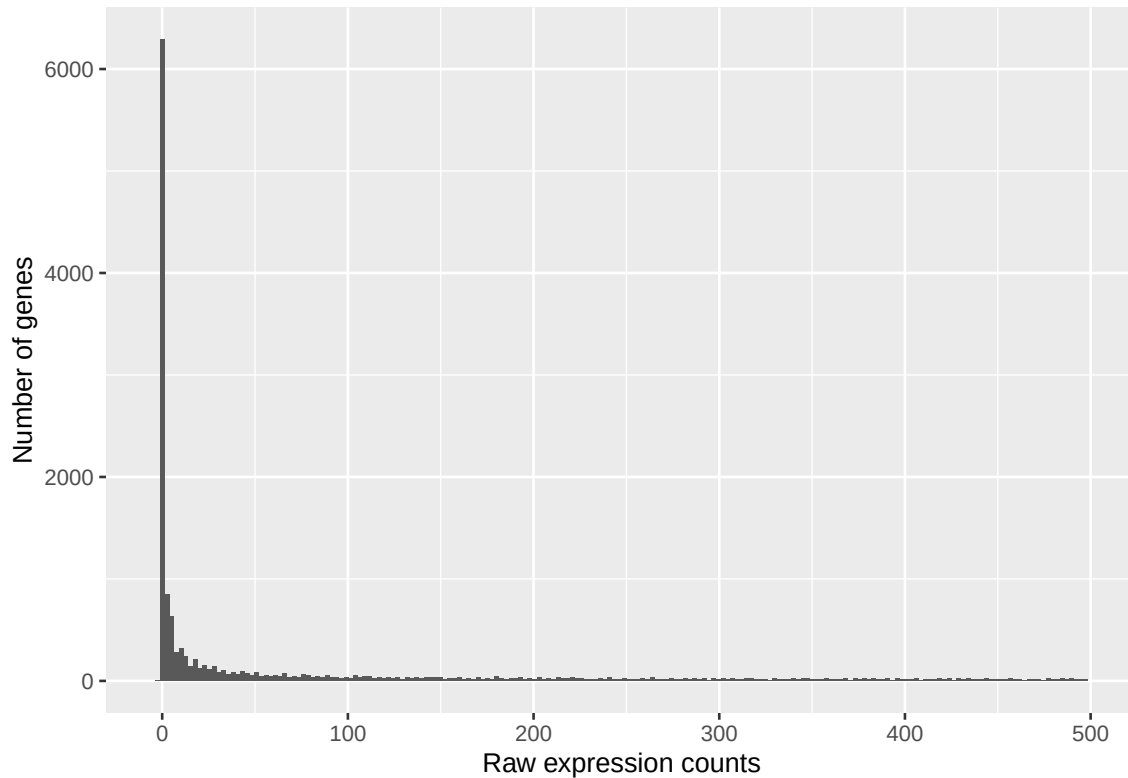
To determine the appropriate statistical model, we need information about the distribution of counts. To get an idea about how RNA-seq counts are distributed, let's plot the counts for a single sample, 'Mov10_oe_1':

```
ggplot(data) +
  geom_histogram(aes(x = Mov10_oe_1), stat = "bin", bins = 200) +
  xlab("Raw expression counts") +
  ylab("Number of genes")
```



If we zoom in close to zero, we can see a large number of genes with counts of zero:

```
ggplot(data) +  
  geom_histogram(aes(x = Mov10_oe_1), stat = "bin", bins = 200) +  
  xlim(-5, 500) +  
  xlab("Raw expression counts") +  
  ylab("Number of genes")
```



These images illustrate some common features of RNA-seq count data, including a **low number of counts associated with a large proportion of genes**, and a long right tail due to the **lack of any upper limit for expression**.

1.9.2 Modeling count data

Count data is often modeled using the **binomial distribution**, which models the probability of success/failure outcomes over fixed trials (like counting heads in 10 coin tosses). **When dealing with very large numbers of cases but very small probabilities of events (like lottery tickets), the Poisson distribution is more appropriate.**

Poisson Distribution Formula

$$P(X = x) = \frac{\lambda^x e^{-\lambda}}{x!}$$

where

$x = 0, 1, 2, 3, \dots$

λ = mean number of occurrences in the interval

e = Euler's constant ≈ 2.71828

Where:

- λ = the average number of “successes”
- x = the specific number you’re asking about
- $P(X = x)$ = probability of getting exactly that many events

With RNA-seq data, **we have millions of reads sequenced, but the probability of any single read mapping to a specific gene is extremely low**. Thus, it would be an appropriate situation to use the Poisson distribution. However, a unique property of this distribution is that the mean == variance. Realistically, with RNA-seq data there is always some biological variation present across the replicates (within a sample class). Genes with larger average expression levels will tend to have larger observed variances across replicates.

To test if our data meets this Poisson criteria, we'll plot **mean versus variance** for the **Mov10 overexpression** replicates using the `apply` function:

To calculate the mean and variance of our data, we will use the `apply` function. The `apply` function allows you to apply a function to the margins (rows or columns) of a matrix. The syntax for `apply` is as follows:

```
apply(X, MARGIN, FUN)
```

where `X` is a matrix, `MARGIN` is the margin of the matrix to apply the function to (1 = rows, 2 = columns), and `FUN` is the function to apply.

We will use `MARGIN = 1` to apply the mean and variance of the counts for each row (gene) across the **Mov10 overexpression** replicates. We will then create a data frame with the mean and variance of the counts for each gene.

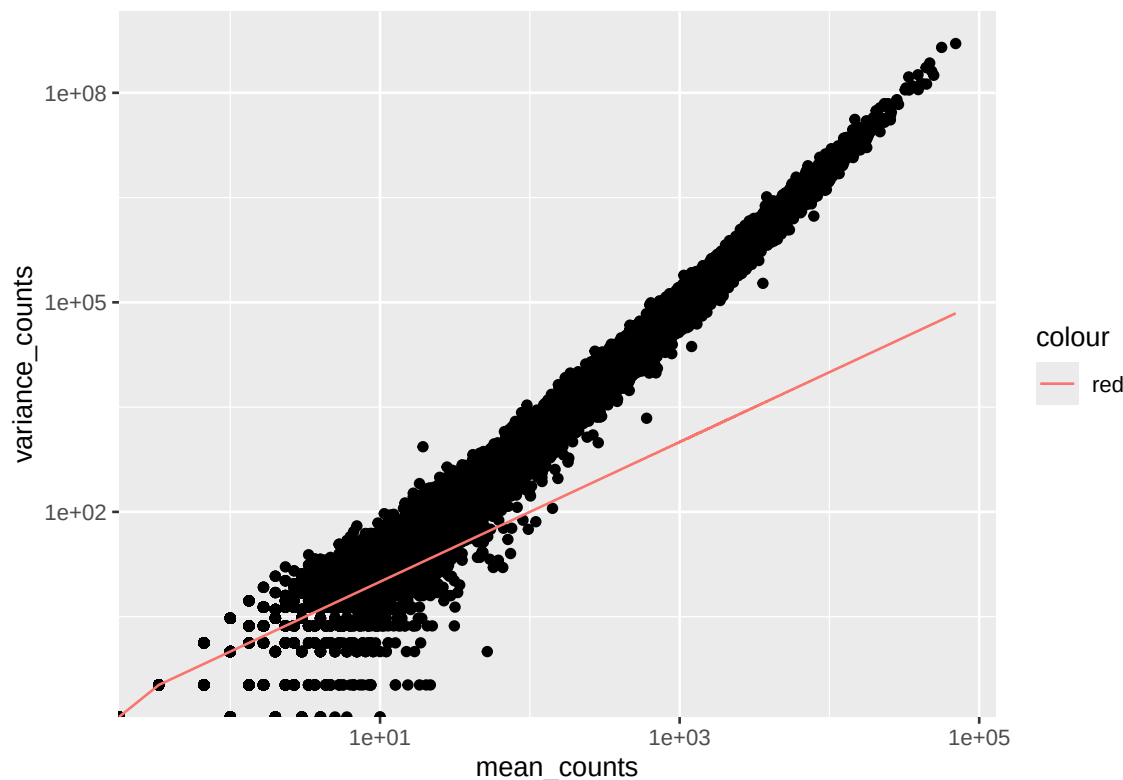
```
# calculate the mean and variance of the counts for each gene
mean_counts <- apply(data[, 3:5], 1, mean)

variance_counts <- apply(data[, 3:5], 1, var)

# for ggplot we need the data to be in a data.frame
df <- data.frame(mean_counts, variance_counts)
```

Run the following code to plot the *mean versus variance* for the 'Mov10 overexpression' replicates:

```
ggplot(df) +
  geom_point(aes(x = mean_counts, y = variance_counts)) +
  geom_line(aes(x = mean_counts, y = mean_counts, color="red")) +
  scale_y_log10() + scale_x_log10()
```



By plotting the *mean versus the variance* of our data we can easily see that the **variance** > **mean** violating the Poisson assumption. Higher mean genes have even higher variance, indicating we need a different model.

The model that fits best, given this type of variability between replicates, is the **Negative Binomial (NB) model**. Essentially, the **NB model** is a good approximation for data where the **variance** > **mean**, as is the case with RNA-Seq count data.

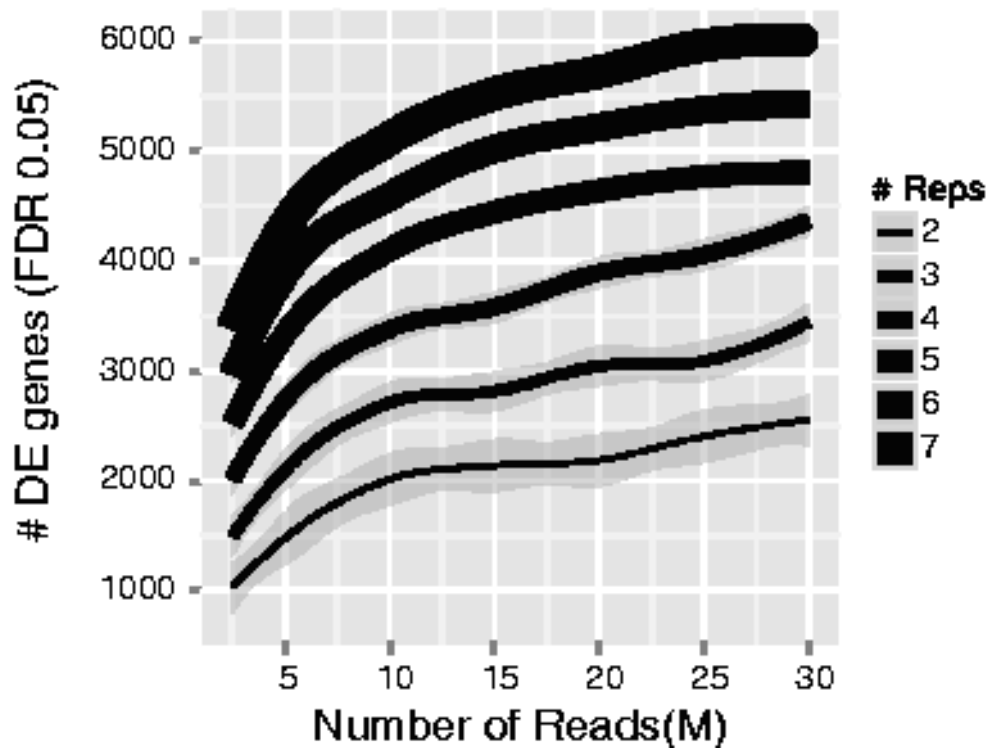
1.9.3 Improving mean estimates (i.e. reducing variance) with biological replicates

Remember, the goal of differential expression analysis is to determine, for each gene, whether the differences in expression (counts) **between groups** is significant given the amount of variation observed **within groups** (replicates). More biological replicates reduce variance and improve mean estimates. Additional replicates provide increasingly precise group means and greater power to detect differences between conditions.

The figure below shows that **increasing replicates identifies more DE genes than increasing sequencing depth** because biological variability between samples is the main source of noise, not technical measurement precision.

Key insight: You can sequence one sample infinitely deep, but you still have $n=1$ for statistics. DESeq2 requires biological replicates to estimate variance and perform statistical tests.

Recommendation: Minimum 20-30M reads per sample, but prioritize replicates over depth.



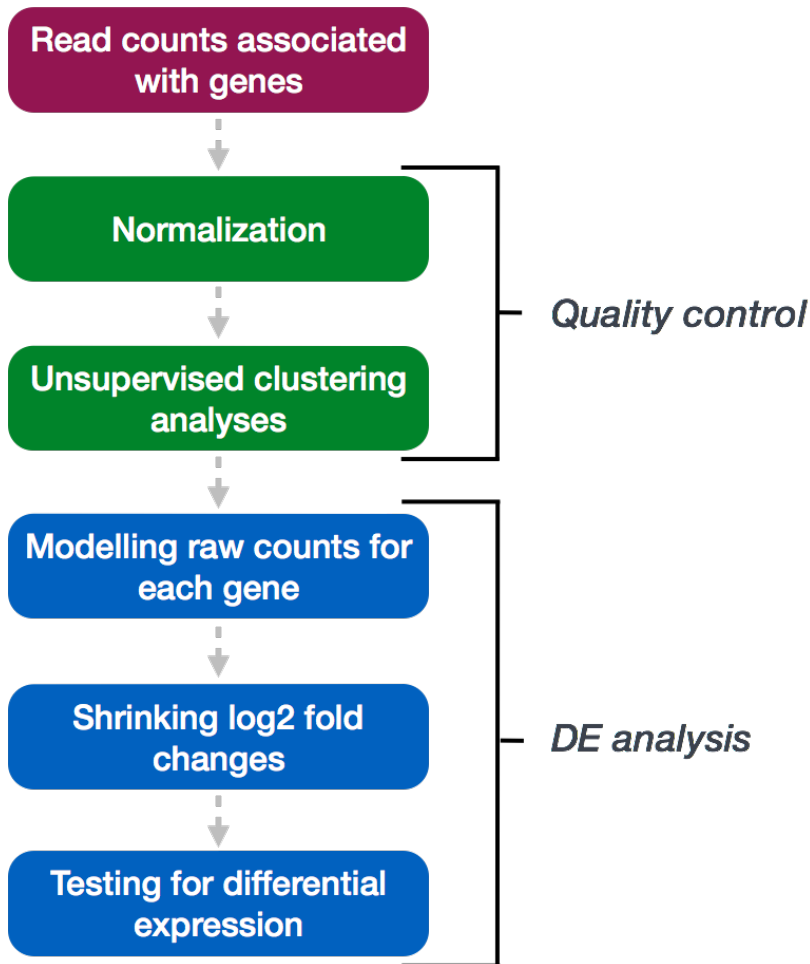
NOTE

- **Biological replicates** = multiple samples (different mice)
- **Technical replicates** = same sample (same mouse) with technical steps replicated
- **Don't spend money on technical replicates - biological replicates are much more useful**

1.9.4 Differential expression analysis workflow

To model counts appropriately when performing a differential expression analysis, a few tools are generally recommended as best practice, e.g. **DESeq2** and **EdgeR**. Both these tools use the negative binomial model, employ similar methods, and typically, yield similar results. They are pretty stringent, and have a good balance between sensitivity and specificity (reducing both false positives and false negatives).

For this tutorial, we will be using **DESeq2** for the DE analysis, and the analysis steps with **DESeq2** are shown in the flowchart below. DESeq2 first normalizes the count data to account for differences in library sizes and RNA composition between samples. Then, we will use the normalized counts to make some plots for QC at the gene and sample level. The final step is to use the appropriate functions from the DESeq2 package to perform the differential expression analysis.



We will go in-depth into each of these steps in the following lessons, but additional details and helpful suggestions regarding DESeq2 can be found in the DESeq2 vignette. As you go through this workflow and questions arise, you can reference the vignette from within RStudio:

```
vignette("DESeq2")
```

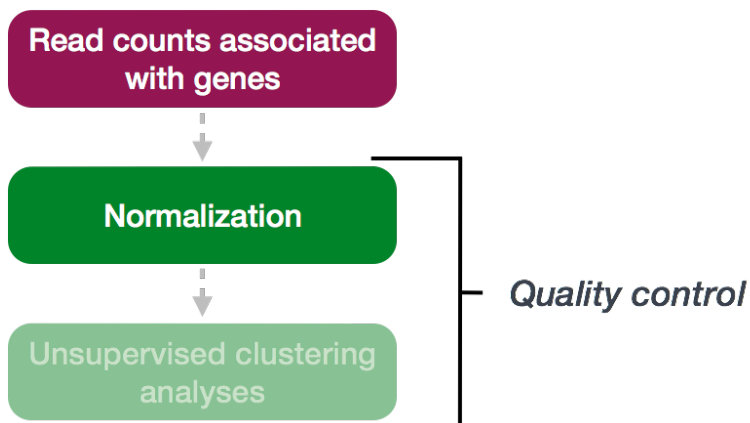
This is very convenient, as it provides a wealth of information at your fingertips! Be sure to use this as you need during the workshop.

Chapter 2

Count normalization

2.1 Normalization

The first step in the DE analysis workflow is count normalization, which is necessary to make accurate comparisons of gene expression between samples.

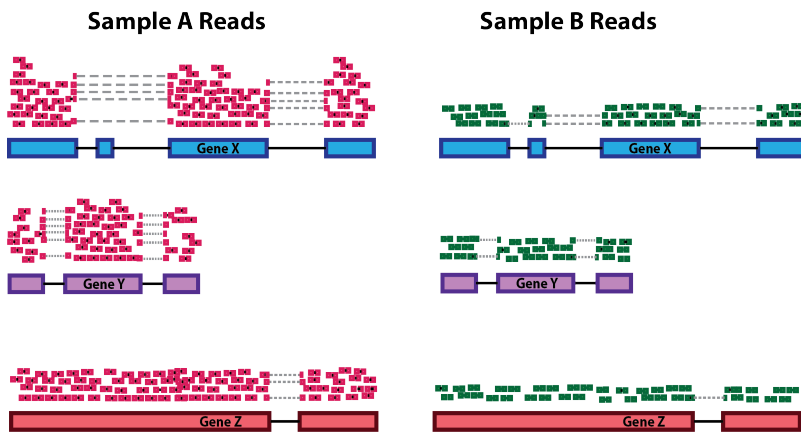


The counts of mapped reads for each gene is proportional to the expression of RNA (“interesting”) in addition to many other factors (“uninteresting”). Normalization is the process of scaling raw count values to account for the “uninteresting” factors. In this way the expression levels are more comparable between and/or within samples.

The main factors often considered during normalization are:

- **Sequencing depth:**

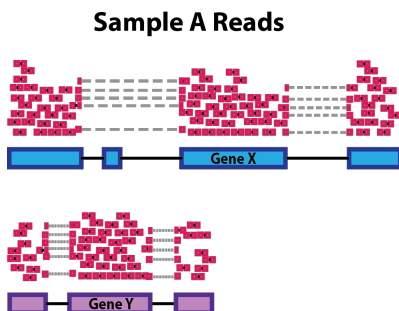
Accounting for sequencing depth is necessary for comparison of gene expression between samples. In the example below, each gene appears to have doubled in expression in *Sample A* relative to *Sample B*, however this is a consequence of *Sample A* having double the sequencing depth.



NOTE: In the figure above, each pink and green rectangle represents a read aligned to a gene. Reads connected by dashed lines connect a read spanning an intron.*

- **Gene length:**

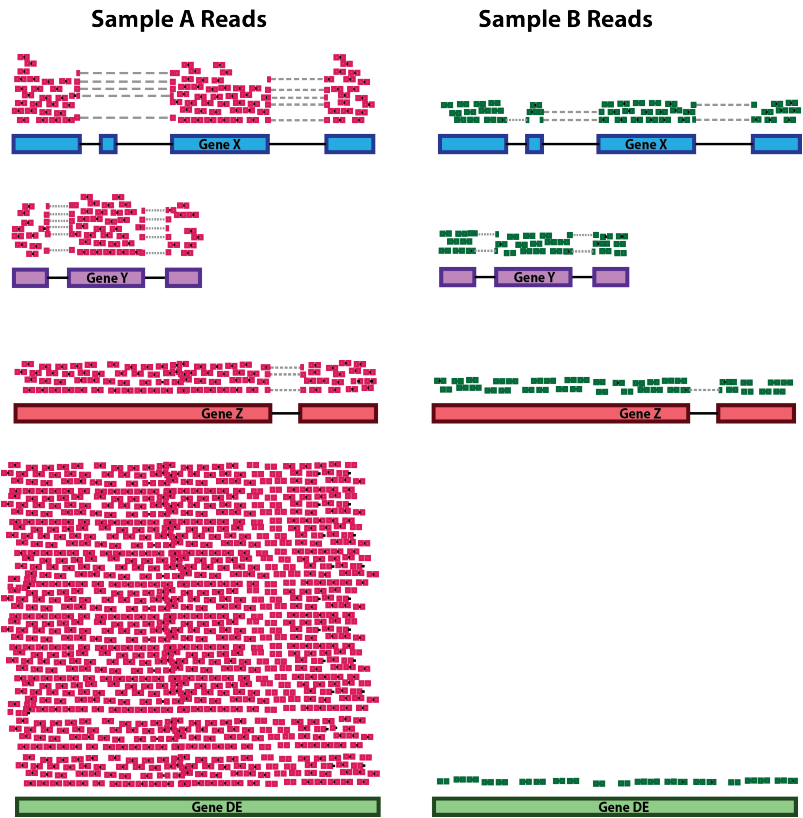
Accounting for gene length is necessary for comparing expression between different genes within the same sample. In the example, *Gene X* and *Gene Y* have similar levels of expression, but the number of reads mapped to *Gene X* would be many more than the number mapped to *Gene Y* because *Gene X* is longer.



- **RNA composition:**

RNA composition bias arises when a few highly expressed genes dominate the overall expression profile of a sample. Accounting for RNA composition is recommended for accurate comparison of expression between samples, and is particularly important when performing differential expression analyses [1]

In the example, if we were to divide each sample by the total number of counts to normalize, the counts would be greatly skewed by the DE gene, which takes up most of the counts for *Sample A*, but not *Sample B*. Most other genes for *Sample A* would be divided by the larger number of total counts and appear to be less expressed than those same genes in *Sample B*.



While normalization is essential for differential expression analyses, it is also necessary for exploratory data analysis, visualization of data, and whenever you are exploring or comparing counts between or within samples.

2.1.1 Common normalization methods

Several common normalization methods exist to account for these differences:

Normalization method	Description	Accounted factors	Recommendations for use
TPM (transcripts per kilobase million)	counts per length of transcript (kb) per million reads mapped	sequencing depth and gene length	gene count comparisons within a sample or between samples of the same sample group; NOT for DE analysis
RPKM/FPKM (reads/fragments per kilobase of exon per million reads/fragments mapped)	similar to TPM	sequencing depth and gene length	gene count comparisons between genes within a sample; NOT for between sample comparisons or DE analysis
DESeq2's median of ratios [1]	counts divided by sample-specific size factors determined by median ratio of gene counts relative to geometric mean per gene	sequencing depth and RNA composition	gene count comparisons between samples and for DE analysis ; NOT for within sample comparisons

Normalization method	Description	Accounted factors	Recommendations for use
EdgeR's trimmed mean of M values (TMM) [2]	uses a weighted trimmed mean of the log expression ratios between samples	sequencing depth, RNA composition	gene count comparisons between samples and for DE analysis ; NOT for within sample comparisons

Conclusion: Tools like DESeq2 or edgeR are better suited for differential expression analysis, as they:

- Use raw counts for more accurate modeling.
- Apply statistical modeling (e.g., Negative Binomial) to account for variance across replicates.

2.1.2 DESeq2 Normalization: Median of Ratios Method

For differential expression analysis, gene length need not be factored in; however, **sequencing depth** and **RNA composition** must be considered. The median of ratios method neutralizes these differences.

This method involves several steps, even though from the user-end it appears as a single function call in DESeq2.

DESeq2 Normalization: Median of Ratios Method DESeq2 normalizes for sequencing depth and RNA composition using the median of ratios method.

Step 1: Create Pseudo-reference Sample (Row-wise Geometric Mean) Calculate the geometric mean for each gene across all samples to create a reference.

Step 2: Calculate the Ratio of Each Sample to the Reference For each sample, divide each gene count by its geometric mean reference value.

Step 3: Calculate the Normalization Factor for Each Sample (Size Factor) Take the median of all ratios for each sample - this becomes the normalization factor.

Step 4: Calculate the Normalized Count Values Using the Normalization Factor Divide raw counts by the sample's size factor to get normalized counts.

Why this works:

- Robust: Uses median (less sensitive to outliers)
- Assumes most genes aren't differentially expressed
- Accounts for library size and composition differences
- Geometric mean reduces impact of extreme values

The method efficiently corrects technical variation while preserving biological differences, enabling accurate differential expression analysis.

Let's go through an example:

Step 1: Create a Pseudo-reference Sample (Row-wise Geometric Mean)

The **geometric mean** of a set of n positive numbers $x_1, x_2, \dots, x_n$ is defined as the n th root of the product of the n numbers:

$$\text{Geometric Mean} = \left(\prod_{i=1}^n x_i \right)^{\frac{1}{n}}$$

For each gene, a pseudo-reference sample is created that is equal to the geometric mean across all samples, reducing the impact of extreme values.

Gene	Sample A	Sample B	Pseudo-reference Sample
EF2A	1489	906	$\text{sqrt}(1489 * 906) = \mathbf{1161.5}$
ABCD1	22	13	$\text{sqrt}(22 * 13) = \mathbf{17.7}$
...	...	...	...

Step 2: Calculate the Ratio of Each Sample to the Reference

For every gene in a sample, calculate the ratio of the sample count to the reference (sample/ref). This is performed for each sample in the dataset, resulting in ratios for most genes that are similar, as depicted below:

Gene	Sample A	Sample B	Pseudo-reference Sample	Ratio of Sample A/Ref	Ratio of Sample B/Ref
EF2A	1489	906	1161.5	$1489/1161.5 = \mathbf{1.28}$	$906/1161.5 = \mathbf{0.78}$
ABCD1	22	13	17.7	$22/17.7 = \mathbf{1.24}$	$13/17.7 = \mathbf{0.73}$
MEFV	793	410	570.2	$793/570.2 = \mathbf{1.39}$	$410/570.2 = \mathbf{0.72}$
...	...	...	...	...	...

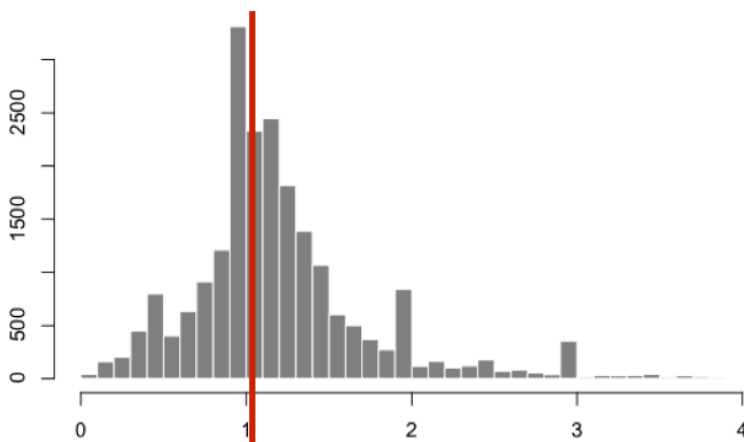
Step 3: Calculate the Normalization Factor for Each Sample (Size Factor)

The median value of all ratios for a given sample is calculated as the normalization factor (size factor):

```
normalization_factor_sampleA <- median(c(1.28, 1.24, 1.39, 1.35, 0.59))
normalization_factor_sampleB <- median(c(0.78, 0.73, 0.72, 0.74, 1.35))
```

The figure below illustrates the median value for the distribution of all gene ratios for a single sample (frequency is on the y-axis).

sample 1 / pseudo-reference sample



The median of ratios method accounts for both sequencing depth and composition bias.

Sequencing Depth:

Problem: Sample A might have 20 million reads while Sample B has 40 million reads

Solution: When you calculate ratios (Sample/Pseudo-reference), genes in the deeper-sequenced sample will have consistently higher ratios across most genes

Result: The median ratio for Sample A might be 0.5, for Sample B might be 2.0

Normalization: Dividing each sample's counts by its median ratio equalizes sequencing depth

RNA Composition Bias:

Problem: Sample A might have a few highly expressed genes (like ribosomal genes) that “steal” sequencing reads from other genes

Example: If 50% of reads in Sample A come from just 10 highly expressed genes, all other genes appear artificially low

Solution: These outlier genes create extreme ratios, but the median is robust to outliers

Result: The median ratio reflects the typical gene's behavior, not the extreme outliers

This method is robust as it assumes that not all genes are differentially expressed, allowing normalization factors to account for both sequencing depth and RNA composition efficiently.

The key insight: By using the median rather than the mean, the method ignores extreme outliers that cause composition bias while still correcting for overall sequencing depth differences. The median represents what a “typical” gene experienced, not what the most extreme genes did.

Step 4: Calculate the Normalized Count Values Using the Normalization Factor

Each raw count value in a given sample is divided by that sample's normalization factor to generate normalized count values across all genes:

Gene	Sample A	Sample B
EF2A	1489 / 1.3 = 1145.39	906 / 0.77 = 1176.62
ABCD1	22 / 1.3 = 16.92	13 / 0.77 = 16.88
...	...	...

Note that normalized count values may not always be whole numbers.

Exercise points = +1

Determine the normalized counts for your gene of interest, PD1, given the raw counts and size factors below.

NOTE: You will need to run the code below to generate the raw counts dataframe (PD1) and the size factor vector (size_factors), then use these objects to determine the normalized counts values:

```
# Raw counts for PD1
PD1 <- c(21, 58, 17, 97, 83, 10)
names(PD1) <- paste0("Sample", 1:6)
PD1 <- data.frame(PD1)
```

```
PD1 <- t(PD1)

# Size factors for each sample
size_factors <- c(1.32, 0.70, 1.04, 1.27, 1.11, 0.85)
```

2.2 Count normalization of Mov10

Now that we know the theory of count normalization, we will normalize the counts for the Mov10 dataset using DESeq2. This requires a few steps:

1. Ensure the row names of the metadata dataframe are present and in the same order as the column names of the counts dataframe.
2. Create a DESeqDataSet object
3. Generate the normalized counts

2.2.1 1. Match the metadata and counts data

We should always make sure that we have sample names that match between the two files, and that the samples are in the right order. DESeq2 will output an error if this is not the case.

```
### Check that sample names match in both files
all(colnames(data) %in% rownames(meta))
```

2.3 [1] TRUE

```
all(colnames(data) == rownames(meta))
```

2.4 [1] FALSE

The colnames of our data don't match the rownames of our metadata so we need to reorder them. We can use the `match` function:

```
idx <- match(rownames(meta), colnames(data))
data <- data[,idx]

all(colnames(data) == rownames(meta))
```

2.5 [1] TRUE

Exercise points = +2

Suppose we had the same sample names in the `colnames` of the counts matrix and the `rownames` of the metadata file, but they were in a different order. Write the line(s) of code required to create a new matrix with columns ordered such that they were identical to the row names of the metadata.

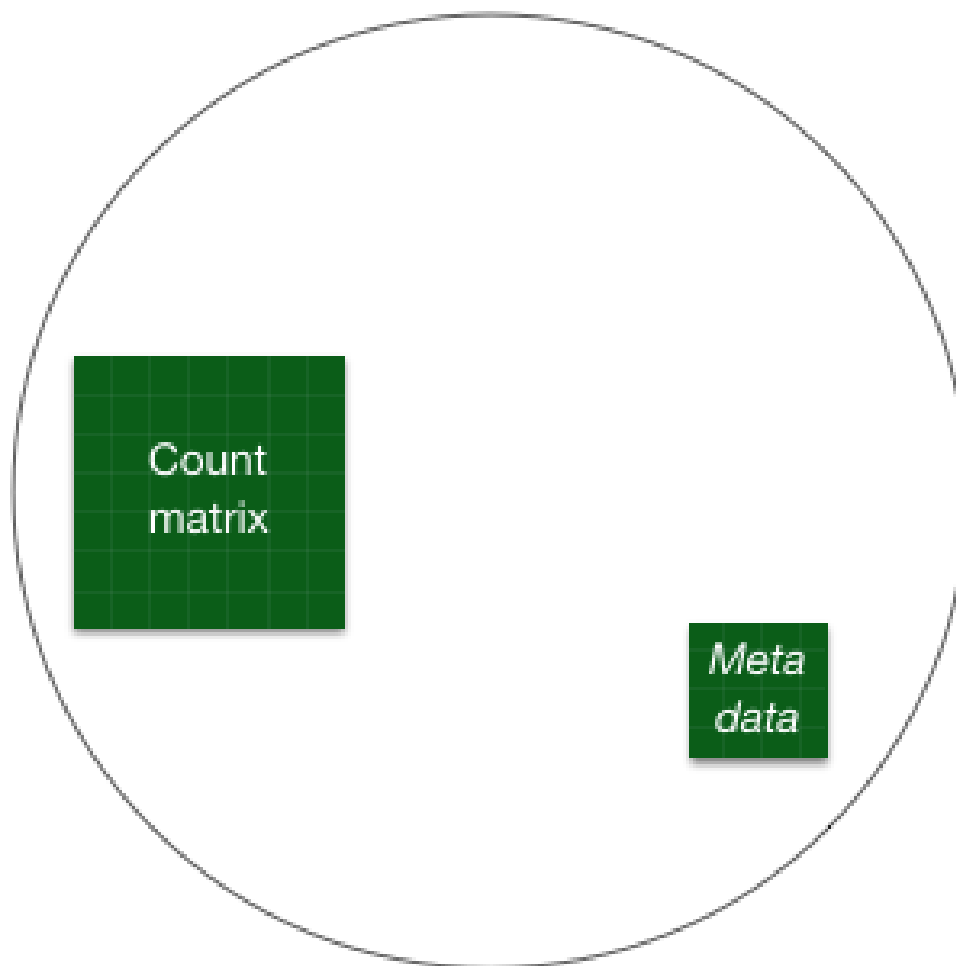
2.5.1 2. Create DESeq2 object

Bioconductor software packages often define and use a custom class within R for storing data (input data, intermediate data and also result data). These custom data structures are similar to `lists` in that they can contain multiple different data types/structures within them. But, unlike lists they have pre-specified `data slots`, which hold specific types/classes of data. The data stored in these pre-specified slots can be accessed by using specific package-defined functions.

Let's start by creating the `DESeqDataSet` object and then we can talk a bit more about what is stored inside it. To create the object we will need the `count matrix` and the `metadata` table as input. We will also need to specify a `design formula`. The design formula specifies the column(s) in the metadata table and how they should be used in the analysis. For our dataset we only have one column we are interested in, that is `~sampletype`. This column has three factor levels, which tells DESeq2 that for each gene we want to evaluate gene expression change with respect to these different levels.

```
## Create DESeq2Dataset object
```

```
dds <- DESeqDataSetFromMatrix(countData = data, colData = meta, design = ~ sampletype)
```



You can use DESeq-specific functions to access the different slots and retrieve information, if you wish. For example, suppose we wanted the original count matrix we would use `counts()`:

```
head(counts(dds[,1:5]))
```

```
2.6  Irrel_kd_1 Irrel_kd_2 Irrel_kd_3 Mov10_kd_2 Mov10_kd_3
```

```
2.7  1/2-SBSRNA4 45 31 39 57 41
```

```
2.8  A1BG 77 58 40 71 40
```

```
2.9  A1BG-AS1 213 172 126 256 177
```

```
2.10 A1CF 0 0 0 0 1
```

```
2.11 A2LD1 91 80 50 146 81
```

```
2.12 A2M 9 8 4 10 9
```

As we go through the workflow we will use the relevant functions to check what information gets stored inside our object. We can also run:

```
slotNames(dds)
```

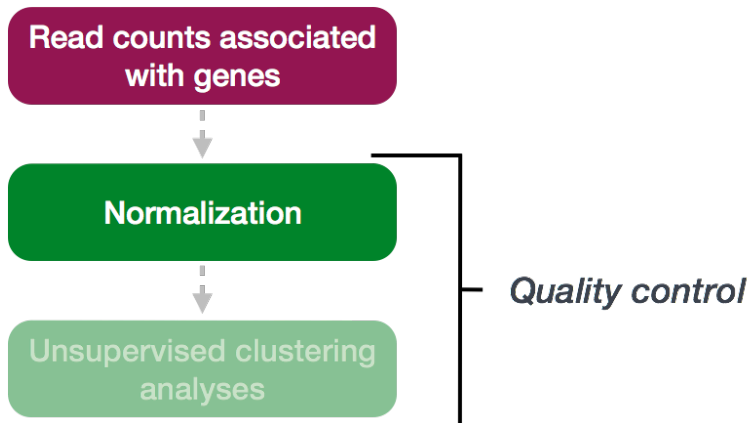
```
2.13 [1] "design" "dispersionFunction" "rowRanges"
```

```
2.14 [4] "colData" "assays" "NAMES"
```

```
2.15 [7] "elementMetadata" "metadata"
```

2.15.1 3. Generate the Mov10 normalized counts

The next step is to normalize the count data in order to be able to make fair gene comparisons between samples.



To perform the **median of ratios method** of normalization, DESeq2 has a single `estimateSizeFactors()` function that will generate size factors for us. We will use the function in the example below, but **in a typical RNA-seq analysis this step is automatically performed by the `DESeq()` function**, which we will see later.

```
dds <- estimateSizeFactors(dds)
```

By assigning the results back to the `dds` object we are filling in the slots of the `DESeqDataSet` object with the appropriate information. We can take a look at the normalization factor applied to each sample using:

```
sizeFactors(dds)
```

```
2.16  Irrel_kd_1  Irrel_kd_2  Irrel_kd_3  Mov10_kd_2  Mov10_kd_3
      Mov10_oe_1 Mov10_oe_2
```

```
2.17  1.1224020 0.9625632 0.7477715 1.5646728 0.9351760 1.2016082 1.1205912
```

```
2.18  Mov10_oe_3
```

```
2.19  0.6534987
```

Now, to retrieve the normalized counts matrix from `dds`, we use the `counts()` function and add the argument `normalized=TRUE`.

```
normalized_counts <- counts(dds, normalized=TRUE)
```

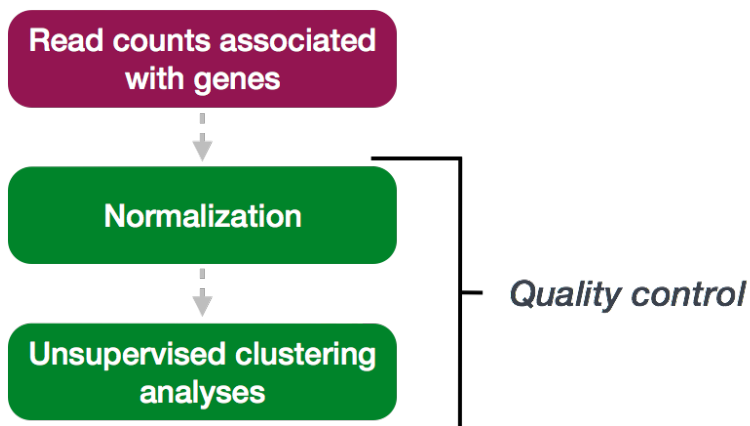
We can save this normalized data matrix to file for later use:

```
write.table(normalized_counts, file="data/normalized_counts.txt", sep="\t", quote=F, col.names=NA)
```

Chapter 3

Quality Control (QC)

The next step in the DESeq2 workflow is QC, which includes sample-level and gene-level steps to perform QC checks on the count data to help us ensure that the samples/replicates look good.

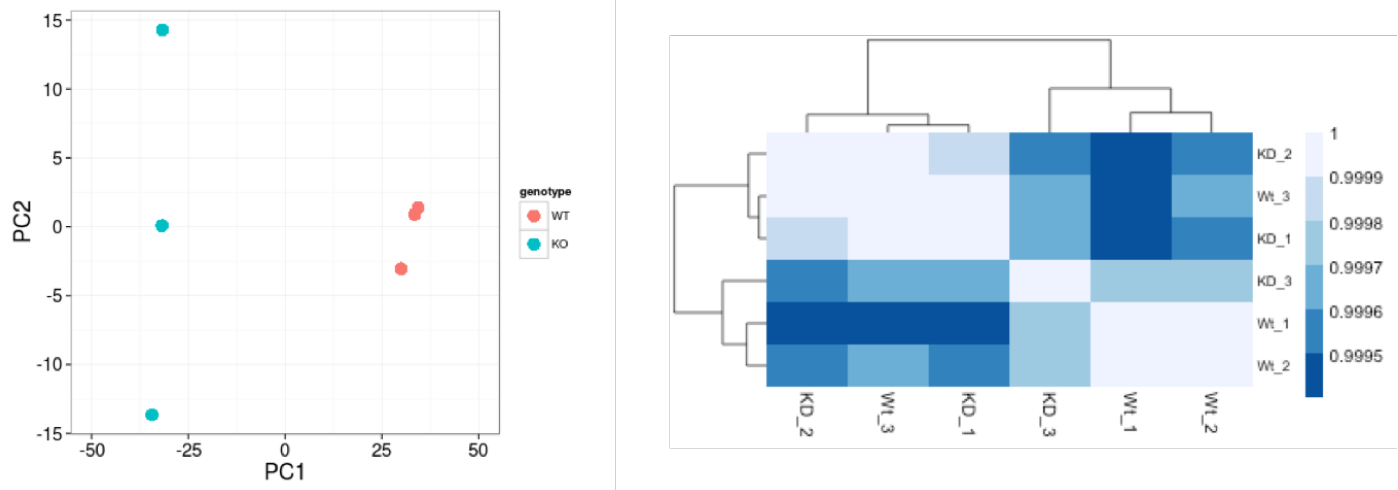


3.1 Sample-level QC

A useful initial step in an RNA-seq analysis is often to assess overall similarity between samples:

- Which samples are similar to each other, which are different?
- Does this fit to the expectation from the experiment's design?
- What are the major sources of variation in the dataset?

To explore the similarity of our samples, we will be performing sample-level QC using Principal Component Analysis (PCA) and hierarchical clustering methods. Our sample-level QC allows us to see how well our replicates cluster together, as well as, observe whether our experimental condition represents the major source of variation in the data. Performing sample-level QC can also identify any sample outliers, which may need to be explored further to determine whether they need to be removed prior to DE analysis.



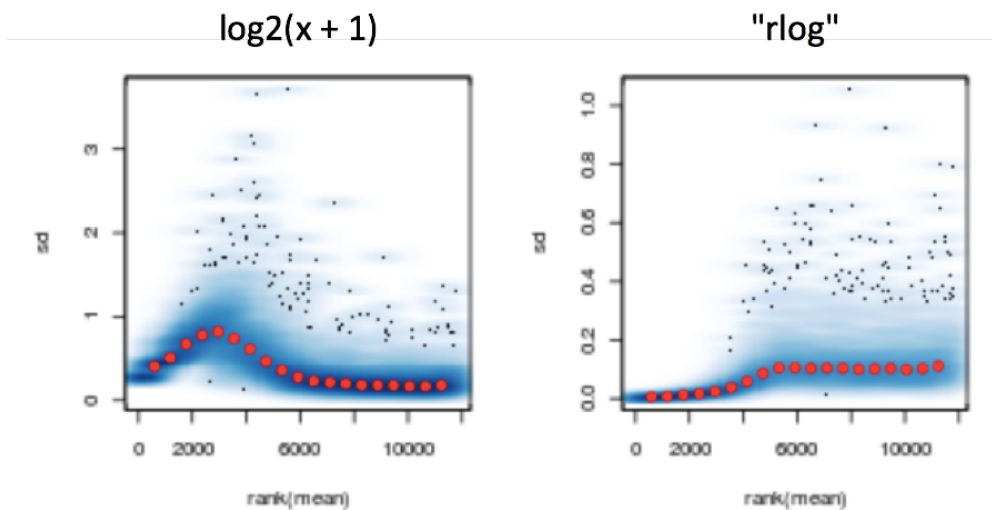
When using these unsupervised clustering methods, DESeq2 uses a regularized log transform (rlog) of the normalized counts for sample-level QC as it moderates the variance across the mean, improving the clustering.

What it does: - Converts counts to log2 scale (makes highly and lowly expressed genes more comparable)

- Stabilizes variance across expression levels (prevents highly variable genes from dominating)
- Results in data where all genes contribute more equally to PCA and clustering

Why this matters: Without transformation, highly expressed genes would dominate the PCA and clustering simply because they have larger count values, masking biologically meaningful patterns from other genes.

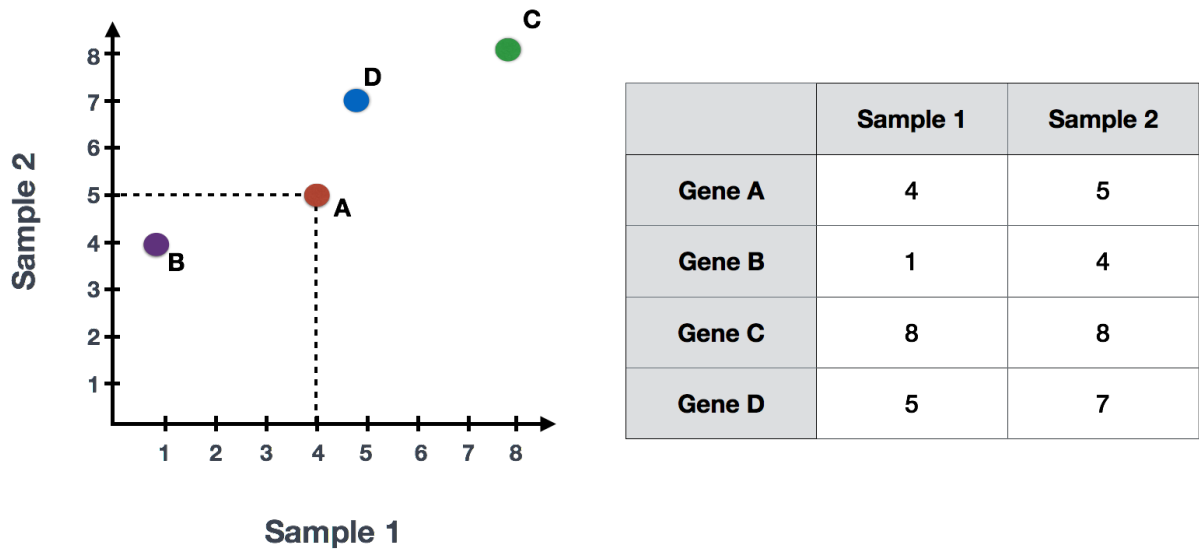
When to use it: Use rlog-transformed data for sample-level quality control like PCA plots, heatmaps, and sample clustering. Don't use it for differential expression analysis - that uses the original count data.



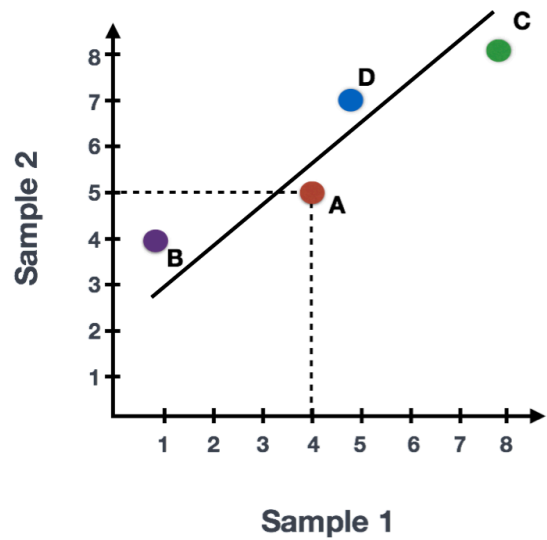
3.1.1 Principal Component Analysis

Principal Component Analysis (PCA) is a technique used to emphasize variation and bring out strong patterns in a dataset (dimensionality reduction). Details regarding PCA are given below (based on materials from StatQuest, and if you would like a more thorough description, we encourage you to explore StatQuest’s video.

Suppose we had a dataset with two samples and four genes. Based on this expression data we want to evaluate the relationship between these samples. We could plot the counts of one sample versus another, with Sample 1 on the x-axis and Sample 2 on the y-axis as shown below:



For PCA analysis, the first step is finding the **first principal component** (PC1), the direction through your data that shows the most variation. In this example, the most variation is along the diagonal - that’s where the data is most spread out – capturing the longest possible distance through this set of data points. The genes at the ends of this line (Gene B and Gene C) contribute most to the variability because they have the largest range of expression values across the samples.



After drawing this line and establishing the amount of contribution per gene, **PCA will compute a per sample score**. The per sample PC1 score is computed by taking the product of each gene's loading (its contribution to PC1) and its normalized expression in the sample, summed across all genes. We could draw another line through the data representing the second most amount of variation in the data (PC2) and compute scores, followed by a third line and so on until you hit the total number of samples in your dataset.

The formula to calculate a sample's PC1 score is as follows:

$$\text{Sample1 PC1 score} = (\text{expr}_{\text{GeneA}} \times \text{loading}_{\text{GeneA}}) + (\text{expr}_{\text{GeneB}} \times \text{loading}_{\text{GeneB}}) + \dots + (\text{expr}_{\text{GeneN}} \times \text{loading}_{\text{GeneN}})$$

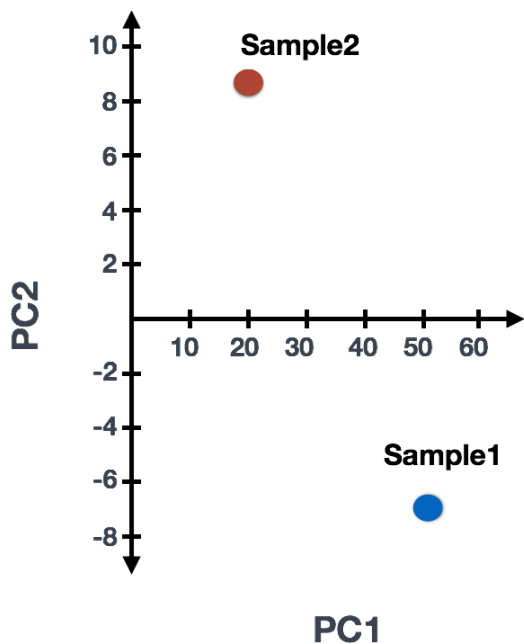
More compact notation:

$$\text{PC1}_j = \sum_{i=1}^n x_{ij} \cdot w_{i1}$$

Where:

- PC1_j is the PC1 score for sample j
- x_{ij} is the normalized expression value of gene i in sample j
- w_{i1} is the PC1 loading (weight) for gene i
- n = the total number of genes in the dataset
- i = index for genes (goes from 1 to n)
- j = index for samples

In reality, your dataset will have larger dimensions (more samples, and many, many more genes). The initial sample-to-sample plot, will therefore be in n -dimensional space with n axes representing the total number of samples you have. The end result is a 2-dimensional matrix with rows representing samples and columns reflecting scores for each of the principal components. To evaluate the results of a PCA, we usually plot principal components against each other, starting with PCs that explain the most amount of variation in your data.



	PC1	PC2
Sample1	51	-7
Sample2	21	8.5

If two samples have similar levels of expression for the genes that contribute significantly to the variation represented by PC1, they will be plotted close together on the PC1 axis. Therefore, we would expect that biological replicates to have similar scores (since the same genes are changing) and cluster together on PC1 and/or PC2, and the samples from different treatment groups to have different scores.

3.1.1.1 Interpreting PCA plots

In real datasets with many samples and genes, PCA reduces the complex, high-dimensional data into a 2-dimensional space, where each sample is represented by its scores for the principal components. Typically, we plot the first two principal components (PC1 and PC2), as they explain the most variation in the data.

When interpreting PCA plots, **biological replicates** (samples from the same condition) should cluster together because they have similar expression patterns for the genes driving variation in PC1. **Samples from different treatment groups** will often separate along these axes, reflecting differences in their gene expression profiles.

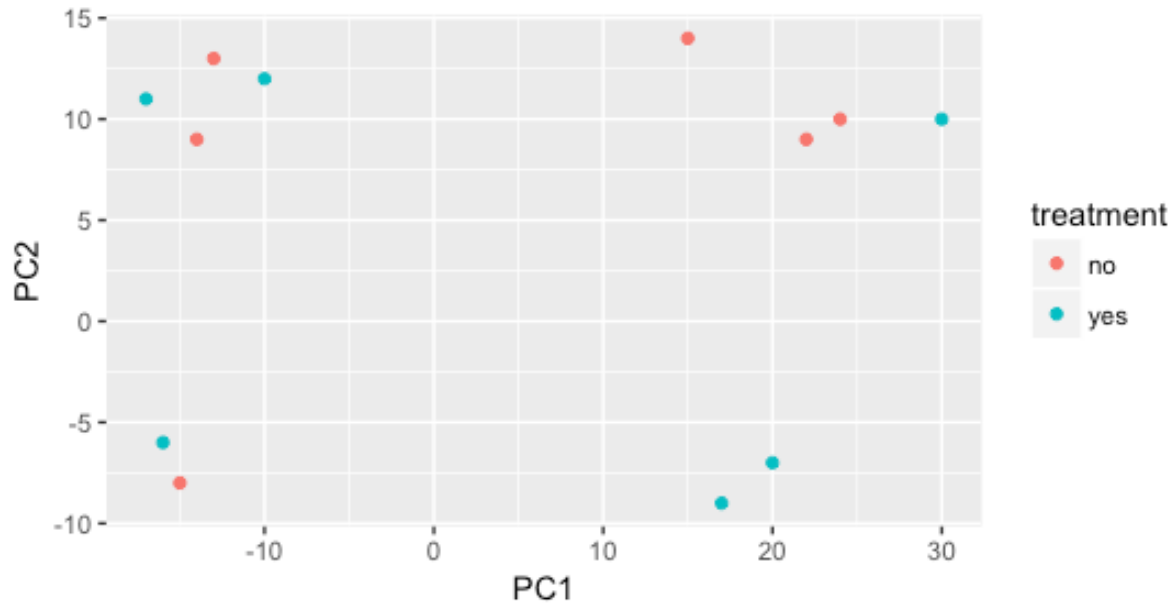
This is best understood by looking at example PCA plots, where you can visualize how biological replicates cluster and treatment groups separate based on their gene expression patterns.

3.1.1.2 Interpreting PCA plots example

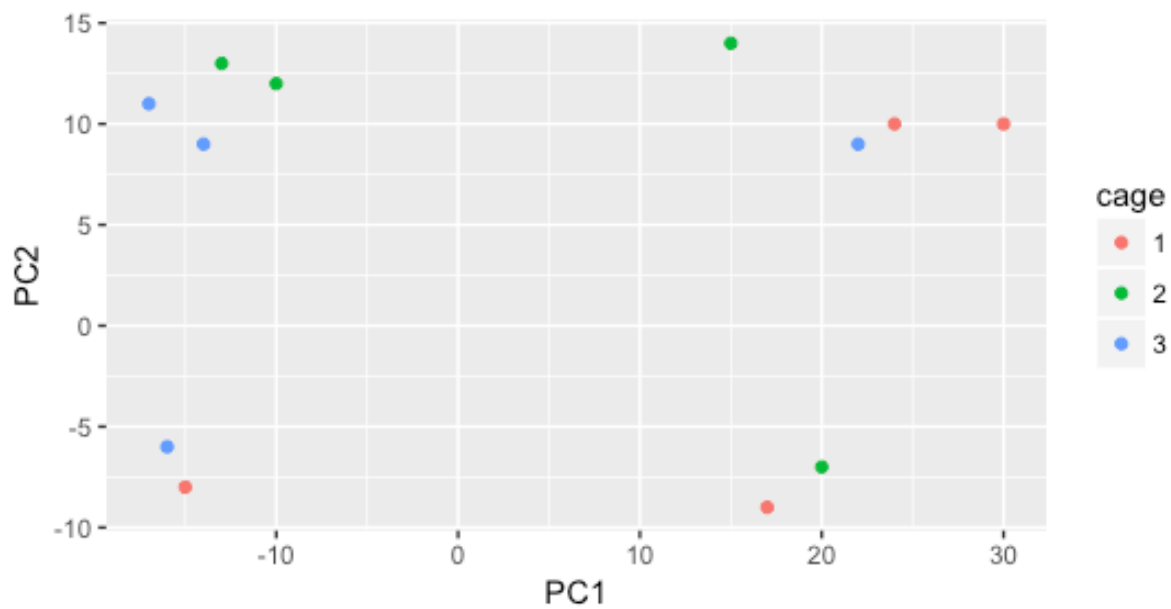
We have an example dataset and a few associated PCA plots below to get a feel for how to interpret them. The metadata for the experiment is displayed below. The main condition of interest is **treatment**.

sample	strain	date	cage	treatment	replicate	sex
B1	BALB/cJ	20180515	1	yes	1	M
B2	C57BL/6J	20180515	2	yes	1	M
B3	BALB/cJ	20180515	3	no	1	M
B4	C57BL/6J	20180515	1	no	1	F
B5	BALB/cJ	20180515	2	yes	2	F
B6	C57BL/6J	20180515	3	yes	2	M
B7	BALB/cJ	20180515	1	no	2	M
B8	C57BL/6J	20180515	2	no	2	M
B9	BALB/cJ	20180515	3	yes	3	F
B10	C57BL/6J	20180307	1	yes	3	F
B11	BALB/cJ	20180307	2	no	3	M
B12	C57BL/6J	20180307	3	no	3	M

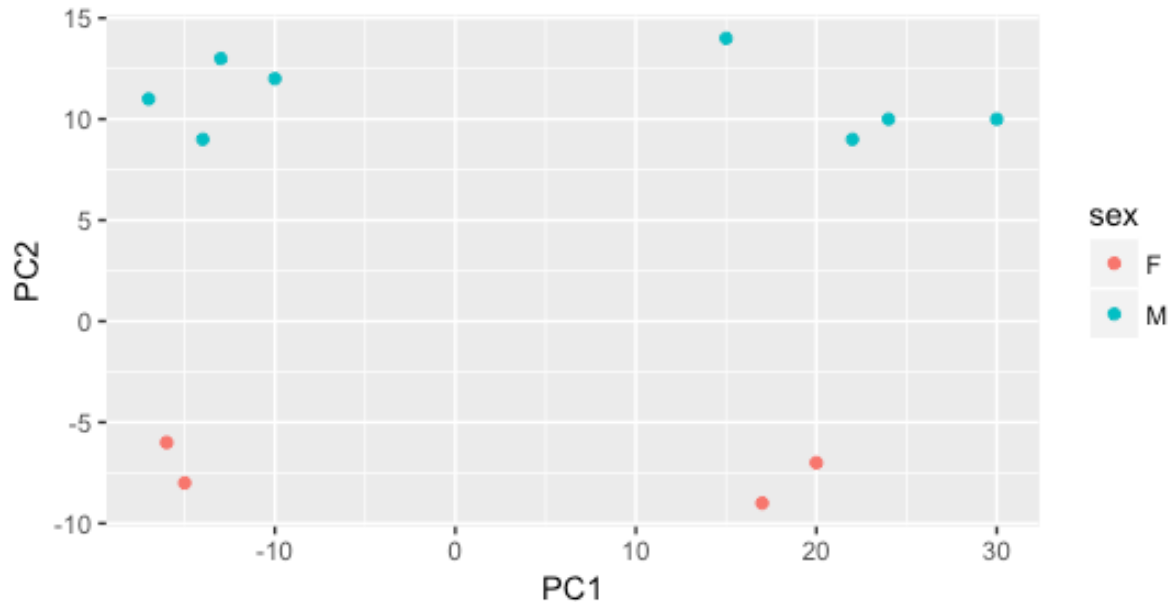
When visualizing on PC1 and PC2, we don't see the samples separate by **treatment**, so we decide to explore other sources of variation present in the data. We hope that we have included all possible known sources of variation in our metadata table, and we can use these factors to color the PCA plot.



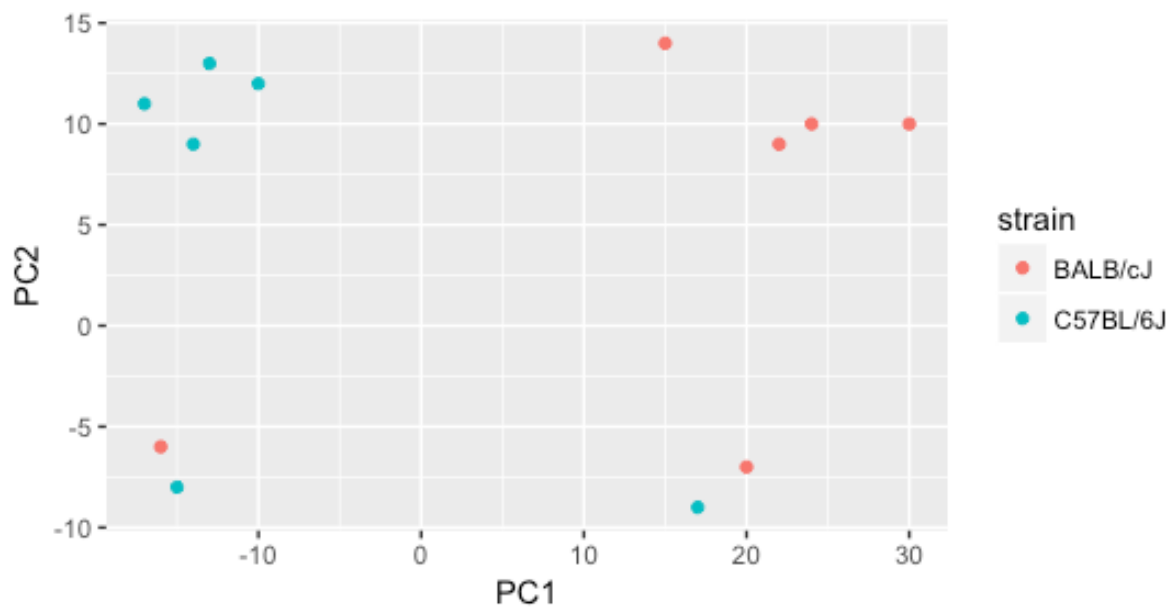
We begin by coloring the points in the PCA plot based on the `cage` factor, but this factor does not seem to explain the variation observed on PC1 or PC2.



Next, we color the points by the `sex` factor. Here, we observe that `sex` separates the samples along PC2, which is valuable information. We can use this in our model to account for variation due to `sex` and potentially regress it out to focus on other sources of variation.



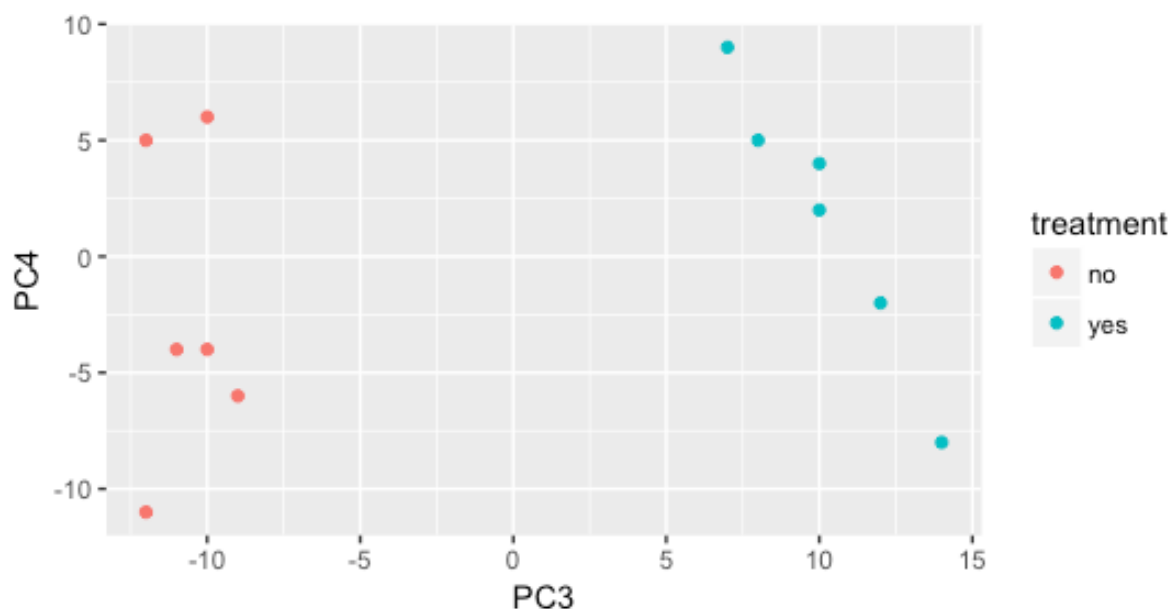
Next we explore the `strain` factor and find that it explains the variation on PC1.



It's great that we have been able to identify the sources of variation for both PC1 and PC2. By accounting for it in our model, we should be able to detect more genes differentially expressed due to `treatment`.

Worrisome about this plot is that we see two samples that do not cluster with the correct strain. This would indicate a likely **sample swap** and should be investigated to determine whether these samples are indeed the labeled strains. If we found there was a switch, we could swap the samples in the metadata. However, if we think they are labeled correctly or are unsure, we could just remove the samples from the dataset.

Still we haven't found if `treatment` is a major source of variation after `strain` and `sex`. So, we explore PC3 and PC4 to see if `treatment` is driving the variation represented by either of these PCs.



We find that the samples separate by **treatment** on PC3, and are optimistic about our DE analysis since our condition of interest, **treatment**, is separating on PC3 and we can regress out the variation driving PC1 and PC2.

Design formula: strain + sex + treatment:

Gene Expression = $\mu + (\text{strain}) + (\text{sex}) + (\text{treatment}) +$

DESeq2 estimates separate coefficients ($\beta_1, \beta_2, \beta_3$) for each factor, then tests specifically for β_3 (treatment effect).

Key Point: You're not making strain/sex disappear - you're statistically controlling for them so you can get a cleaner estimate of the treatment effect.

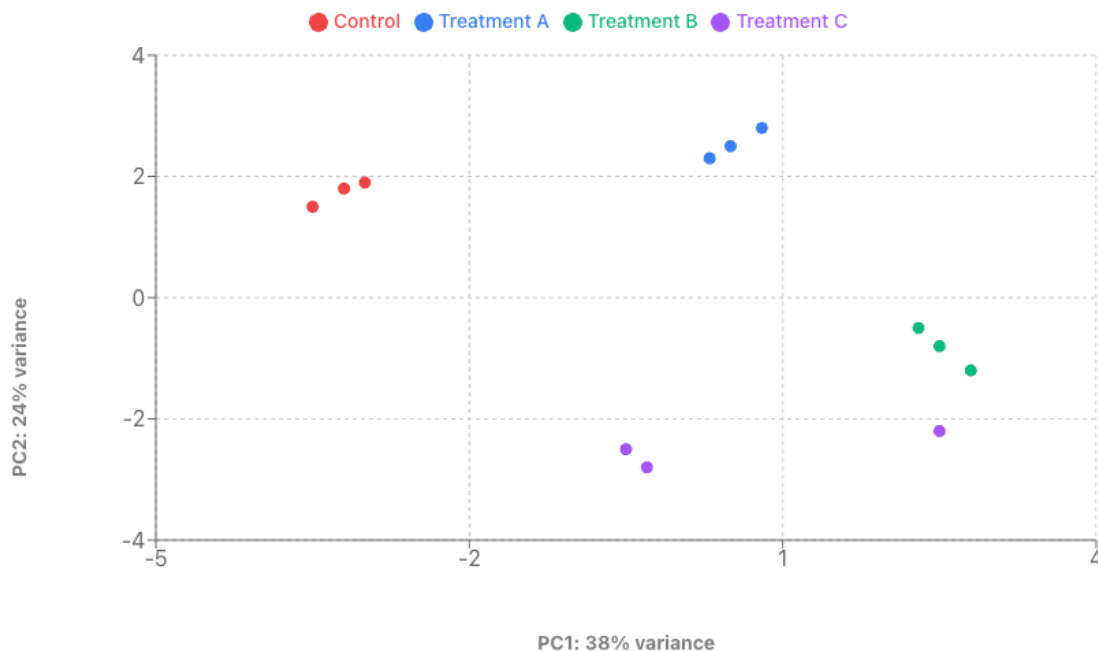
Exercise points = +3

Now let's practice interpreting a PCA plot. In a typical RNA-seq experiment, you'll have multiple samples (different treatments, replicates, etc.) and thousands of genes. After PCA calculates the principal component scores, we plot the samples (not genes) in this new coordinate system.

Below is a practice dataset with 4 treatment groups (Control and 3 drugs) with 3 biological replicates each (12 samples total). Each point represents one sample, positioned based on its PC1 and PC2 scores.

Practice PCA Analysis Exercise

RNA-seq data from 4 sample groups: Control and 3 different drug treatments (A, B, C), with 3 biological replicates each



Analysis Questions

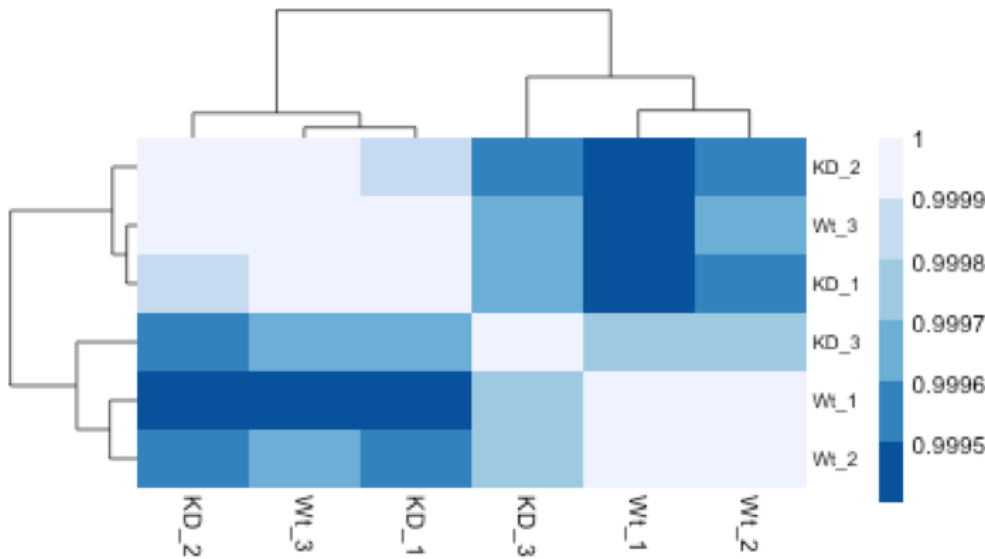
1. What are the main sources of variation in this dataset (PC1 and PC2)?
2. Do the sample groups and replicates cluster appropriately?
3. What concerns should be addressed before proceeding with analysis?

3.1.2 Hierarchical Clustering Heatmap

Similar to PCA, hierarchical clustering is another, complementary method for identifying strong patterns in a dataset and potential outliers. The heatmap displays **the correlation of gene expression for all pairwise combinations of samples** in the dataset. Since the majority of genes are not differentially expressed, samples generally have high correlations with each other (values higher than 0.80). Samples below 0.80 may indicate an outlier in your data and/or sample contamination.

The hierarchical tree can indicate which samples are more similar to each other based on the normalized gene expression values. The color blocks indicate substructure in the data, and you would expect to see your replicates cluster together as a block for each sample group. Additionally, we expect to see samples clustered similar to the groupings observed in a PCA plot.

In the plot below, we would be quite concerned about 'Wt_3' and 'KD_3' samples not clustering with the other replicates. We would want to explore the PCA to see if we see the same clustering of samples.



3.2 Gene-level QC

Prior to differential expression analysis, it's beneficial to remove genes that have little chance of being detected as differentially expressed (such as genes with very low counts across all samples). This filtering increases the power to detect truly differentially expressed genes. DESeq2 performs this filtering automatically, so we don't need to do additional gene filtering steps for our analysis.

3.3 Mov10 quality assessment and exploratory analysis using DESeq2

Now that we have a good understanding of the QC steps normally employed for RNA-seq, let's implement them for the Mov10 dataset we are going to be working with.

Transform normalized counts using the rlog transformation Before creating PCA plots and other visualizations, we need to transform our normalized counts using the regularized log transformation (rlog).

Why transform the data? Raw RNA-seq count data has a problem: genes with low expression show high variability (noise), while highly expressed genes show less variability. This means highly expressed genes would dominate any clustering or PCA analysis. The rlog transformation solves this by:

Converting counts to log2 scale (makes genes more comparable) Stabilizing variance across all expression levels (so all genes contribute more equally)

The result: Data that's appropriate for visualization and quality control, where patterns reflect biological differences rather than technical noise from expression level differences.

3.3.1 Transform counts for data visualization

```
rld <- rlog(dds, blind=TRUE)
```

The `blind=TRUE` argument means the transformation doesn't use sample group information, which is important for unbiased quality assessment.

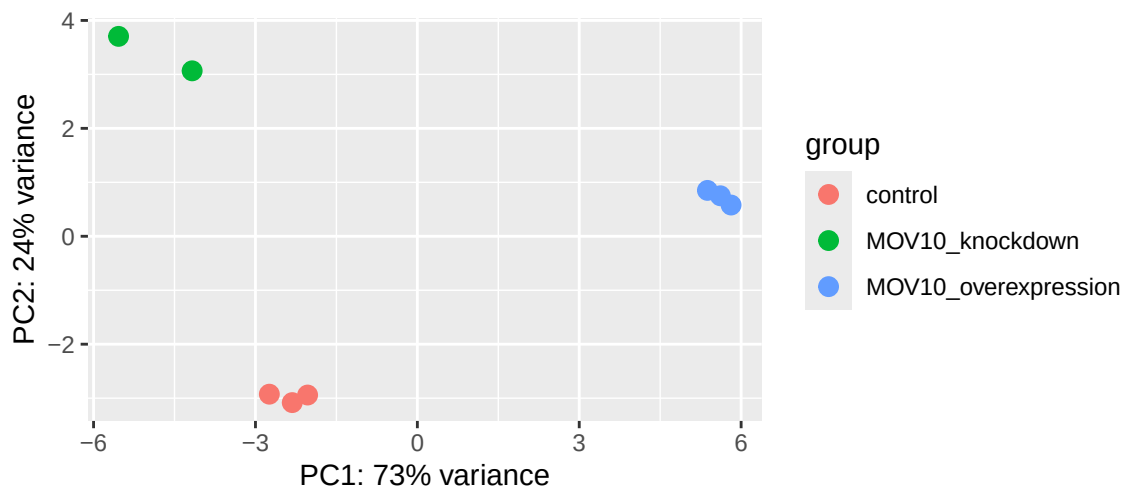
Note: Use rlog-transformed data only for visualizations (PCA, heatmaps, clustering). For differential expression analysis, DESeq2 uses the original count data.

3.3.2 Principal components analysis (PCA)

DESeq2 has a built-in function for plotting PCA plots, that uses `ggplot2` under the hood. This is great because it saves us having to type out lines of code and having to fiddle with the different `ggplot2` layers. In addition, it takes the `rlog` object as an input directly, hence saving us the trouble of extracting the relevant information from it.

The function `plotPCA()` requires two arguments as input: an `rlog` object and the `intgroup` (the column in our metadata that we are interested in).

```
### Plot PCA
plotPCA(rld, intgroup="sampletype")
```



What does this plot tell you about the similarity of samples? Does it fit the expectation from the experimental design? By default the function uses the *top 500 most variable genes*. You can change this by adding the `ntop` argument and specifying how many genes you want to use to draw the plot.

3.3.3 Hierarchical Clustering

Since there is no built-in function for heatmaps in DESeq2 we will be using the `pheatmap()` function from the `pheatmap` package. This function requires a matrix/dataframe of numeric values as input, and so the first thing we need to is retrieve

that information from the `rld` object:

```
### Extract the rlog matrix from the object
rld_mat <- assay(rld) ## assay() is function from the "SummarizedExperiment" package that was loaded when you
```

Then we need to compute the pairwise correlation values for samples. We can do this using the `cor()` function:

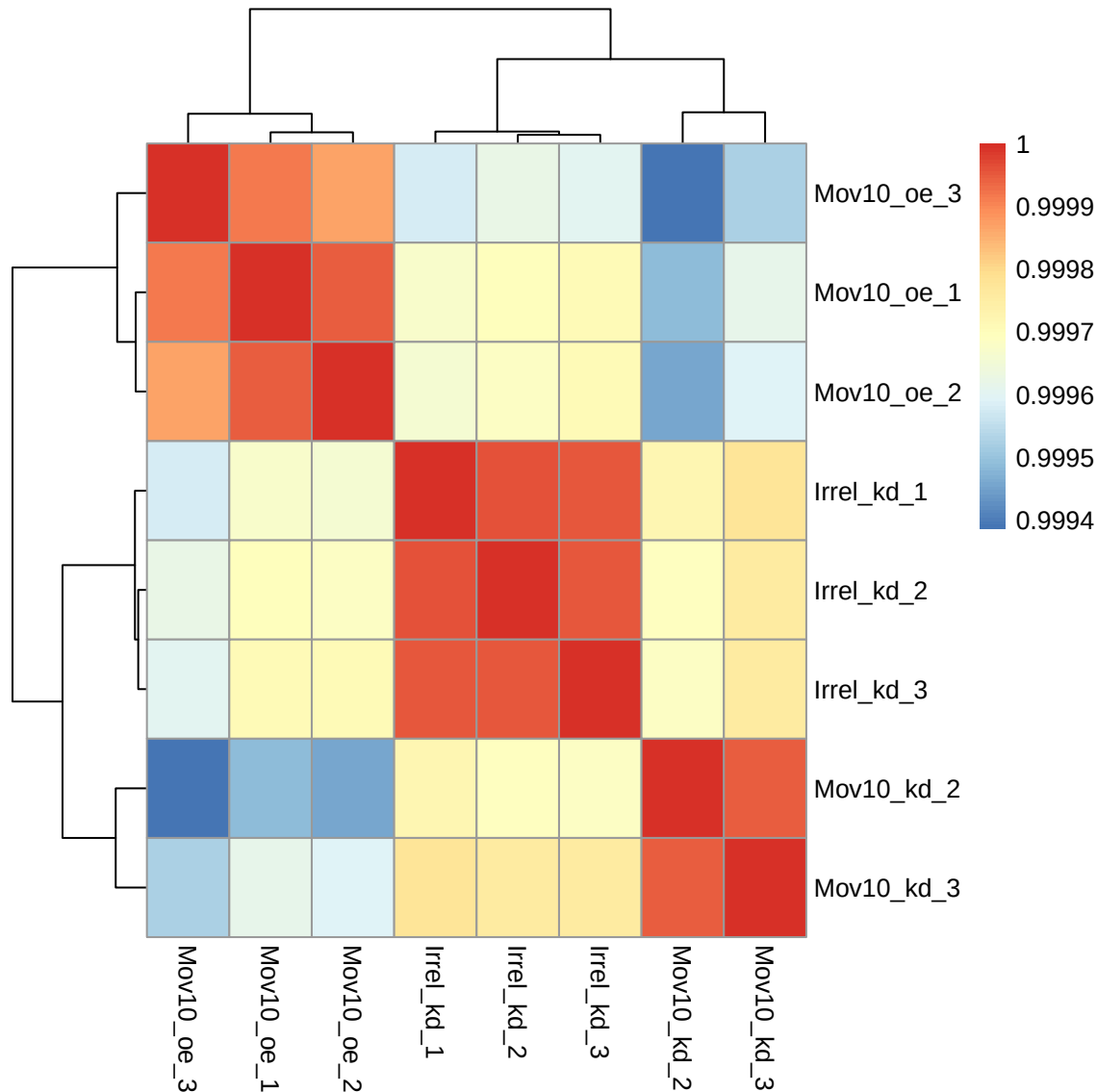
```
### Compute pairwise correlation values
rld_cor <- cor(rld_mat) ## cor() is a base R function

head(rld_cor) ## check the output of cor(), make note of the rownames and colnames
```


3.4	Irrel_kd_1	Irrel_kd_2	Irrel_kd_3	Mov10_kd_2	Mov10_kd_3	Mov10_oe_1
3.5	Irrel_kd_1	1.0000000	0.9999614	0.9999532	0.9997202	0.9997748 0.9996700
3.6	Irrel_kd_2	0.9999614	1.0000000	0.9999544	0.9996918	0.9997568 0.9996984
3.7	Irrel_kd_3	0.9999532	0.9999544	1.0000000	0.9996816	0.9997574 0.9997067
3.8	Mov10_kd_2	0.9997202	0.9996918	0.9996816	1.0000000	0.9999492 0.9994868
3.9	Mov10_kd_3	0.9997748	0.9997568	0.9997574	0.9999492	1.0000000 0.9996154
3.10	Mov10_oe_1	0.9996700	0.9996984	0.9997067	0.9994868	0.9996154 1.0000000
3.11	Mov10_oe_2	Mov10_oe_3				
3.12	Irrel_kd_1	0.9996599	0.9995804			
3.13	Irrel_kd_2	0.9996825	0.9996227			
3.14	Irrel_kd_3	0.9997090	0.9996026			
3.15	Mov10_kd_2	0.9994565	0.9993869			
3.16	Mov10_kd_3	0.9995905	0.9995235			
3.17	Mov10_oe_1	0.9999505	0.9999196			

And now to plot the correlation values as a heatmap:

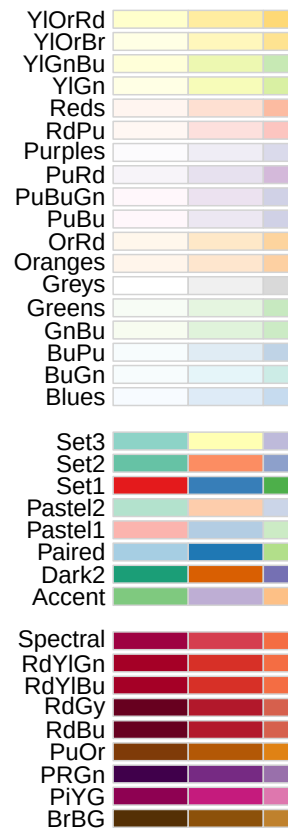
```
### Plot heatmap
pheatmap(rld_cor)
```



Overall, we observe pretty high correlations across the board (> 0.999) suggesting no outlying sample(s). Also, similar to the PCA plot you see the samples clustering together by sample group. Together, these plots suggest to us that the data are of good quality and we have the green light to proceed to differential expression analysis.

Exercise points = +6

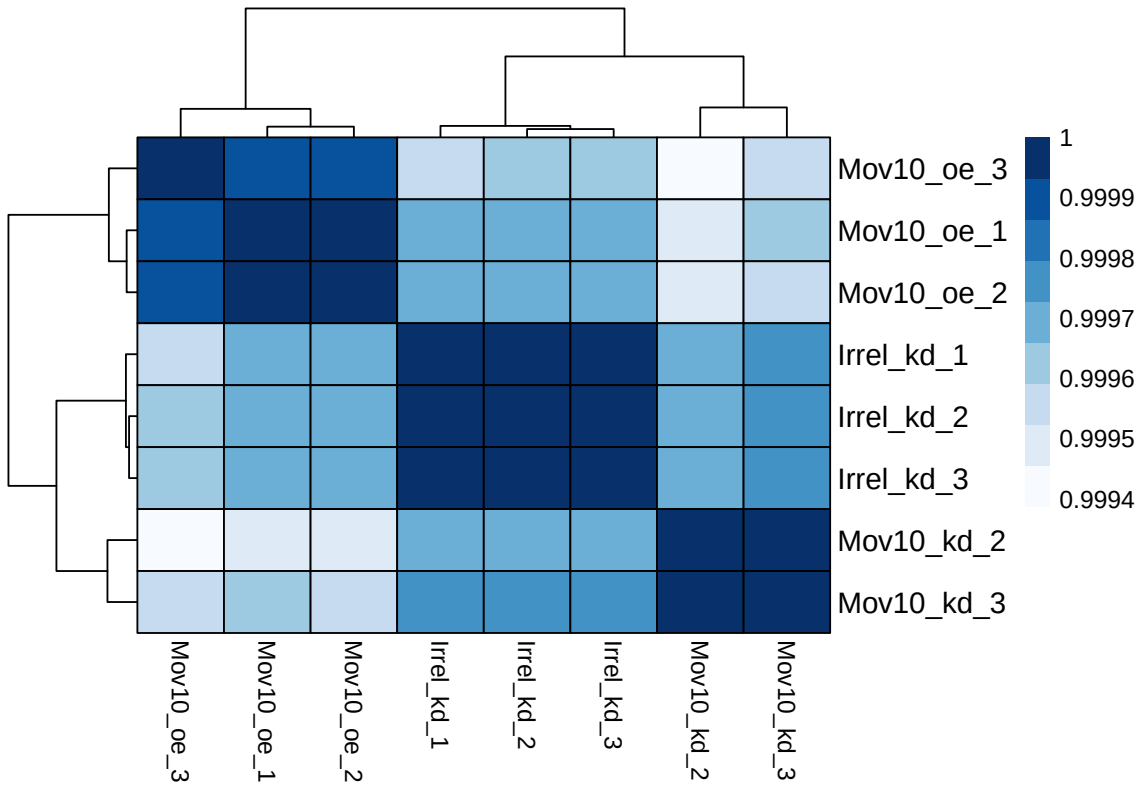
The pheatmap function has a number of different arguments that we can alter from default values to enhance the aesthetics of the plot. Add the arguments `color`, `border_color`, `fontsize_row`, `fontsize_col`, `show_rownames` and `show_colnames` to your pheatmap. How does your plot change (plot shown below)? Take a look through the help pages (`?pheatmap`) and identify what each of the added arguments is contributing to the plot.



The library RColorBrewer provides different palettes for plotting. We can display all of the palettes using:

```
# The color palette "Blues" is a good choice for this heatmap
heat.colors <- brewer.pal(9, "Blues")

pheatmap(rld_cor,
  color = heat.colors,
  border_color = "black",
  fontsize_row = 12,
  fontsize_col = 10,
  show_rownames = TRUE,
  show_colnames = TRUE)
```



Identify what each of the added arguments is contributing to the plot.

Chapter 4

Differential expression analysis with DESeq2

The final step in the differential expression analysis workflow is fitting the raw counts to the NB model and performing the statistical test for differentially expressed genes. In this step we essentially want to determine whether the mean expression levels of different sample groups are significantly different.

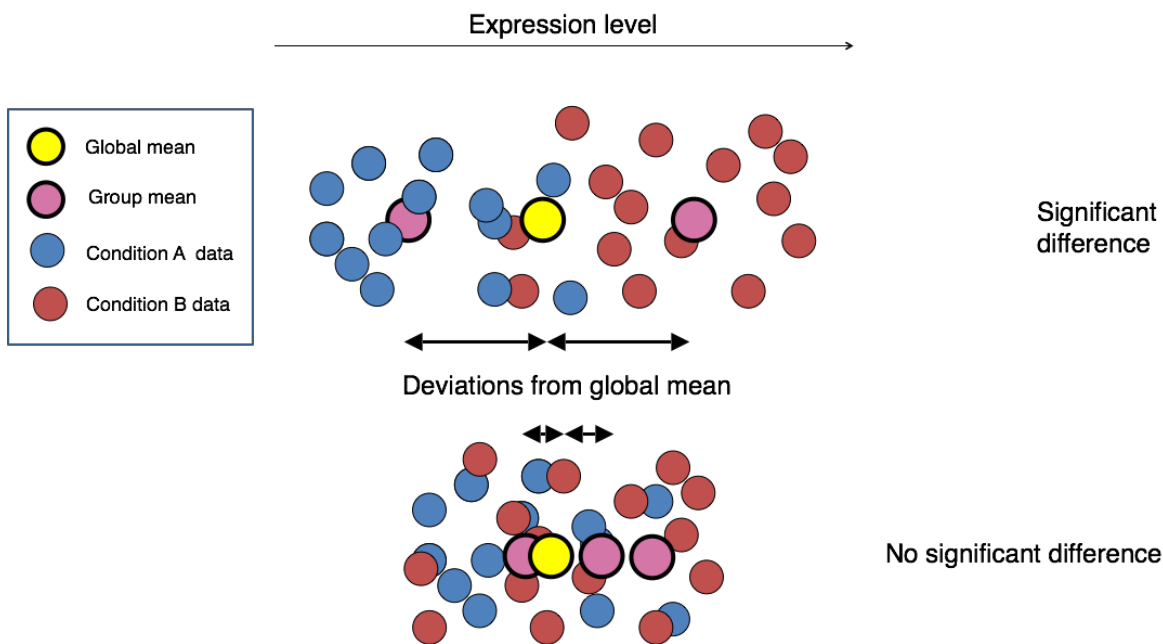
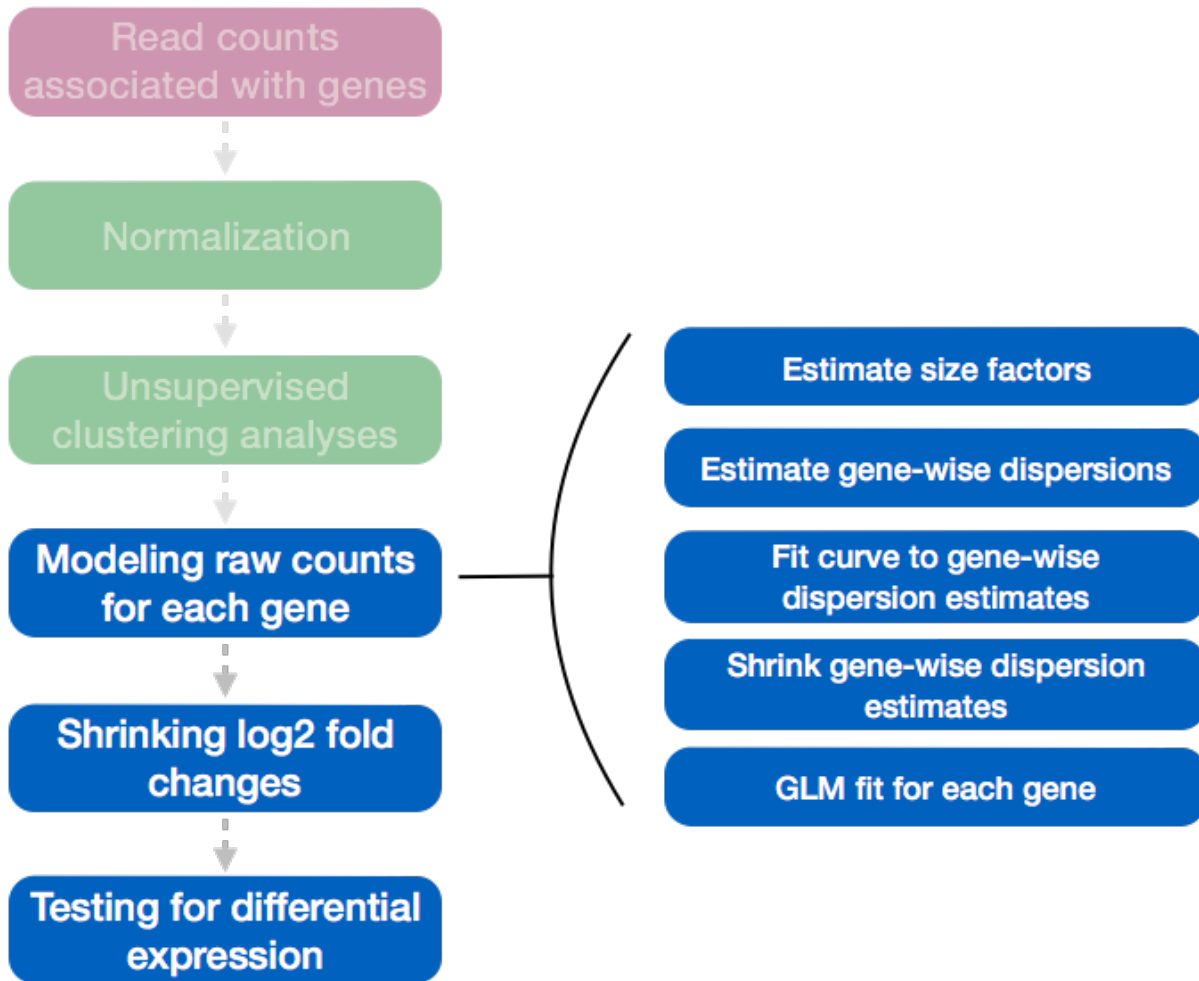


Image credit: Paul Pavlidis, UBC

Differential expression analysis with DESeq2 involves multiple steps as displayed in the flowchart below in blue. Briefly, DESeq2 will model the raw counts, using normalization factors (size factors) to account for differences in library depth. Then, it will estimate the gene-wise dispersions and shrink these estimates to generate more accurate estimates of dispersion to model the counts. Finally, DESeq2 will fit the negative binomial model and perform hypothesis testing using the Wald test or Likelihood Ratio Test.



4.1 Running DESeq2

Prior to performing the differential expression analysis, it is a good idea to know what **sources of variation** are present in your data, either by exploration during the QC and/or prior knowledge. Once you know the major sources of variation, you can control for them in the statistical model by including them in your **design formula**.

4.1.1 Design formula

A design formula tells the statistical software the known sources of variation to control for, as well as, the factor of interest to test for during differential expression testing. For example, if you know that sex is a significant source of variation in your data, then sex should be included in your model. **The design formula should have all of the factors in your metadata that account for major sources of variation in your data. The last factor entered in the formula should be the condition of interest.**

For example, suppose you have the following metadata:

	sex	age	litter	treatment
sample1	M	11	1	Ctrl
sample2	M	13	2	Ctrl
sample3	M	11	1	Treat
sample4	M	13	1	Treat
sample5	F	11	1	Ctrl
sample6	F	13	1	Ctrl
sample7	F	11	1	Treat
sample8	F	13	2	Treat

If you want to examine the expression differences between treatments, and you know that major sources of variation include **sex** and **age**, then your design formula would be:

```
design <- ~ sex + age + treatment
```

The tilde (~) should always proceed your factors and tells DESeq2 to model the counts using the following formula. Note the **factors included in the design formula need to match the column names in the metadata**.

Exercise points = +3

1. Suppose you wanted to study the expression differences between the two age groups in the metadata shown above, and major sources of variation were **sex** and **treatment**, how would the design formula be written?

the default behavior of `results` is to return the comparison of the last group in the design formula, so you

2. Based on our Mov10 metadata dataframe, which factors could we include in our design formula?
3. What would you do if you wanted to include a factor in your design formula that is not in your metadata?

4.1.2 MOV10 Differential Expression Analysis

Running Differential Expression in Two Lines of Code

To obtain differential expression results from our raw count data, we only need to run two lines of code!

First, we create a DESeqDataSet, as we did in the ‘Count normalization’ lesson, specifying the location of our raw counts and metadata, and applying our design formula:

```
## Create DESeq object
dds <- DESeqDataSetFromMatrix(countData = data, colData = meta, design = ~ sampletype)
```

Next, we run the actual differential expression analysis with a single call to the DESeq() function. This function handles everything—from **normalization** to **linear** modeling—all in one step. During execution, DESeq2 will print messages detailing the steps being performed: estimating size factors, estimating dispersions, gene-wise dispersion estimates, modeling the mean-dispersion relationship, and statistical testing for differential expression.

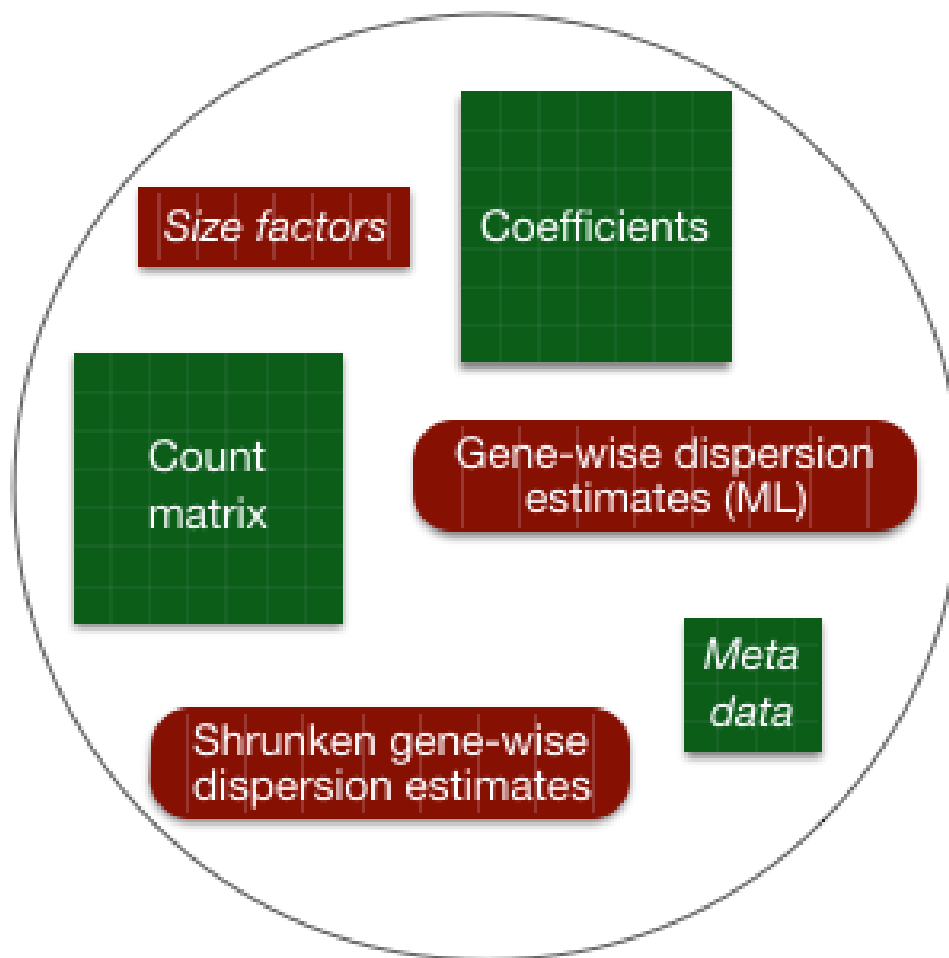
```
## Run analysis  
dds <- DESeq(dds)
```

By re-assigning the result to back to the same variable name (`dds`), we update our `DESeqDataSet` object, which will now contain the results of each step in the analysis, effectively filling in the `slots` of our `DESeqDataSet` object.

4.1.3 DESeq2 differential gene expression analysis workflow

Everything from normalization to linear modeling was carried out by the use of a single function!

With the 2 lines of code above, we just completed the workflow for the differential gene expression analysis with DESeq2. The steps in the analysis are output below:



Everything from normalization to linear modeling was carried out by the use of a single function! This function will print out a message for the various steps it performs:

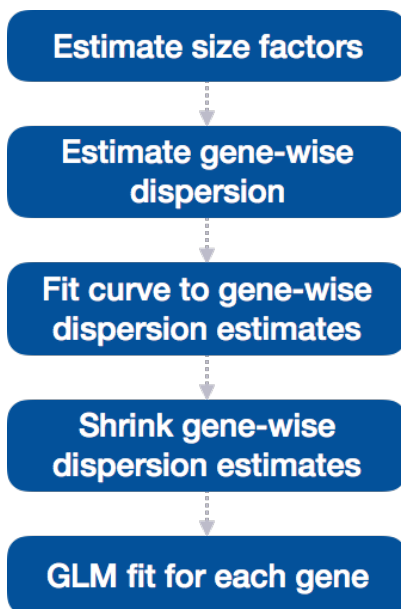
```
estimating size factors  
estimating dispersions  
gene-wise dispersion estimates
```

```
mean-dispersion relationship  
final dispersion estimates  
fitting model and testing
```

NOTE: There are individual functions available in DESeq2 that would allow us to carry out each step in the workflow in a step-wise manner, rather than a single call. We demonstrated one example when generating size factors to create a normalized matrix. By calling `DESeq()` the individual functions for each step are run for you.

4.2 DESeq2 differential gene expression analysis workflow

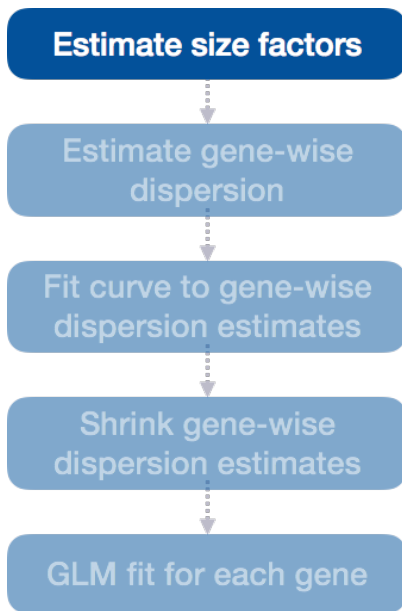
With the 2 lines of code above, we just completed the workflow for the differential gene expression analysis with DESeq2. The steps in the analysis are output below:



We will be taking a detailed look at each of these steps to better understand how DESeq2 is performing the statistical analysis and what metrics we should examine to explore the quality of our analysis.

4.2.1 Step 1: Estimate size factors

The first step in the differential expression analysis is to estimate the size factors, which is exactly what we already did to normalize the raw counts.



DESeq2 will automatically estimate the size factors when performing the differential expression analysis. However, if you have already generated the size factors using `estimateSizeFactors()`, as we did earlier, then DESeq2 will use these values.

4.2.1.1 MOV10 DE analysis: examining the size factors

Let's take a quick look at size factor values we have for each sample:

```
## Check the size factors
sizeFactors(dds)
```

```
4.3  Irrel_kd_1  Irrel_kd_2  Irrel_kd_3  Mov10_kd_2  Mov10_kd_3
     Mov10_oe_1 Mov10_oe_2
```

```
4.4  1.1224020 0.9625632 0.7477715 1.5646728 0.9351760 1.2016082 1.1205912
```

```
4.5  Mov10_oe_3
```

```
4.6  0.6534987
```

These numbers should be identical to those we generated initially when we had run the function `estimateSizeFactors(dds)`.

Exercise points = +2

Take a look at the total number of reads for each sample:

4.7. IRREL_KD_1 IRREL_KD_2 IRREL_KD_3 MOV10_KD_2 MOV10_KD_3 MOV10_OE_1 MOV10_OE_2 61

```
## Total number of raw counts per sample
colSums(counts(dds))
```

4.7 Irrel_kd_1 Irrel_kd_2 Irrel_kd_3 Mov10_kd_2 Mov10_kd_3
Mov10_oe_1 Mov10_oe_2

4.8 22687366 19381680 14962754 32826936 19360003 23447317 21713289

4.9 Mov10_oe_3

4.10 12737889

How do the numbers correlate with the size factor?

Now take a look at the total depth after normalization using:

```
## Total number of normalized counts per sample
colSums(counts(dds, normalized=T))
```

4.11 Irrel_kd_1 Irrel_kd_2 Irrel_kd_3 Mov10_kd_2 Mov10_kd_3
Mov10_oe_1 Mov10_oe_2

4.12 20213226 20135487 20009794 20980064 20701989 19513279 19376636

4.13 Mov10_oe_3

4.14 19491836

How do the normalized values across samples compare with the total counts taken for each sample?

4.14.1 Step 2: Estimate gene-wise dispersion

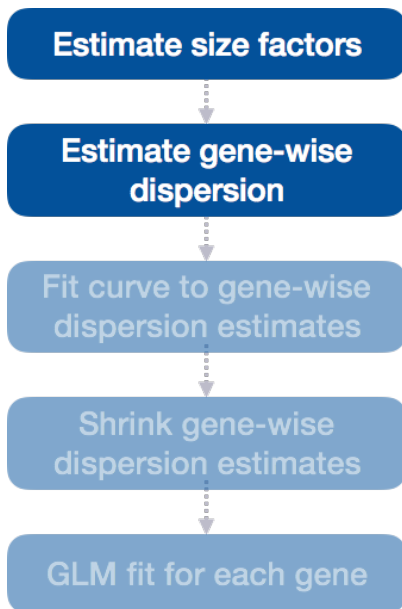
DESeq2 estimates dispersion for each gene - a measure of how much gene expression varies between biological replicates. DESeq2's dispersion parameter is specifically designed for this. It tells you after accounting for the mean-variance relationship in count data, how variable is this gene? High dispersion genes are legitimately noisy (biologically or technically), regardless of expression level.

Models variance as:

$$\text{Var}(Y_{ij}) = \mu_{ij} + \alpha_i \mu_{ij}^2$$

OR

$$\alpha = \frac{\text{Var}(Y) - \mu}{\mu^2}$$



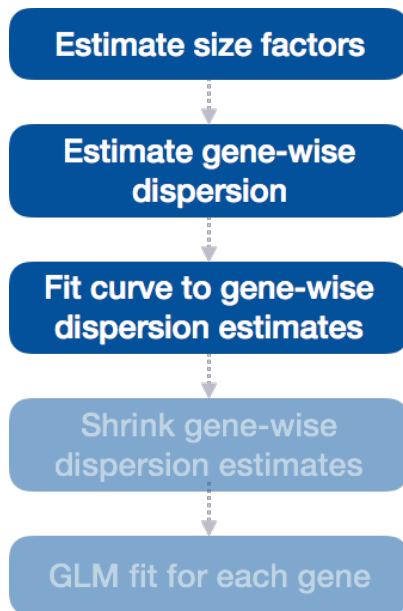
Key pattern observed in RNA-seq data:

- **Low-expression genes** tend to be more variable between replicates (**higher dispersion**)
- **High-expression genes** tend to be more consistent between replicates (**lower dispersion**)

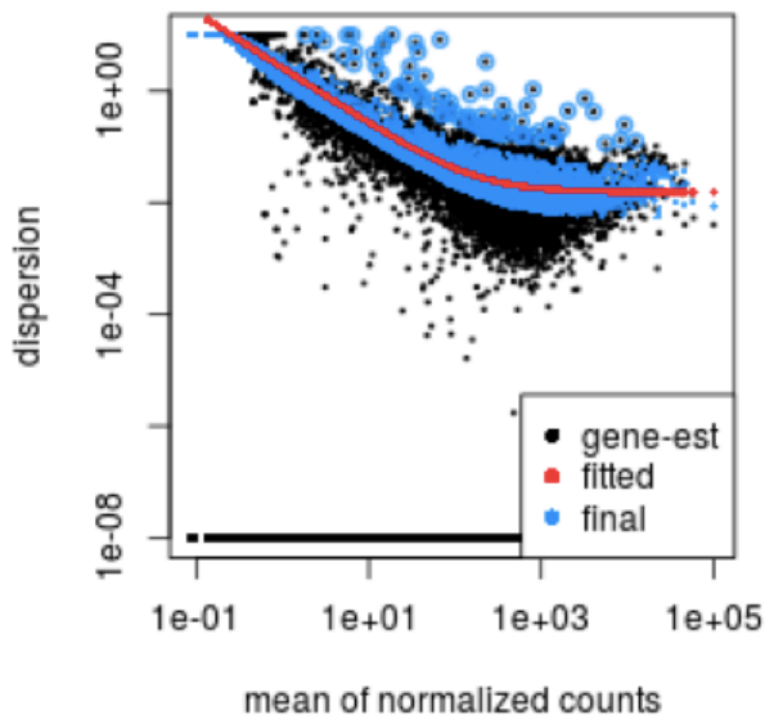
Why this matters: With only a few replicates per group, individual gene dispersion estimates can be noisy and unreliable. DESeq2 improves these estimates by using information from all genes, assuming that genes with similar expression levels should have similar variability using a statistical method called maximum likelihood estimation (MLE). In other words, **given the count values of the replicates, the most likely estimate of dispersion is calculated.**

4.14.2 Step 3: Fit Curve to Gene-wise Dispersion Estimates

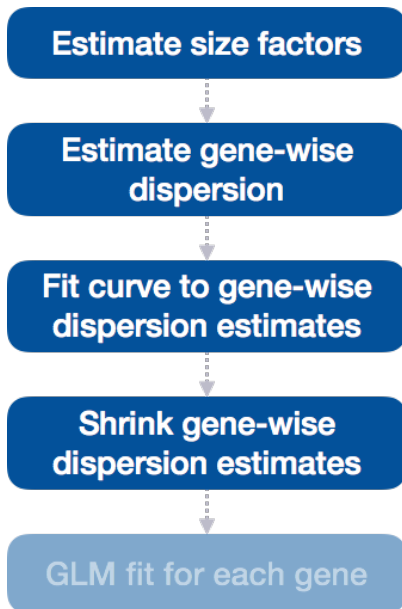
After estimating gene-wise dispersions, DESeq2 fits a smooth curve of dispersion vs. mean expression across all genes.



This curve is displayed as a red line in the figure below, which plots the estimate for the **expected dispersion value for genes of a given expression strength**. Each black dot is a gene with an associated mean expression level and maximum likelihood estimation (MLE) of the dispersion.



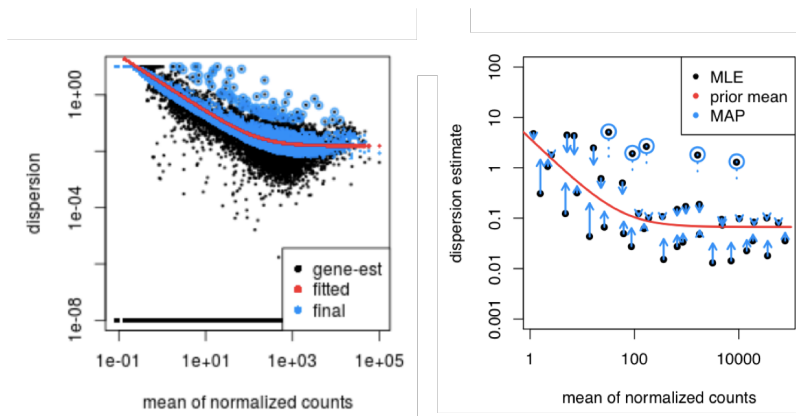
4.14.3 Step 4: Shrink gene-wise dispersion estimates toward the values predicted by the curve



DESeq2 then “shrinks” each gene’s noisy estimate toward this curve. Genes with **unreliable estimates (low counts)** get adjusted more, while genes with **reliable estimates (high counts)** stay closer to their original values.

This shrinkage method is particularly important to reduce false positives in the differential expression analysis. Genes with low dispersion estimates are shrunk towards the curve, and the more accurate, higher shrunk values are output for fitting of the model and differential expression testing.

Dispersion estimates that are slightly above the curve are also shrunk toward the curve for better dispersion estimation; however, genes with extremely high dispersion values are not. This is due to the likelihood that the gene does not follow the modeling assumptions and has higher variability than others for biological or technical reasons. Shrinking the values toward the curve could result in false positives, so these values are not shrunk. These genes are shown surrounded by blue circles below.

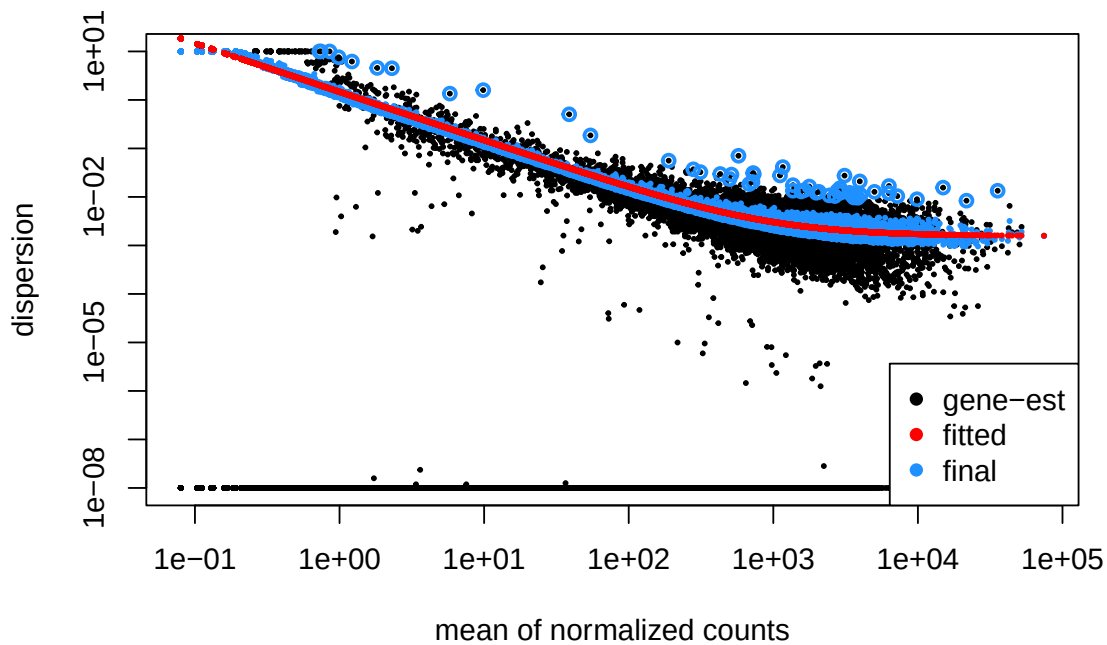


This entire dispersion estimation and shrinkage process happens automatically when you run `DESeq(dds)`.

4.14.4 MOV10 Differential Expression Analysis: Exploring Dispersion Estimates

Now, let's explore the dispersion estimates for the MOV10 dataset:

```
## Plot dispersion estimates
plotDispEsts(dds)
```



You should see genes scattered around a decreasing curve, with dispersion decreasing as mean expression increases. Since we have a small sample size, many genes show shrinkage toward the fitted trend (red line), which is expected and beneficial for the analysis.

Exercise points = +1

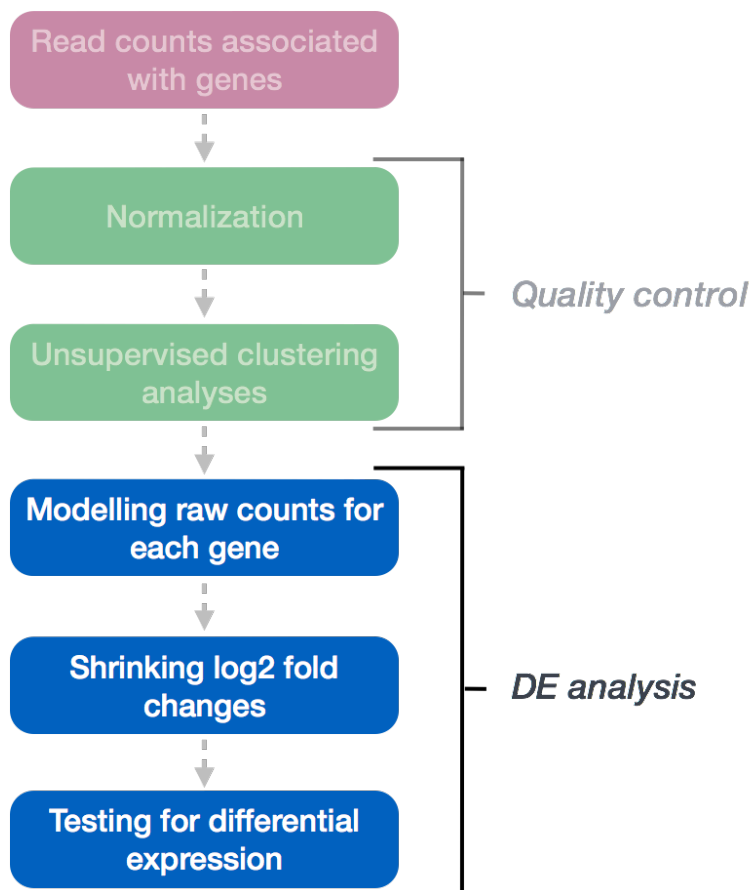
Do you think our data are a good fit for the model?

Chapter 5

Negative Binomial model fitting

5.1 Generalized Linear Model fit for each gene

The final step in the DESeq2 workflow is fitting the Negative Binomial model for each gene and performing differential expression testing.



Exercise points = +1

In the DESeq2 workflow, what is the purpose of the design matrix in the context of fitting the Negative Binomial model for each gene?

DESeq2 fits the Negative Binomial model to the normalized count data for each gene, taking into account the size factors and dispersion estimates calculated earlier. The model estimates **beta coefficients** for each experimental condition – these are the statistical parameters that represent the **log2 fold changes** for each experimental condition compared to the baseline. For example, if your experiment compares MOV10 overexpression vs. control, DESeq2 calculates a **beta coefficient** that tells you how much gene expression changed due to the MOV10 overexpression (on the log2 scale).

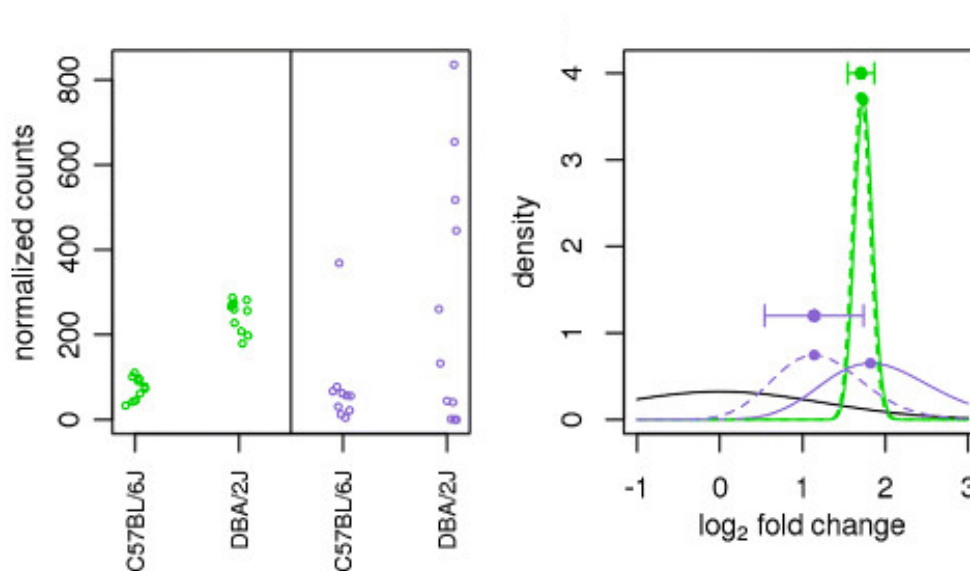
However, these initial log2 fold change estimates can be **unreliable** for genes with low counts or high dispersion. These genes have high uncertainty in their fold change estimates (large lfcSE), which can lead to **inflated fold changes** that may not be biologically meaningful.

5.2 Shrunk log2 foldchanges (LFC)

To generate more accurate log2 foldchange estimates, DESeq2 allows for the **shrinkage of the LFC estimates toward zero** when the information for a gene is low, which could include:

- Low counts
- High dispersion values

As with the shrinkage of dispersion estimates, **LFC shrinkage uses information from all genes to generate more accurate estimates**. DESeq2 looks at the pattern of fold changes across all genes in your dataset and uses this information to adjust estimates for genes with little reliable data (low counts or high dispersion) toward more reasonable values.



For example, in the figure above, Looking at the figure, the green and purple genes both show similar average expression levels between the two sample groups, but they differ in how much variability they have. The green gene has very consistent values (low variability), while the purple gene has much more scattered data points (high variability). Because the green gene has reliable, consistent data, DESeq2 doesn't need to adjust its fold change estimate much - the original and adjusted estimates are nearly the same. However, for the purple gene with noisy, variable data, DESeq2 pulls the fold change estimate toward a more moderate value to account for the uncertainty.

This shows how DESeq2 tailors the amount of adjustment based on data quality - reliable genes get little adjustment, while noisy genes get more adjustment toward reasonable values.

To generate the shrunken log2 fold change estimates, you have to run an additional step on your results object (that we will create below) with the function `lfcShrink()`.

NOTE: LFC shrinkage does not change which genes are called significantly differentially expressed (determined by padj values). It only provides more accurate fold change estimates for downstream analysis. Shrunk LFCs are particularly important for:

- Ranking genes by effect size
- Subsetting significant genes by fold change thresholds
- Functional analysis tools like GSEA that use fold changes as input
- Visualization and interpretation of results

5.3 Hypothesis testing using the Wald test

For each gene, DESeq2 tests whether there is a significant difference in expression between your experimental conditions.

Null hypothesis (H₀): There is no difference between groups (log2 fold change = 0)

DESeq2 uses the **Wald test** to calculate a p-value for each gene, which represents the probability of observing a fold change this large (or larger) if the null hypothesis were true. If the p-value is small (typically < 0.05 after multiple testing correction), we reject the null hypothesis and conclude the gene is differentially expressed.

5.4 Creating contrasts

To indicate to DESeq2 the two groups we want to compare, we can use **contrasts**. A **contrast** is provided to DESeq2 as an argument in the `results()` function, the function that performs differential expression testing using the Wald test.

In the contrast you can specify the comparison of interest and the levels to compare. The level given last is the base level for the comparison. The syntax is given below:

```
# General syntax
contrast <- c("condition", "level_to_compare", "base_level")
results(dds, contrast = contrast)
```

5.4.0.1 MOV10 DE analysis: contrasts and Wald tests

We have three sample classes so we can make three possible pairwise comparisons:

1. Control vs. Mov10 overexpression
2. Control vs. Mov10 knockdown
3. Mov10 knockdown vs. Mov10 overexpression

We are really only interested in #1 and #2 from above. Using the design formula we provided `~ sampletype`, indicating that this is our main factor of interest.

5.4.1 Building the results table

To build our results table we will use the `results()` function. To specify which groups to compare, we supply the contrast using the `contrast` argument. In this example, we'll first extract the unshrunk results for **MOV10 overexpression vs. Control**, then apply log2 fold change shrinkage.

```
# Define contrasts
contrast_oe <- c("samplotype",
                 "MOV10_overexpression",
                 "control")

# Extract results table (unshrunk)
res_tableOE_unshrunk <- results(dds, contrast = contrast_oe)

# Check available coefficients for shrinkage
resultsNames(dds)
```

5.5 [1] “Intercept”

5.6 [2] “samplotype__MOV10_knockdown_vs_control”

5.7 [3] “samplotype__MOV10_overexpression_vs_control”

```
# Shrink log2 fold changes
res_tableOE <- lfcShrink(dds = dds,
                        coef = "samplotype_MOV10_overexpression_vs_control",
                        res = res_tableOE_unshrunk)

# Save the results for future use
saveRDS(res_tableOE, file = "data/res_tableOE.RDS")
```

Understanding the contrast order:

This calculates fold changes as: **MOV10_overexpression / control**

Interpreting the results:

- **Positive log2FC** → gene is upregulated in MOV10_overexpression (higher expression than control)
- **Negative log2FC** → gene is downregulated in MOV10_overexpression (lower expression than control)

For example, log2FC = -2 means the gene has 4-fold lower expression in MOV10_overexpression compared to control.

STOPPED HERE

MA Plot

A **MA plot** visualizes the relationship between the **mean expression** (on the x-axis) and the **log2 fold change** (on the y-axis) for all genes in a dataset. Genes that are significantly differentially expressed are highlighted, while most other genes cluster around zero log2 fold change.

When **shrinkage** is applied, as in DESeq2, large log2 fold change estimates for genes with low counts or high variability are pulled toward more moderate values, reducing the noise in the plot. This helps focus attention on the more reliable fold changes and makes the plot cleaner, especially for low-expression genes that tend to have more variable LFC estimates without shrinkage.

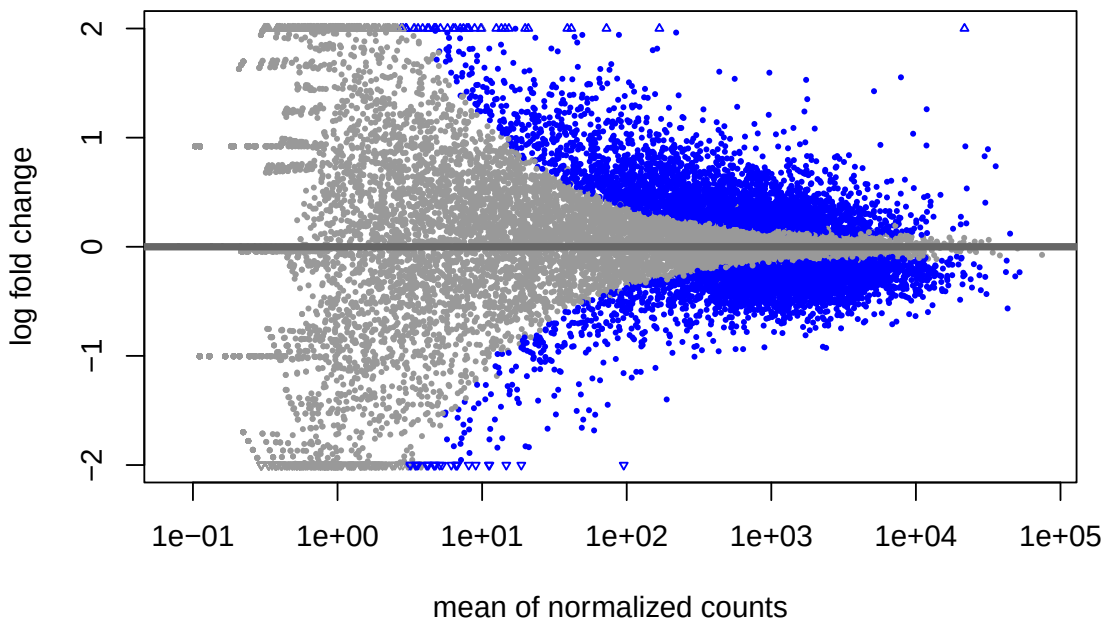
This plot is also a great way to illustrate the effect of LFC shrinkage.

Lots of shrinkage = noisy data or many lowly expressed genes Minimal shrinkage = high quality, well-powered dataset

DESeq2 provides a simple function to generate an MA plot.

Unshrunk results:

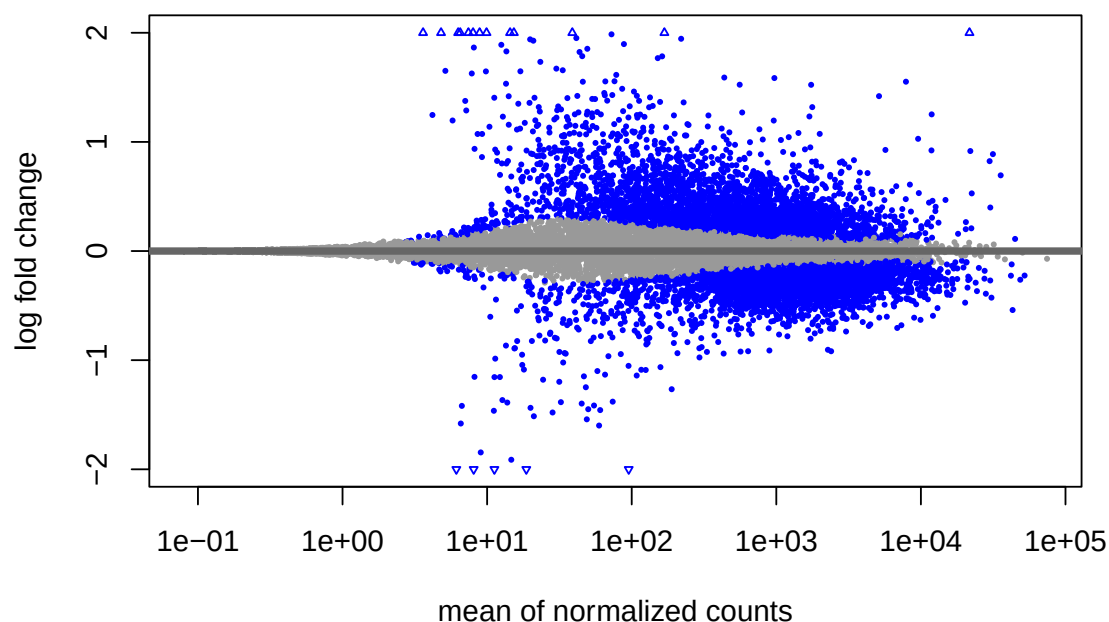
```
plotMA(res_tableOE_unshrunk, ylim=c(-2,2))
```



Notice the wide spread of fold changes, especially for low-expression genes.

Shrunk results:

```
plotMA(res_tableOE, ylim=c(-2,2))
```



After shrinkage, extreme fold changes from low-count genes are moderated, while high-confidence changes are preserved.

What to look for: Significant genes should be distributed across expression levels. If all your DE genes are low-expression, this may indicate quality issues.

5.7.1 MOV10 DE analysis: results exploration

The results table looks very much like a `data.frame` and in many ways it can be treated like one (i.e. when accessing/subsetting data). However, it is important to recognize that it is actually stored in a `DESeqResults` object. When we start visualizing our data, this information will be helpful.

```
class(res_table0E)
```

5.8 [1] “DESeqResults”

5.9 attr(“package”)

5.10 [1] “DESeq2”

Let’s go through some of the columns in the results table to get a better idea of what we are looking at. To extract information regarding the meaning of each column we can use `mcols()`:

```
mcols(res_tableOE, use.names=TRUE)
```

5.11 DataFrame with 5 rows and 2 columns

5.12 type description

5.13

5.14 baseMean intermediate mean of normalized c..

5.15 log2FoldChange results log2 fold change (MA..

5.16 lfcSE results posterior SD: sample..

5.17 pvalue results Wald test p-value: s..

5.18 padj results BH adjusted p-values

- baseMean: intermediate mean of normalized counts for all samples
- log2FoldChange: results log2 fold change (MLE): condition MOV10_overexpression vs control
- lfcSE: results standard error: condition MOV10_overexpression vs control
- pvalue: results Wald test p-value: condition MOV10_overexpression vs control
- padj: results BH adjusted p-values

lfcSE: Standard error of the log2FoldChange (uncertainty in the estimate). This is used to calculate the p-value - genes with high uncertainty are less likely to be significant.

5.18.1 How DESeq2 determines significance

DESeq2 uses the **Wald test** to ask: “Is the fold change large compared to its uncertainty?”

The logic:

1. Calculate how many “standard errors” away from zero the fold change is: $\text{Wald statistic} = \text{log2FC} / \text{lfcSE}$
2. Convert this to a p-value: “How likely is a value this extreme if there’s really no difference?”
3. Large fold changes with low uncertainty $\rightarrow$ small p-values $\rightarrow$ significant

Example:

- Gene A: $\log_2\text{FC} = 2$, $\text{lfcSE} = 0.5 \rightarrow \text{Wald} = 4 \rightarrow \text{very small p-value}$
- Gene B: $\log_2\text{FC} = 2$, $\text{lfcSE} = 2.0 \rightarrow \text{Wald} = 1 \rightarrow \text{large p-value}$

Now let's look at the results:

```
head(res_table0E)
```

5.19 log2 fold change (MAP): sampletype MOV10_overexpression vs control

5.20 Wald test p-value: sampletype MOV10 overexpression vs control

5.21 DataFrame with 6 rows and 5 columns

5.22 baseMean log2FoldChange lfcSE pvalue padj

5.23

5.24 1/2-SBSRNA4 45.652040 0.16820235 0.209100 0.1610752 0.2750250

5.25 A1BG 61.093102 0.13645896 0.176779 0.2401909 0.3716156

5.26 A1BG-AS1 175.665807 -0.04046122 0.116928 0.6789312 0.7840468

5.27 A1CF 0.237692 0.00626747 0.210057 0.7945932 NA

5.28 A2LD1 89.617985 0.29017719 0.195395 0.0333343 0.0774672

5.29 A2M 5.860084 -0.09619722 0.233561 0.1057823 0.1976308

```
names(res_table0E)
```

5.30 [1] “baseMean” “log2FoldChange” “lfcSE” “pvalue”

5.31 [5] “padj”

5.31.1 Why are some p-values NA?

DESeq2 sets p-values to NA in three cases:

1. **Zero counts:** All samples have zero counts for the gene (can’t test what doesn’t exist)
2. **Outliers detected by Cook’s distance:** One sample is pulling the results too much (quality control to prevent false positives)
3. **Low mean counts (independent filtering):** Gene filtered out for having too few reads overall

This is DESeq2 being conservative - better to say “we don’t know” than to give you a misleading result.

5.31.2 Multiple Test Correction

The problem: When testing 20,000 genes, even if NONE are truly different, we’d expect ~1,000 to have $p < 0.05$ just by chance!

Example: - You test 20,000 genes - Find 3,000 with $p < 0.05$ - But ~1,000 are false positives (33% of your “hits”!)

This is called the **multiple testing problem**.

5.31.3 The Solution: Adjusted p-values (padj)

DESeq2 uses the **Benjamini-Hochberg (BH) method** to control the False Discovery Rate (FDR).

What does $\text{padj} < 0.05$ mean?

“Among all genes I call significant, I expect only 5% to be false positives.”

Example:

Threshold	Genes significant	Expected false positives
$p < 0.05$ (unadjusted)	3,000	~1,000 (33%)
$\text{padj} < 0.05$ (BH adjusted)	500	~25 (5%)

Much better! We go from 3,000 genes (many false) to 500 genes (mostly real).

The rule: Always use `padj`, not `pvalue`, to determine significance.

```
# Quick check - how many genes at different thresholds?
sum(res_table0E$pvalue < 0.05, na.rm = TRUE) # Unadjusted
```

5.32 [1] 7555

```
sum(res_tableOE$padj < 0.05, na.rm = TRUE)      # Adjusted - much fewer!
```

5.33 [1] 6464

Now let's extract the significant genes using both padj and fold change thresholds.

5.33.1 Extracting Significant Differentially Expressed Genes

In some cases, using only the FDR threshold doesn't sufficiently reduce the number of significant genes, making it difficult to extract biologically meaningful results. To increase stringency, we can apply an additional fold change threshold.

Let's start by setting the thresholds for both adjusted p-value ($FDR < 0.05$) and log2 fold change ($|\log_2FC| > 0.58$, corresponding to a 1.5-fold change):

```
### Set thresholds
padj.cutoff <- 0.05
lfc.cutoff <- 0.58 # log 1.5 fold change
```

Next, we'll convert the results table to a tibble for easier subsetting:

```
res_tableOE_tb <- res_tableOE %>%
  data.frame() %>%
  rownames_to_column(var = "gene") %>%
  as_tibble()
```

Now, we can filter the table to retain only the genes that meet the significance and fold change criteria:

```
sigOE <- res_tableOE_tb %>%
  dplyr::filter(padj < padj.cutoff &
    abs(log2FoldChange) > lfc.cutoff)

# Save the results for future use
saveRDS(sigOE, "data/sigOE.RDS")
```

Exercise points = +2

How many genes are differentially expressed in the **Overexpression vs. Control** comparison based on the criteria we just defined?

Does this reduce our results? How many genes are differentially expressed in the Overexpression vs. Control comparison based on our criteria, and what percentage is this of all genes tested?

STOPPED HERE

5.33.2 MOV10 DE Analysis: Control vs. Knockdown

After examining the overexpression results, let's move on to the comparison between **Control vs. Knockdown**. We'll use contrasts in the `results()` function to extract the results table and store it in the `res_tableKD` variable.

```
# define contrast
contrast_kd <- c("sampletype",
                 "MOV10_knockdown",
                 "control")

# extract results table
res_tableKD_unshrunk <- results(dds,
                               contrast = contrast_kd)

resultsNames(dds)
```

5.34 [1] “Intercept”

5.35 [2] “sampletype_MOV10_knockdown_vs_control”

5.36 [3] “sampletype_MOV10_overexpression_vs_control”

```
# shrink log2 fold changes
res_tableKD <- lfcShrink(dds = dds,
                        coef = "sampletype_MOV10_knockdown_vs_control",
                        res = res_tableKD_unshrunk)

## Save results for future use
saveRDS(res_tableKD, file = "data/res_tableKD.RDS")
```

Next, let's perform the same analysis for the **Control vs. Mov10 knockdown** comparison. We'll use the same thresholds for adjusted p-value ($FDR < 0.05$) and log2 fold change ($|\log_2FC| > 0.58$).

```
res_tableKD_tb <- res_tableKD %>%
  data.frame() %>%
  rownames_to_column(var = "gene") %>%
  as_tibble()

sigKD <- res_tableKD_tb %>%
  dplyr::filter(padj < padj.cutoff &
               abs(log2FoldChange) > lfc.cutoff)

# We'll save this object for use in the homework
saveRDS(sigKD, "data/sigKD.RDS")
```

Exercise points = +1

How many genes are differentially expressed in the **Knockdown compared to Control** comparison based on our padj.cutoff & lfc.cutoff?

With our subset of significant genes identified, we can now proceed to visualize the results and explore patterns of differential expression.

Chapter 6

Visualizing RNA-seq results

During this lesson, we will get you started with some basic and more advanced visualization techniques to explore and interpret the results of your differential expression analysis.

Let's start by loading a few libraries:

```
# load libraries
library(tidyverse)
library(ggplot2)
library(ggrepel)
library(RColorBrewer)
library(DESeq2)
library(pheatmap)
library(dplyr)
```

We will be working with three different data objects we have already created in earlier lessons:

- Metadata for our samples (a dataframe): `meta`
- Normalized expression data for every gene in each of our samples (a matrix): `normalized_counts`
- Tibble versions of the DESeq2 results we generated in the last lesson: `res_tableOE_tb` and `res_tableKD_tb`

Let's create tibble objects from the `meta` and `normalized_counts` data frames before we start plotting. This will enable us to use the `tidyverse` functionality more easily.

Check to make sure the column names of the `normalized_counts` matrix are the same as the row names of the `meta` data frame. If they are not, you will need to reorder the columns of `normalized_counts` to match the row names of `meta`.

```
# Read in the metadata
meta <- read.table("data/Mov10_full_meta.txt",
                  header=T, row.names=1)

# Create a tibble for meta data
mov10_meta <- meta %>%
  rownames_to_column(var="samplename") %>%
  as_tibble()
```

```
# read in the normalized counts
normalized_counts <- read.delim("data/normalized_counts.txt", row.names=1)

all(mov10_meta$samplename == colnames(normalized_counts))
```

6.1 [1] TRUE

```
# Create a tibble for normalized_counts
normalized_counts <- normalized_counts %>%
  data.frame() %>%
  rownames_to_column(var="gene") %>%
  as_tibble()
```

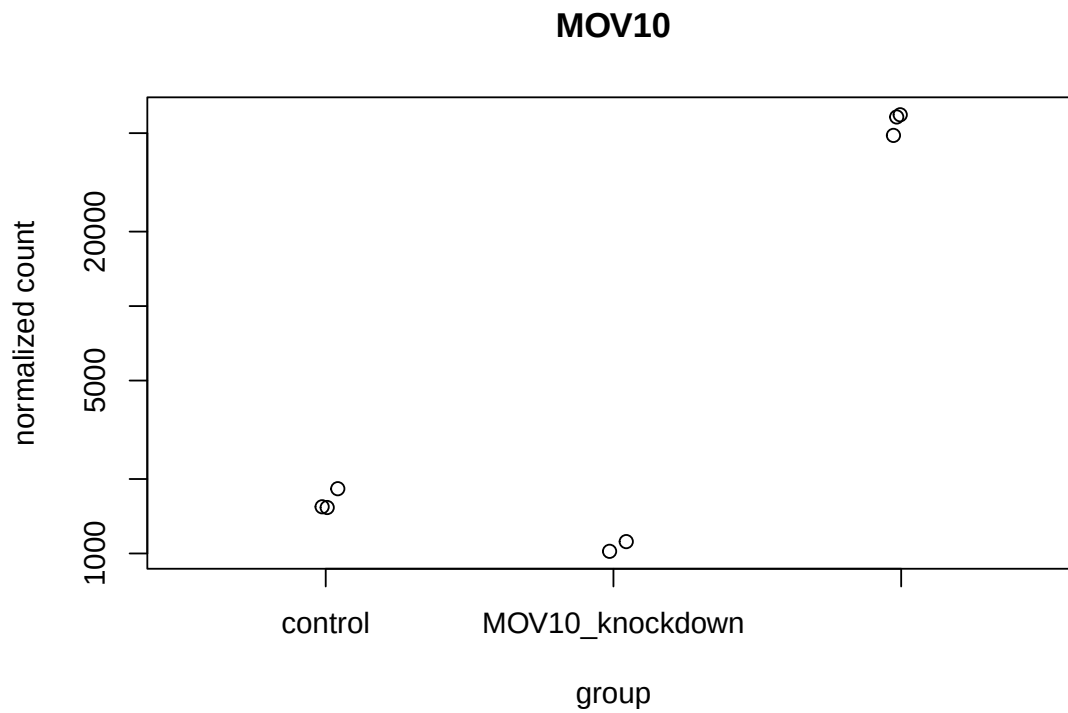
6.1.1 Plotting significant DE genes

One way to visualize results would be to simply plot the expression data for a handful of genes. We could do that by picking out specific genes of interest or selecting a range of genes.

6.1.1.1 Using DESeq2 plotCounts() to plot expression of a single gene

To pick out a specific gene of interest to plot, for example MOV10, we can use the `plotCounts()` from DESeq2. `plotCounts()` is a function that allows us to plot the normalized counts for a single gene across all samples.

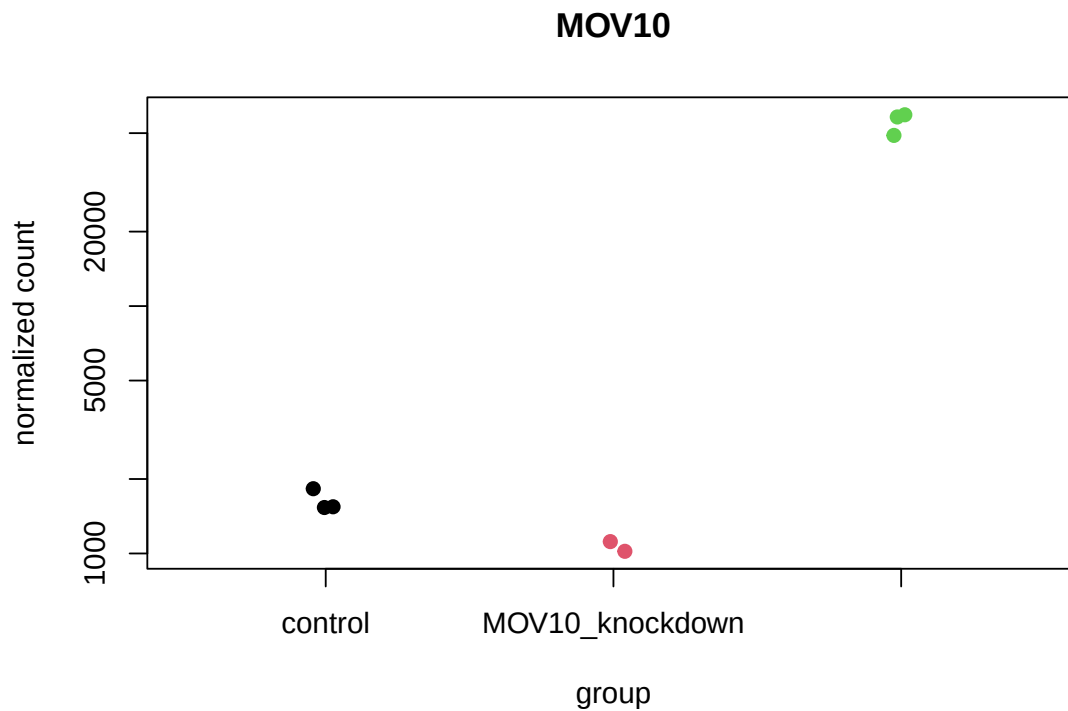
```
plotCounts(dds,
  gene="MOV10",
  intgroup="sampletype")
```



```
# We can also color the points by sample type
samplotype = as.factor(mov10_meta$samplotype)

library(RColorBrewer)

plotCounts(dds, gene="MOV10",
            intgroup="samplotype",
            col = as.numeric(samplotype),
            pch = 19)
```



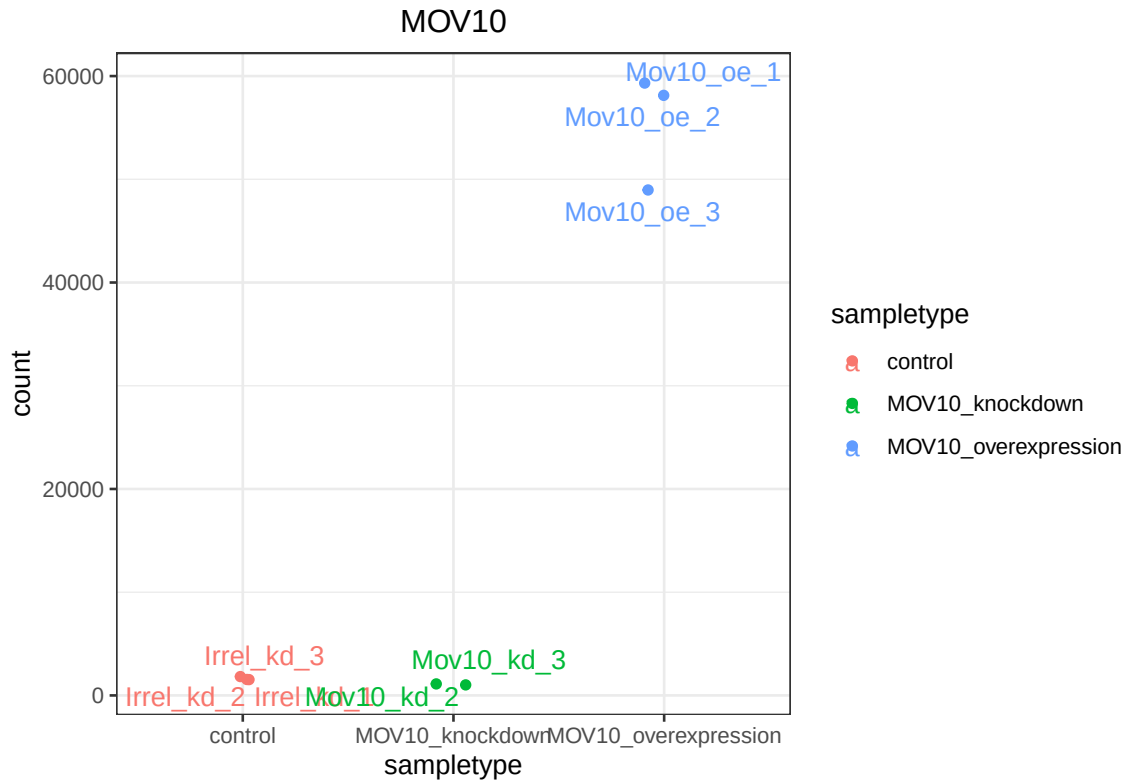
This function only allows for plotting the counts of a single gene at a time.

6.1.1.2 Using ggplot2 to plot expression of a single gene

We can also use ggplot2 to plot the MOV10 counts. We can save the output of `plotCounts()` to a variable specifying the `returnData=TRUE` argument. This will save the normalized counts for the gene MOV10 to a data frame object. We can then use ggplot2 to plot the normalized counts for MOV10 across all samples.

```
# Save `plotCounts()` to a data frame object
d <- plotCounts(dds, gene = "MOV10", intgroup = "samplotype",
  returnData=TRUE)

# Plot using ggplot2
ggplot(d, aes(x = samplotype, y = count, color = samplotype)) +
  geom_point(position = position_jitter(w = 0.1, h = 0)) +
  geom_text_repel(aes(label = rownames(d))) +
  theme_bw() +
  ggtitle("MOV10") +
  theme(plot.title = element_text(hjust = 0.5))
```



Note that in the plot below (code above), we are using `geom_text_repel()` from the `ggrepel` package to label our individual points on the plot.

6.1.1.3 Using ggplot2 to plot multiple genes (e.g. top 20)

Often it is helpful to check the expression of multiple genes of interest at the same time. This often first requires some data wrangling.

We are going to plot the normalized count values for the **top 20 differentially expressed genes (by padj values)**.

To do this, we first need to determine the gene names of our top 20 genes by ordering our results and extracting the top 20 genes (by padj values):

```
res_tableOE <- readRDS("data/res_tableOE.RDS")

res_tableOE_tb <- res_tableOE %>%
  data.frame() %>%
  rownames_to_column(var="gene") %>%
  as_tibble()

## Order results by padj values
top20_sigOE_genes <- res_tableOE_tb %>%
  arrange(padj) %>%
  #Arrange rows by padj values
  pull(gene) %>%
  #Extract character vector of ordered genes
```

```
head(n=20)
#Extract the first 20 genes
```

Then, we can extract the normalized count values for these top 20 genes:

```
## Normalized counts for top 20 significant genes
top20_sigOE_norm <- normalized_counts %>%
  filter(gene %in% top20_sigOE_genes)
```

Now that we have the normalized counts for each of the top 20 genes for all 8 samples, to plot using `ggplot()`, we need to `pivot_longer` `top20_sigOE_norm` from a wide format to a long format so the counts for all samples will be in a single column to allow us to give `ggplot` the one column with the values we want it to plot:

Current data format (wide): Your data has one row per gene and separate columns for each sample (like `Sample1`, `Sample2`, `Sample3`, etc.). This is convenient for looking at, but `ggplot` needs the data structured differently.

What `ggplot` needs (long format): `ggplot` works best when all the values you want to plot are in a single column, with another column identifying which sample each value came from.

What `pivot_longer` does: It transforms your data from:

Gene Sample1 Sample2 Sample3 GeneA 100 150 120 GeneB 200 180 220

To: Gene Sample Count GeneA Sample1 100 GeneA Sample2 150
GeneA Sample3 120 GeneB Sample1 200 GeneB Sample2 180 GeneB Sample3 220

Why this helps: Now `ggplot` can use the “Count” column for the y-axis and color/group by the “Sample” column to create your visualization.

Simple analogy: Think of it like converting a spreadsheet where data goes across columns into a list where everything is stacked vertically - making it easier for `ggplot` to understand what to plot.

The `pivot_longer()` function in the **tidyr** package will perform this operation and will output the normalized counts for all genes for *Mov10_oe_1* listed in the first 20 rows, followed by the normalized counts for *Mov10_oe_2* in the next 20 rows, so on and so forth.

	Mov10_oe_1	Mov10_oe_2	Mov10_oe_3	Irrel_kd_1	Irrel_kd_2	Irrel_kd_3
MOV10	59319.6663	58130.0281	48964.1381	1544.010015	1533.405731	1826.761190
H1FO	13694.1471	13185.0041	12283.1157	4280.106238	4296.860505	4787.558609
HSPA6	491.8408	444.4083	384.0865	7.127571	3.116678	6.686534
HIST1H...	2666.4264	2745.8719	2610.5638	931.929876	920.458996	928.090971
TXNIP	7154.5781	7528.1688	8113.2527	2722.732028	2846.566195	2915.328985
NEAT1	29542.0743	28240.4493	30082.6924	14847.620827	15355.874063	16234.905450
KLF10	2183.7400	2232.7499	2433.0577	931.929876	1020.192702	941.464039
INSIG1	16559.4736	18116.3295	20018.4028	7361.889643	7576.644986	7888.773250
NR1D1	1663.6038	1775.8483	1502.6809	563.078090	525.679743	549.633125
WDFY1	1778.4499	1756.2158	1895.9487	834.816725	857.086537	857.213706
HSPA1A	40305.9818	43833.1110	43926.6375	24397.674692	22399.567057	22086.960331
HMGCS1	15683.1481	15828.2516	17752.1397	8750.874994	8411.914775	8715.228898
HSPA1B	39351.4278	38343.1519	41718.5229	24109.899023	21831.292711	21228.409318
LAMC1	6939.8658	6017.3592	6720.7480	3579.822411	3269.395553	3448.914428
TMCO1	2075.5517	1978.4199	2156.0870	1100.318735	1030.581630	1060.484351
ADAMTS1	13252.2393	13267.9960	14442.2631	6717.735436	6454.640792	6801.542761
ZFP36L1	1838.3695	1929.3387	1773.5307	994.296120	947.470208	949.487881
C1orf95	385.3169	399.7890	385.6167	96.222205	98.694813	106.984550
MRC2	752.3251	769.2368	661.0572	239.664567	237.906445	243.389851
COL2A1	4066.2171	4134.4246	3825.5624	2186.382329	2194.141534	2356.334712



	gene	samplename	normalized_c
1	MOV10	Mov10_oe_1	59319.6
2	H1FO	Mov10_oe_1	13694.1
3	HSPA6	Mov10_oe_1	491.8
4	HIST1H1C	Mov10_oe_1	2666.4
5	TXNIP	Mov10_oe_1	7154.5
6	NEAT1	Mov10_oe_1	29542.0
7	KLF10	Mov10_oe_1	2183.7
8	INSIG1	Mov10_oe_1	16559.4
9	NR1D1	Mov10_oe_1	1663.6
10	WDFY1	Mov10_oe_1	1778.4
11	HSPA1A	Mov10_oe_1	40305.9
12	HMGCS1	Mov10_oe_1	15683.1
13	HSPA1B	Mov10_oe_1	39351.4
14	LAMC1	Mov10_oe_1	6939.8
15	TMCO1	Mov10_oe_1	2075.5
16	ADAMTS1	Mov10_oe_1	13252.2
17	ZFP36L1	Mov10_oe_1	1838.3
18	C1orf95	Mov10_oe_1	385.3
19	MRC2	Mov10_oe_1	752.3
20	COL2A1	Mov10_oe_1	4066.2
21	MOV10	Mov10_oe_2	58130.0
22	H1FO	Mov10_oe_2	13185.0
23	HSPA6	Mov10_oe_2	444.4
24	HIST1H1C	Mov10_oe_2	2745.8
25	TXNIP	Mov10_oe_2	7528.1
26	NEAT1	Mov10_oe_2	28240.4
27	KLF10	Mov10_oe_2	2232.7
28	INSIG1	Mov10_oe_2	18116.3
29	NR1D1	Mov10_oe_2	1775.8
30	WDFY1	Mov10_oe_2	1756.2

```
# Pivot the data frame
pivoted_top20_sigOE <- top20_sigOE_norm %>%
  pivot_longer(colnames(top20_sigOE_norm)[2:9],
    names_to = "samplename",
    values_to = "normalized_counts")

## Check the column header in the "pivoted" data frame
head(pivoted_top20_sigOE)
```


6.2 # A tibble: 6 x 3

6.3 gene samplename normalized__counts

6.4

6.5 1 ADAMTS1 Irrel_kd__1 6718.

6.6 2 ADAMTS1 Irrel_kd__2 6455.

6.7 3 ADAMTS1 Irrel_kd__3 6802.

6.8 4 ADAMTS1 Mov10_kd__2 6869.

6.9 5 ADAMTS1 Mov10_kd__3 8731.

6.10 6 ADAMTS1 Mov10_oe__1 13252.

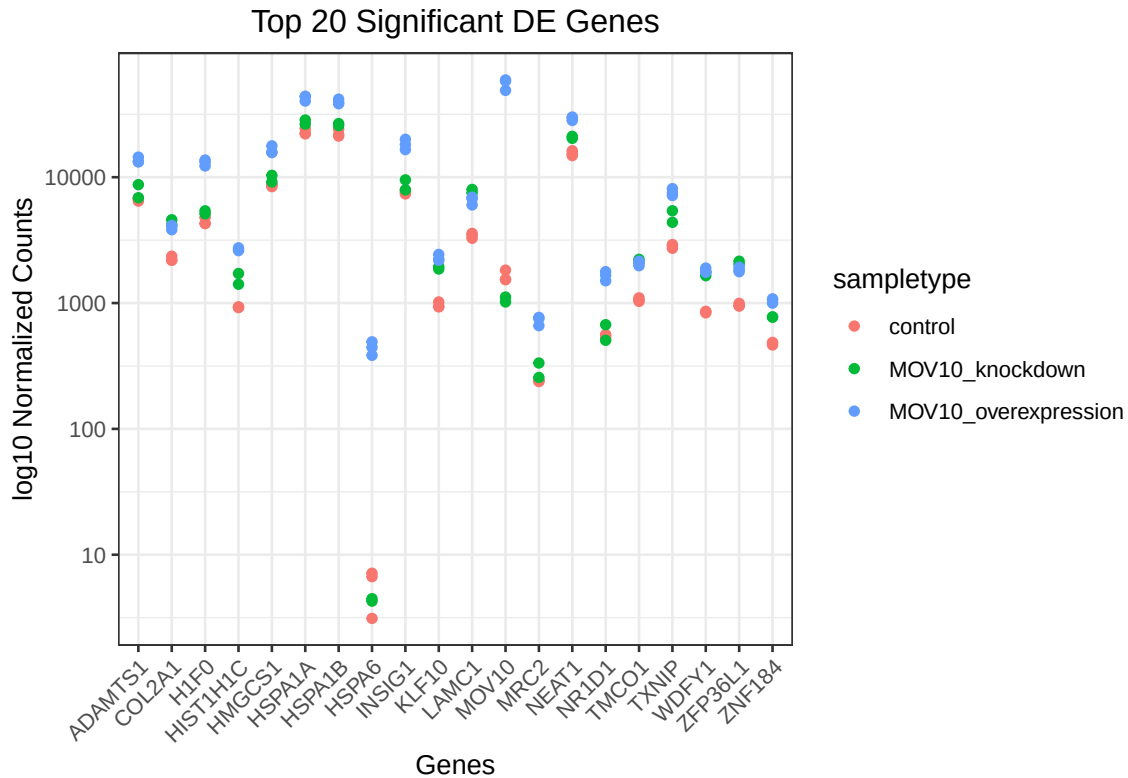
Now, if we want our counts colored by sample group, then we need to combine the metadata information with the melted normalized counts data into the same data frame for input to `ggplot()`:

`inner_join(x,y)` will merge 2 data frames by the `colname` in `x` that matches a column name in `y` in this case `samplename` column.

```
pivoted_top20_sigOE <- inner_join(
  mov10_meta,
  pivoted_top20_sigOE) #adds the sampletype column
```

Now that we have a data frame in a format that can be utilized by `ggplot` easily, let's plot!

```
## plot using ggplot2
ggplot(pivoted_top20_sigOE) +
  geom_point(aes(x = gene, y = normalized_counts,
    color = sampletype)) +
  scale_y_log10() +
  xlab("Genes") +
  ylab("log10 Normalized Counts") +
  ggtitle("Top 20 Significant DE Genes") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 45,
    hjust = 1)) +
  theme(plot.title = element_text(hjust = 0.5))
```



6.10.1 Heatmap

We will plot a heatmap of the `sigOE` genes to visualize their expression across all samples using `pheatmap()`.

```
sigOE = readRDS("data/sigOE.RDS")

norm_OEsig <- normalized_counts[,c(1,2:4,7:9)] %>%
  filter(gene %in% sigOE$gene) %>%
  data.frame() %>%
  column_to_rownames(var = "gene")

# We've moved the gene column back to the rownames to get a matrix required for pheatmap
```

Now let's draw the heatmap using `pheatmap`. We can also add annotations to the heatmap to indicate the sample type. The `annotation_col` argument in `pheatmap()` requires a data frame with the same row names as the column names of the matrix used to generate the heatmap.

```
### Annotate our heatmap (optional)
annotation <- mov10_meta %>%
  filter(samplename %in% colnames(norm_OEsig)) %>%
  dplyr::select(samplename, sampletype) %>%
  data.frame(row.names = "samplename")

annotation$sampletype = factor(annotation$sampletype)
```

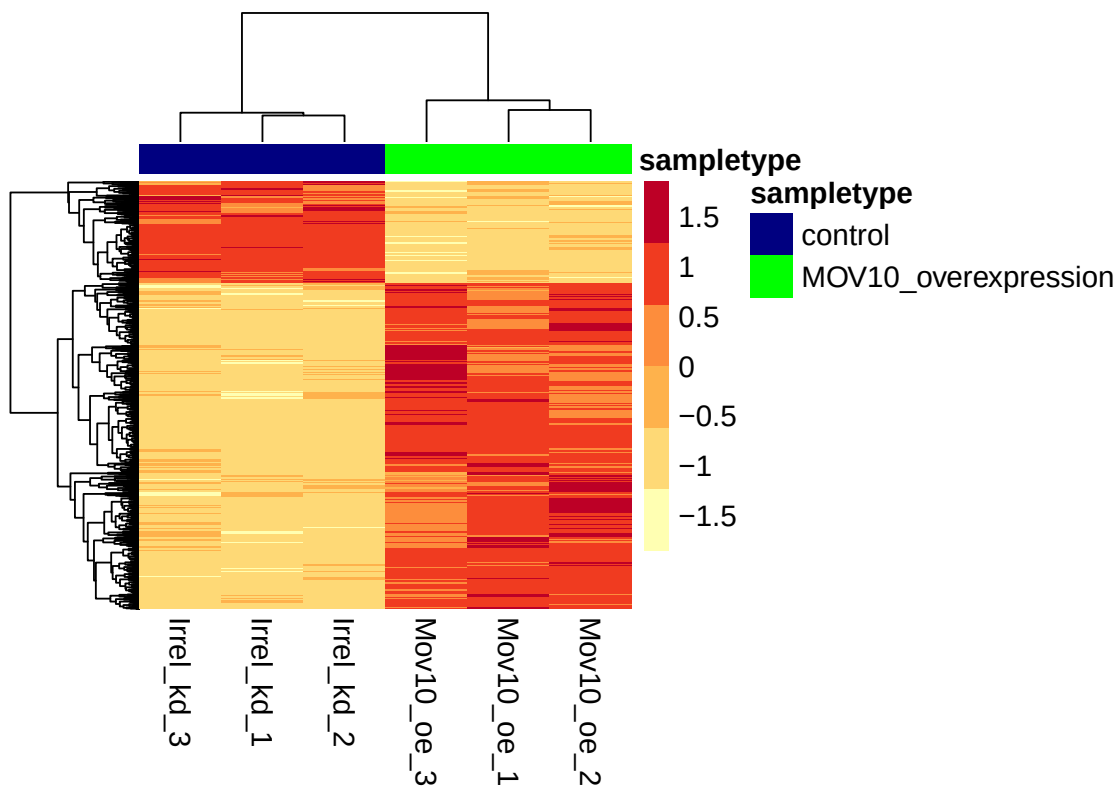
```

### Set a color palette
heat_colors <- brewer.pal(6, "YlOrRd")

### Set annotation colors
ann_colors = list(
  samplotype = c(control="navy",
    MOV10_overexpression= "green"))

### Run pheatmap
pheatmap(norm_OEsig,
  annotation_colors = ann_colors,
  color = heat_colors,
  cluster_rows = T,
  show_rownames = F,
  annotation_col = annotation,
  border_color = NA,
  fontsize = 12,
  scale = "row",
  fontsize_col = 12)

```



NOTE: There are several additional arguments we have included in the function for aesthetics. One important one is `scale="row"`, in which Z-scores are plotted, rather than the actual normalized count value.

6.10.2 Volcano plots

A commonly used plot in RNA-seq analysis is the **volcano plot**, where log transformed adjusted p-values are plotted on the y-axis and log2 fold change values on the x-axis.

To generate a volcano plot, we first need to have a column in our results data indicating whether or not the gene is considered differentially expressed based on p-adjusted values.

```
## create a column for thresholding in the results table
res_tableOE_tb <- res_tableOE_tb %>%
  mutate(threshold_OE = padj < 0.05 &
         abs(log2FoldChange) >= 0.58)

names(res_tableOE_tb)
```

6.11 [1] “gene” “baseMean” “log2FoldChange” “lfcSE”

6.12 [5] “pvalue” “padj” “threshold_OE”

```
names(sigOE)
```

6.13 [1] “gene” “baseMean” “log2FoldChange” “lfcSE”

6.14 [5] “pvalue” “padj”

```
table(res_tableOE_tb$threshold_OE)
```

6.15

6.16 FALSE TRUE

6.17 18793 955

Now we can start plotting. The `geom_point` object is most applicable, as this is essentially a scatter plot:

```
ggplot(res_tableOE_tb) +
  geom_point(aes(x = log2FoldChange,
                y = -log10(padj), colour = threshold_OE)) +
  ggtitle("Mov10 overexpression") +
```

```

xlab("log2 fold change") +
ylab("-log10 adjusted p-value") +
theme_bw() +
theme(legend.position = "none",
plot.title = element_text(size = rel(1.5),
                           hjust = 0.5),
axis.title = element_text(size = rel(1.25)))

```



What if we also wanted to know where the top 10 genes (lowest padj) in our DE list are located on this plot? We could label those dots with the gene name on the volcano plot using `geom_text_repel()`.

First, we need to order the `res_tableOE` tibble by `padj`, and add an additional column to it, to include on those gene names we want to use to label the plot.

```

## Create a column to indicate which genes to label
res_tableOE_tb <- res_tableOE_tb %>%
  arrange(padj) %>%
  mutate(genelabels = "")

res_tableOE_tb$genelabels[1:10] <- res_tableOE_tb$gene[1:10]

head(res_tableOE_tb$genelabels, 20)

```

6.18 [1] “MOV10” “H1F0” “HIST1H1C” “TXNIP” “NEAT1” “KLF10”

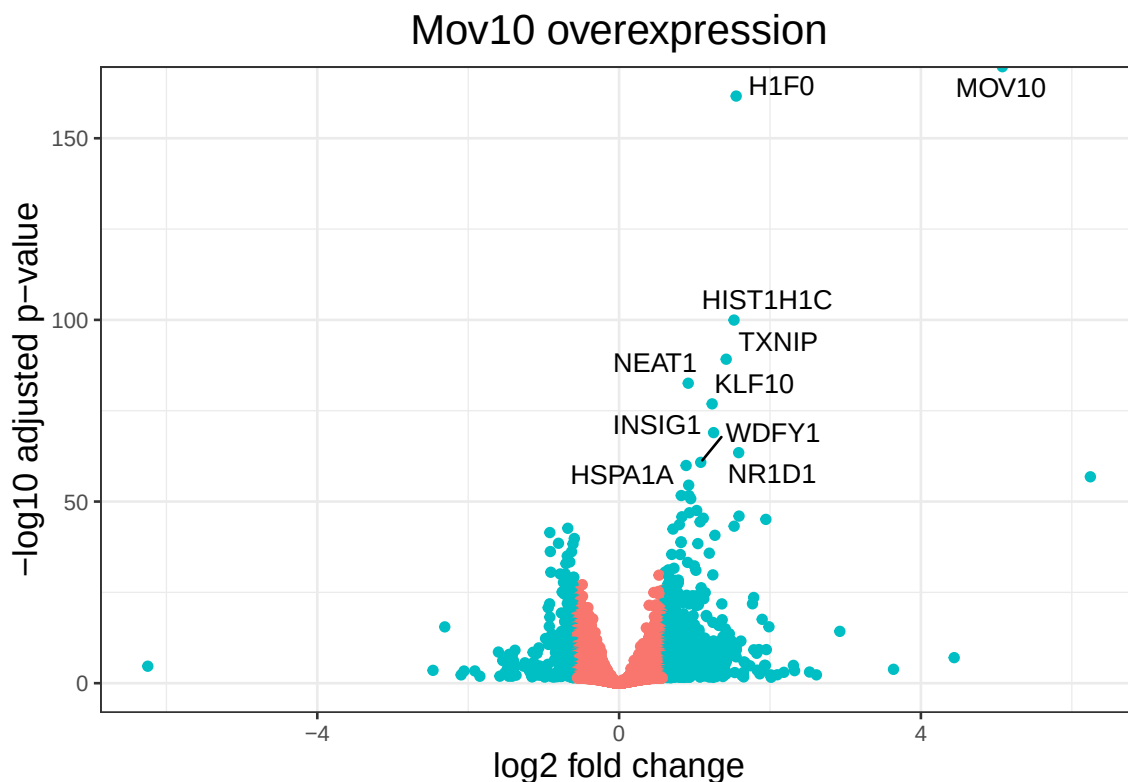
6.19 [7] “INSIG1” “NR1D1” “WDFY1” “HSPA1A” “ ” “ ”

6.20 [13] “ ” “ ” “ ” “ ” “ ” “ ” “ ”

6.21 [19] “ ” “ ” “ ”

Next, we plot it as before with an additional layer for `geom_text_repel()` wherein we can specify the column of gene labels we just created.

```
ggplot(res_tableOE_tb, aes(x = log2FoldChange,
  y = -log10(padj))) +
  geom_point(aes(colour = threshold_OE)) +
  geom_text_repel(aes(label = genelabels)) +
  ggtitle("Mov10 overexpression") +
  xlab("log2 fold change") +
  ylab("-log10 adjusted p-value") +
  theme_bw() +
  theme(legend.position = "none",
  plot.title = element_text(size = rel(1.5),
    hjust = 0.5),
  axis.title = element_text(size = rel(1.25)))
```



Chapter 7

Summary of DGE workflow

Homework: First save a copy of this file `lessons/07-DGE_summarizing_workflow.Rmd` in your `student_notes` folder as `'<lastname>_07-DGE_summarizing_workflow.Rmd'`. Then modify the file to analyze the MOV dataset, starting with `Mov10_full_counts.txt` and `Mov10_full_meta.txt` in your data folder. Compare the “MOV10_knockdown” to the “control”. Include a heatmap** and a volcano plot. Don't forget to read the data in first. points = +6

```
## Setup
library(DESeq2)
```

We have detailed the various steps in a differential expression analysis workflow, providing theory with example code. To provide a more succinct reference for the code needed to run a DGE analysis, we have summarized the steps in an analysis below:

7.0.1 1. Import data into dds object (be careful of the names of the count file and metadata): points = +2

```
# Check that the row names of the metadata equal the column names
# of the **raw counts** data
all(colnames(raw_counts) == rownames(metadata))

# Create DESeq2Dataset object
dds <- DESeqDataSetFromMatrix(countData = raw_counts, colData = metadata, design = ~ condition)
```

7.0.2 2. Exploratory data analysis (PCA & heirarchical clustering) - identifying outliers and sources of variation in the data:

```
# Transform counts for data visualization
rld <- rlog(dds, blind = TRUE)

# Plot PCA
plotPCA(rld, intgroup = "samplotype")
```

```
# Extract the rlog matrix from the object
rld_mat <- assay(rld)

# Compute pairwise correlation values
rld_cor <- cor(rld_mat)

# Plot heatmap
pheatmap(rld_cor)
```

7.0.3 3. Run DESeq2 and normalize counts. Write the normalized counts to a file using write.table: points = +2

```
# Run DESeq2 differential expression analysis
dds <- DESeq(dds)

# Output normalized counts to save as a file to access outside RStudio
normalized_counts <- counts(dds, normalized = TRUE)

write.table(normalized_counts, file = "data/normalized_counts.txt",
            sep = "\t", quote = F, col.names = NA)
```

7.0.4 4. Check the fit of the dispersion estimates:

```
# Plot dispersion estimates
plotDispEsts(dds)
```

7.0.5 5. Create contrasts to perform Wald testing on the shrunken log2 foldchanges between specific conditions: points = +2

```
# Output results of Wald test for contrast
contrast <- c("condition", "level_to_compare", "base_level")

res <- results(dds, contrast = contrast)

coef = resultsNames(dds)

res_table <- lfcShrink(dds, coef = coef[2], res = res,
                    type = "apeglm")
```

7.0.6 6. Output significant results: pointts = +1


```

### Set thresholds
padj.cutoff <- 0.05
lfc.cutoff <- 0.58 ## change in expression of 1.5

# Turn the results object into a data frame
res_df <- res %>%
  data.frame() %>%
  rownames_to_column(var = "gene")

# Subset the significant results
sig_res <- dplyr::filter(res_df, padj < padj.cutoff &
  abs(log2FoldChange) > lfc.cutoff)

```

7.0.7 7. Visualize results: volcano plots, heatmaps, normalized counts plots of top genes, etc. points = +3

7.0.8 8. Make sure to output the versions of all tools used in the DE analysis:

```
sessionInfo()
```


7.1 R version 4.5.1 (2025-06-13)

7.2 Platform: x86_64-apple-darwin20

7.3 Running under: macOS Sequoia 15.5

7.4

7.5 Matrix products: default

7.6 BLAS: /Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/

7.7 LAPACK: /Library/Frameworks/R.framework/Versions/4.5-x86_64/Resour
LAPACK version 3.12.1

7.8

7.9 locale:

7.10 [1] en_CA.UTF-8/en_CA.UTF-8/en_CA.UTF-8/C/en_CA.UTF-
8/en_CA.UTF-8

7.11

7.12 time zone: America/Los_Angeles

7.13 tzcode source: internal

7.14

7.15 attached base packages:

7.16 [1] stats4 stats graphics grDevices utils datasets methods

7.17 [8] base

7.18

7.19 other attached packages:

Chapter 8

Functional analysis of RNA-seq data

8.1 Understanding Gene Ontology (GO) Enrichment

When you perform an RNA-Seq experiment, you often end up with a list of genes that are **up- or down-regulated** in your condition of interest (for example, after MOV10 knockdown).

On their own, these gene lists can be hard to interpret — we want to know *what biological processes these genes are involved in*.

8.1.1 What is the Gene Ontology?

The **Gene Ontology (GO)** provides a structured vocabulary that describes gene functions in three categories:

Category	Meaning	Example
Biological Process (BP)	The biological goal the gene contributes to	<i>cell cycle, DNA repair</i>
Molecular Function (MF)	The activity of the gene product	<i>ATP binding, enzyme activity</i>
Cellular Component (CC)	Where the gene product acts in the cell	<i>nucleus, mitochondrion</i>

Each GO term connects to one or more genes. These connections come from the work of many scientists who have experimentally tested or observed gene functions. Each gene-GO association includes an **evidence code** (like IDA for direct assay, IMP for mutant phenotype) indicating how the relationship was established—you’ll see these codes in enrichment results.

8.1.2 What is GO Enrichment Analysis?

GO enrichment analysis asks whether certain GO terms occur **more often in your list of genes than expected by chance**.

For example, if 10% of all genes in the genome are involved in “DNA repair,” but 30% of your up-regulated genes are, that process is *enriched*.

Statistical tests such as the *hypergeometric test* or *Fisher's exact test* are used to determine whether this enrichment is significant.

In essence, GO enrichment helps you summarize large gene lists into meaningful biological patterns.

8.1.3 Ontology and GO Terms

Ontology: An ontology is a classification of things that are detectable or directly observable and the relationships between those things.

The **Gene Ontology (GO)** project provides an ontology of defined terms representing gene product properties. These terms are connected to each other via formally defined relationships, allowing for consistent annotation and analysis of gene functions across different species and research areas.

GO term hierarchy Some gene products are well-researched, with vast quantities of data available regarding their biological processes and functions. However, other gene products have very little data available about their roles in the cell.

For example, the protein, “p53”, would contain a wealth of information on its roles in the cell, whereas another protein might only be known as a “membrane-bound protein” with no other information available.

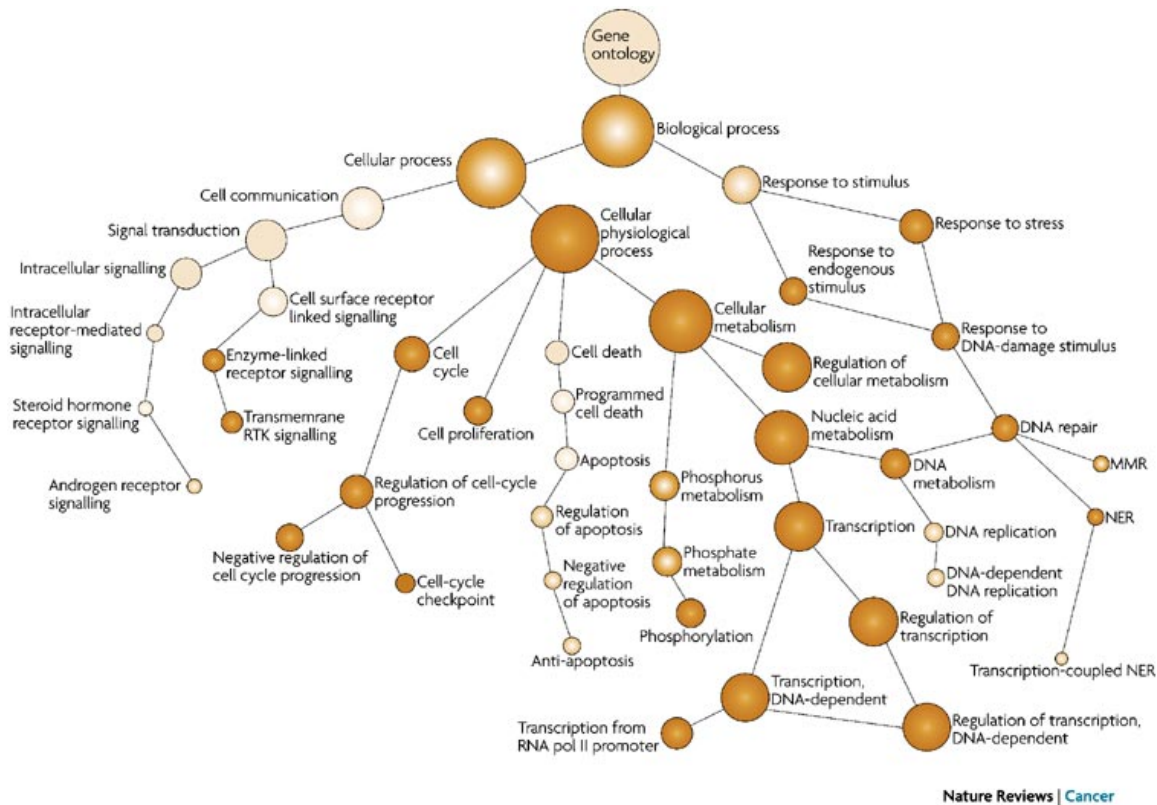
The GO ontologies were developed to describe and query biological knowledge with differing levels of information available. To do this, GO ontologies are loosely hierarchical, ranging from general, ‘parent’, terms to more specific, ‘child’ terms. The GO ontologies are “loosely” hierarchical since ‘child’ terms can have multiple ‘parent’ terms.

Some genes with less information may only be associated with general ‘parent’ terms or no terms at all, while other genes with a lot of information be associated with many terms.

8.1.4 GO Terms (Gene Ontology Terms)

The GO terms describe three aspects of gene products:

- **Biological Process:** Refers to the biological objectives achieved by gene products, such as “cell division” or “response to stress.” Inferred from Direct Assay (IDA)



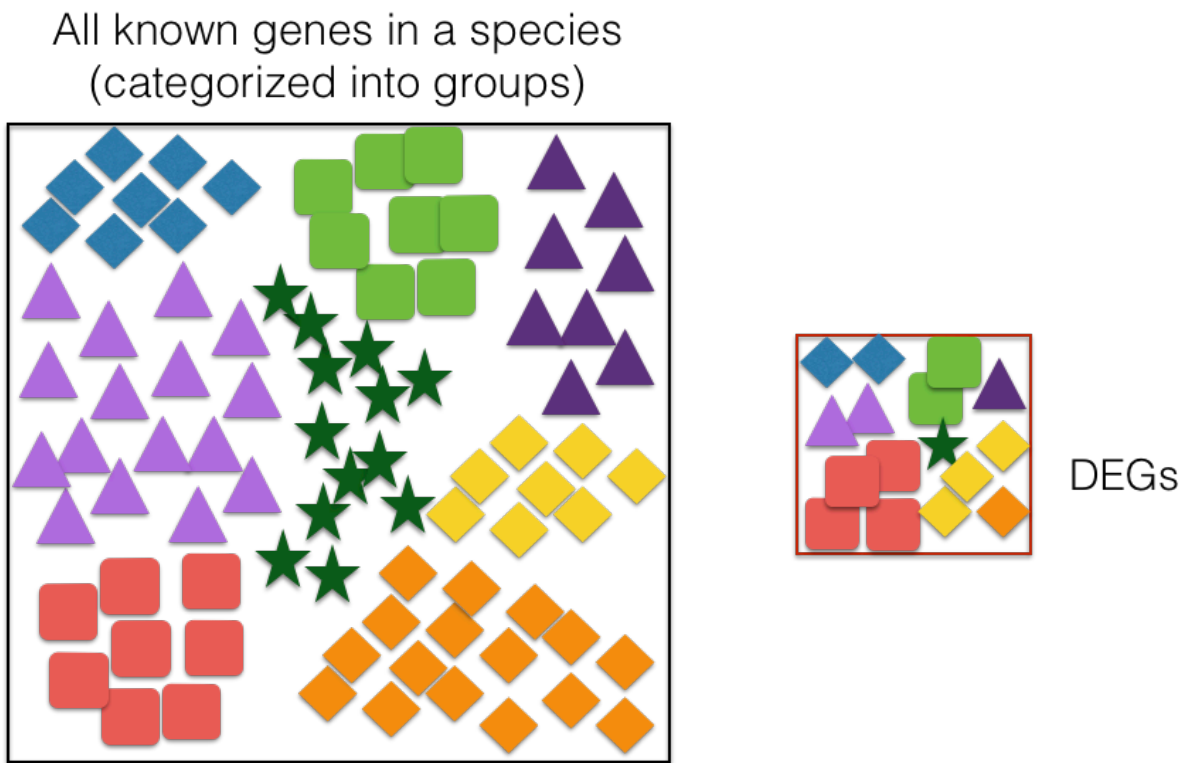
- **Molecular Function:** Describes the biochemical activities at the molecular level, such as “kinase activity” or “DNA binding.”
- **Cellular Component:** Indicates where gene products are located within the cell, like “nucleus” or “mitochondrion.”

8.1.5 GO Enrichment Methods

GO enrichment methods help interpret differentially expressed gene lists from RNA-seq data and include the following:

1. Overrepresentation Analysis (ORA):

- ORA assesses whether specific GO terms are over-represented in a gene list compared to the background set of genes (e.g., all genes in the genome). Over-represented GO terms indicate biological functions or processes that are significantly associated with the genes of interest.



Genes categories	Organism-specific Background	DE results	Over-represented?
Functional category 1	35/13000	25/1000	Likely
Functional category 2	56/13000	4/1000	Unlikely
Functional category 3	90/13000	8/1000	Unlikely
Functional category 4	15/13000	10/1000	Likely
...			
...			

• **Hypergeometric Test:**

The hypergeometric test in GO enrichment analysis is used to determine whether a set of genes is enriched for specific GO terms compared to the background set of genes. The test calculates the probability of observing the overlap between the query gene set and the gene set associated with a particular GO term, given the total number of genes in the dataset and the number of genes in the background set.

Test Analogy: The jar contains:

Red balls

Black balls

The hypergeometric test asks: “If I randomly drew this many balls from the jar, what’s the probability I’d get at least this many red balls just by chance?”

So, it’s like drawing colored balls from a jar where:

Red balls = genes that belong to a specific GO term

Black balls = genes that do NOT belong to that GO term

Total balls in jar = all genes in your background (universe)

You draw from the jar:

You draw a handful of balls = your significant genes (e.g., $p_{adj} < 0.05$)

The test checks how surprising it is that so many of your significant genes are red.

If the probability of getting that many red balls (GO term genes) by chance is very low, then the GO term is overrepresented — meaning your experiment is likely enriched for that biological process.

Note for the curious:

[Click to see the mathematical formula](#)

The hypergeometric test is based on counting combinations — the number of ways to draw q or more “successes” (genes with a given GO term) in k draws from a total of N genes, of which m belong to that GO term.

The formula for the cumulative probability of observing **at least** q successes (the upper-tail probability) in a sample of size k from a population with m successes and $N - m$ failures is:

$$P(X \geq q) = \sum_{i=q}^k \frac{\binom{m}{i} \binom{N-m}{k-i}}{\binom{N}{k}}$$

Where:

- **N:** The **total number of genes in the background set** (e.g., all genes in the genome).
- **m:** The **total number of genes in the background annotated with a specific GO term**.
- **k:** The **total number of genes in your gene list of interest**.
- **q:** The **number of genes in your list that are annotated with that specific GO term**.

8.2 Perform the hypergeometric test: $P(X \geq q)$

In R, the `phyper()` function computes the probability of observing this much overlap (or more) between our gene list and a GO term by chance.

Suppose your background population has $N = 20,000$ genes (e.g., the entire genome). Out of these, $m = 500$ genes are associated with a specific GO term (e.g., “cell cycle regulation”). You have a list of $k = 1000$ genes of interest. In this list, $q = 50$ genes are associated with the “cell cycle regulation” GO term.

A small p-value indicates that observing 50 or more genes with this GO term is unlikely by chance alone, suggesting genuine enrichment.

Question: Is 50 more than we'd expect by chance?

```
# Define parameters
N <- 20000      # Total genes in the background
m <- 500        # Genes with the GO term in background
k <- 1000       # Your gene list size (e.g., DE genes)
q <- 50         # Observed overlap with GO term

# Perform the hypergeometric test:  $P(X \geq q)$ 
# Note: We use q-1 because phyper calculates  $P(X > q-1) = P(X \geq q)$ 
p_value <- phyper(q - 1, m, N - m, k, lower.tail = FALSE)
p_value
```

8.3 [1] 2.551199e-06

A small p-value indicates that observing 50 or more genes with this GO term is unlikely by chance alone, suggesting genuine enrichment.

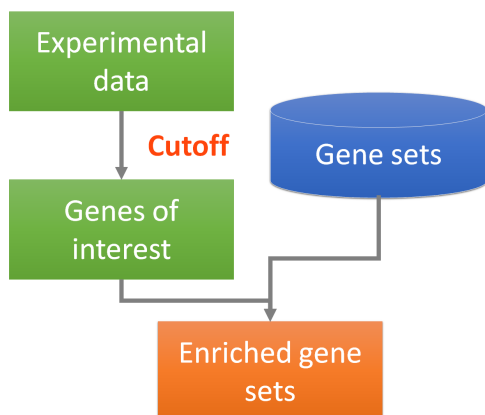
2. Gene Set Enrichment Analysis (GSEA):

GSEA uses the **entire ranked list of genes** (ranked by log2FoldChange or similar metric) rather than just significant ones. It checks whether genes in a pathway tend to cluster at the top or bottom of your ranked list, indicating coordinated expression changes.

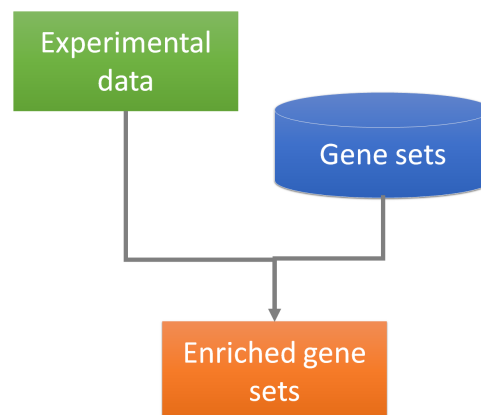
When to use each: - **ORA:** When you have clear significant genes ($\text{padj} < 0.05$) - **GSEA:** When you want to detect subtle coordinated changes across many genes

For this course, we'll focus on **Over-representation Analysis (ORA)** which is more commonly used and straightforward to interpret.

A Over-representation analysis (ORA)



B Gene-set enrichment analysis (GSEA)



8.4 Over-Representation Analysis (ORA)

We'll explore ORA using two different R packages that use the same underlying statistical method (hypergeometric test) but offer different visualization capabilities.

8.4.1 Over-representation Analysis (ORA) using clusterProfiler

```
# Uncomment the following if you haven't yet installed either `G0enrichment` or `org.Hs.eg.db`
# devtools::install_github("gurinina/G0enrichment", force = TRUE)
#
# BiocManager::install("org.Hs.eg.db", force = TRUE)

## Load required libraries
library(org.Hs.eg.db)
library(clusterProfiler)
library(tidyverse)
library(enrichplot)
library(fgsea)
library(igraph)
library(ggraph)
library(visNetwork)
library(GO.db)
library(GOsemSim)
library(GOenrichment)
library(tidydr)
```

8.4.1.1 Running clusterProfiler: we first need to load the results of the differential expression analysis.

```
res_tableOE = readRDS("data/res_tableOE.RDS")

res_tableOE_tb <- res_tableOE %>%
  data.frame() %>% rownames_to_column(var = "gene") %>%
  dplyr::filter(!is.na(log2FoldChange)) %>% as_tibble()
```

To perform the over-representation analysis, we need a list of background genes and a list of significant genes:

```
## background set of genes
allOE_genes <- res_tableOE_tb$gene

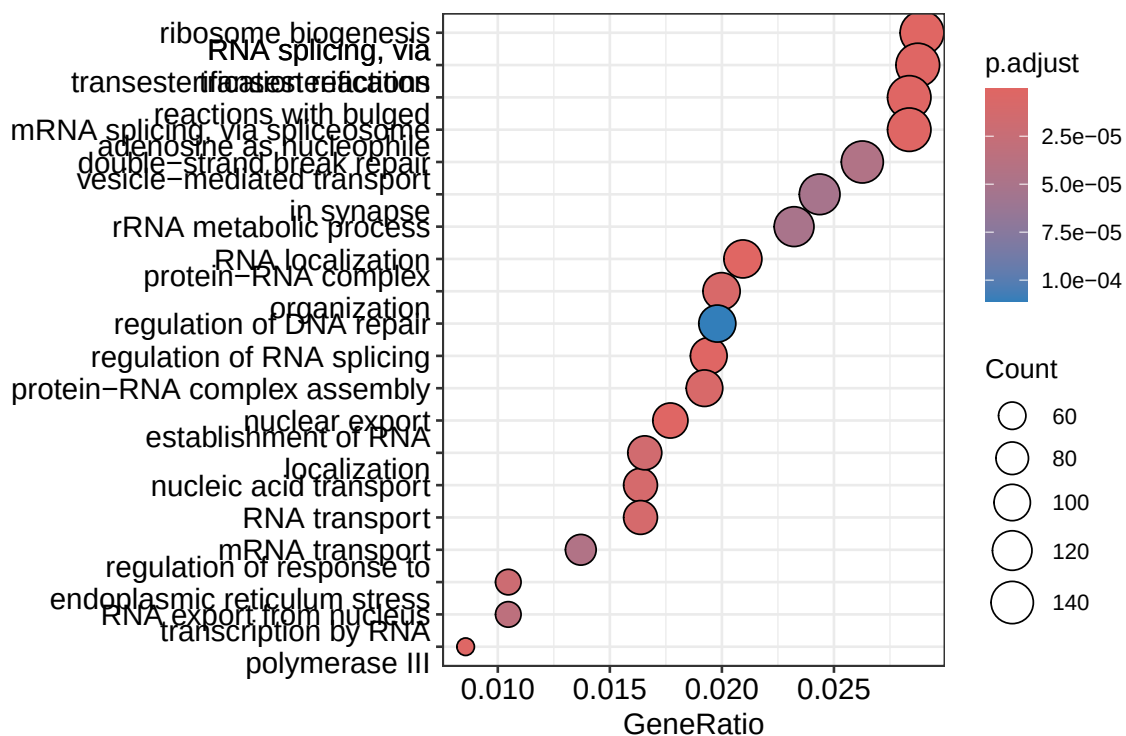
sigOE = dplyr::filter(res_tableOE_tb, padj < 0.05) # Standard padj value

## significant genes
sigOE_genes = sigOE$gene
```

The `enrichGO()` function performs the ORA for the significant genes of interest (`sigOE_genes`) compared to the background gene list (`allOE_genes`) and returns the enriched GO terms and their associated p-values.

```
## Run GO enrichment analysis
ego <- enrichGO(gene = sigOE_genes,
  universe = allOE_genes,
  keyType = "SYMBOL",
  OrgDb = org.Hs.eg.db,
  minGSSize = 20,
  maxGSSize = 300,
  ont = "BP",
  pAdjustMethod = "BH",
  qvalueCutoff = 0.01, # slightly stringent FDR cutoff
  readable = TRUE)
```

```
## Dotplot
dotplot(ego, showCategory=20)
```



8.4.2 Visualizing GO Enrichment Results

After identifying enriched GO terms with `enrichGO()`, we can visualize the relationships between them using several complementary plots from `clusterProfiler`.

8.4.2.1 Network Plot with `emapplot()`

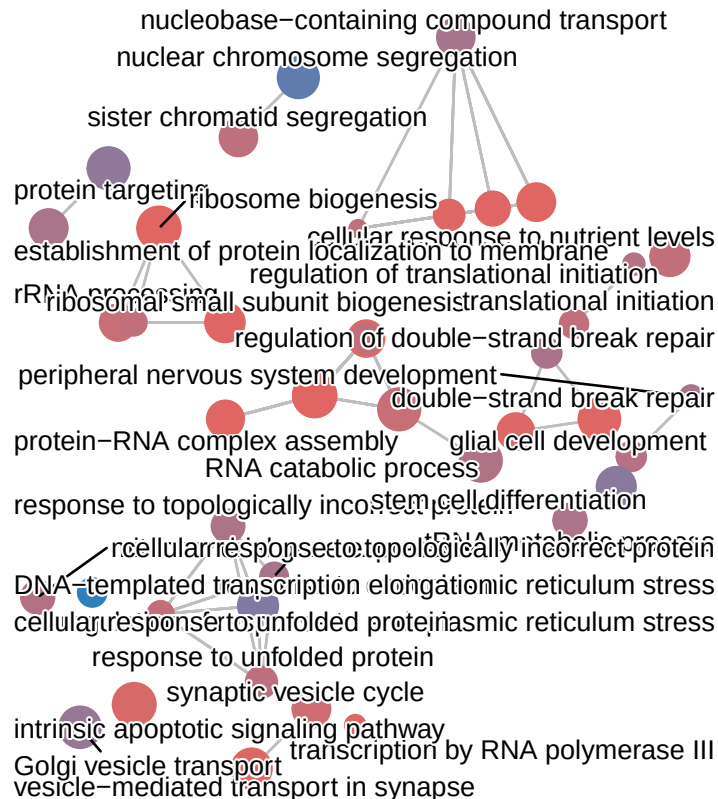
The `emapplot()` function creates a network where: - **Nodes** represent GO terms - **Edges** connect terms that share genes (calculated using Jaccard similarity) - **Clusters** of connected nodes indicate related biological processes

This helps identify groups of functionally related processes affected in your experiment.

```
# Filter to top 50 most significant results for clearer visualization
ego_filtered <- ego
ego_filtered@result <- ego@result[1:50, ] # Take only top 50 results

# Calculate pairwise term similarity based on gene overlap
# "JC" = Jaccard coefficient (size of intersection / size of union)
pwt_filtered <- pairwise_termsim(ego_filtered, method = "JC")

# Create network plot
emapplot(
  pwt_filtered,
  showCategory = 50,
  node_label = "category",
  layout = "mds"
) +
  theme(legend.position = "none")
```



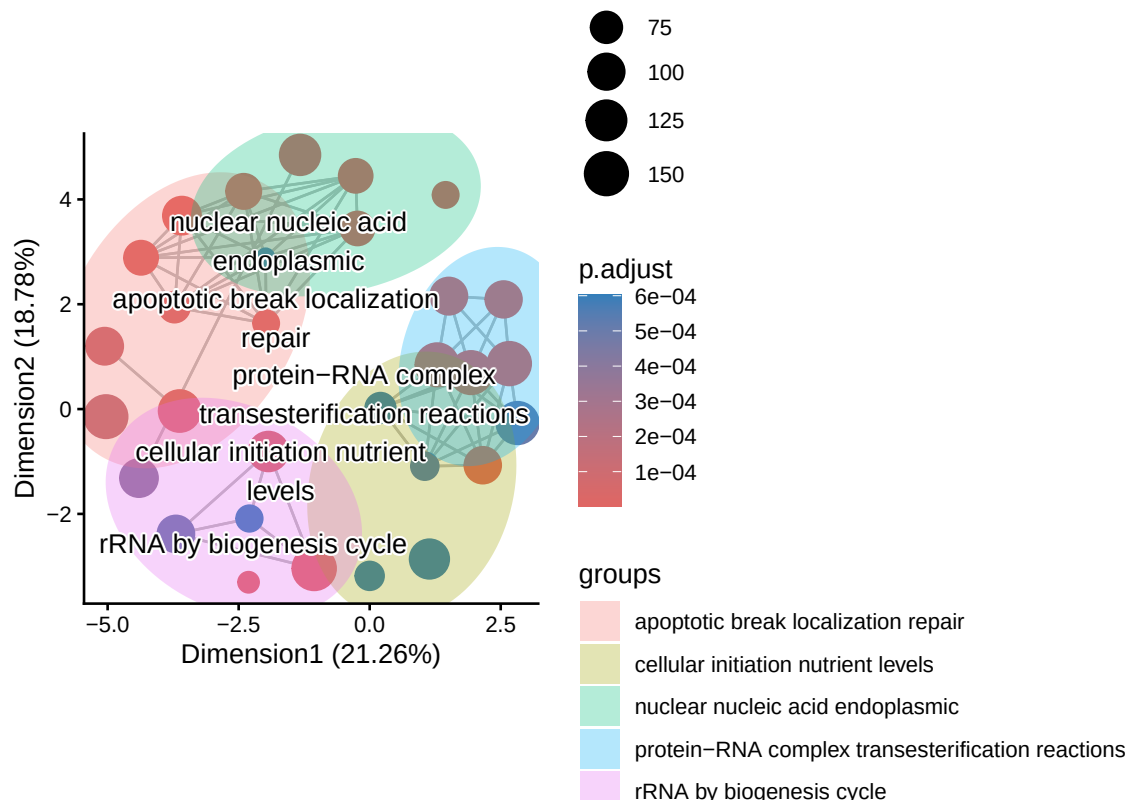
What to look for in the network: - **Clusters** of connected nodes = related biological processes - **Isolated nodes** = unique processes

- **Edge thickness** = strength of gene overlap between terms

8.4.2.2 Overview with ssplot()

The `ssplot()` provides a simplified overview of the most significant terms and their relationships:

```
ssplot(pwt_filtered)
```

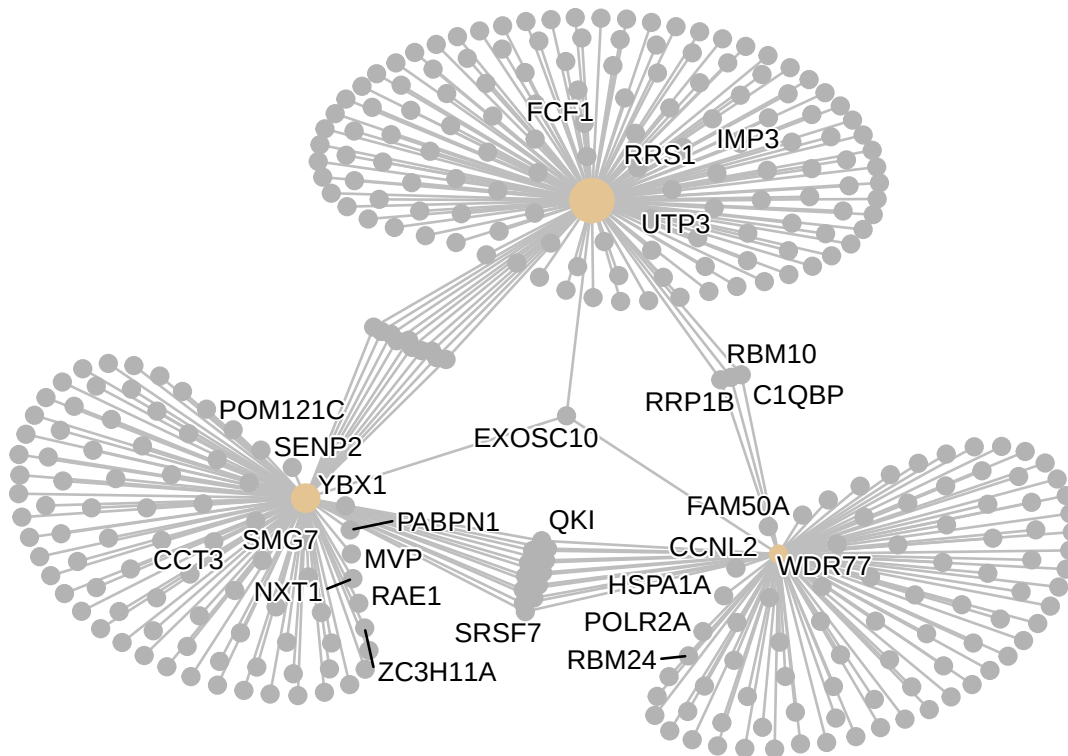


This plot gives you a cleaner, high-level view of term relationships compared to the detailed network above.

8.4.2.3 Gene-Concept Network with cnetplot()

The `cnetplot()` shows which **specific genes** belong to which GO terms. This is useful for identifying genes that may be key regulators involved in multiple biological processes:

```
cnetplot(pwt_filtered, showCategory = 3,  
         categorySize = "pvalue",  
         colorEdge = TRUE) +  
theme(legend.position = "none")
```



Interpretation: In this plot: - **Large circles** = GO terms (size shows significance) - **Small circles** = genes - **Lines** connect genes to their GO terms - Genes connected to multiple terms are potential hub genes affecting several processes

8.4.3 Comparing ORA Methods: enricher vs runGORESP

Now that we've seen ORA with clusterProfiler's `enrichGO()`, let's use another package called `GOenrichment` that performs the same hypergeometric test but adds interactive network visualization using `visNetwork`.

8.4.4 GOenrichment Package Analysis

We're going to run this analysis using TWO different packages. Why? In research, you often want to verify important results using independent methods. We'll see that both give us similar answers (~90% overlap), which builds confidence. They differ slightly because of minor implementation details, which is normal and expected.

`runGORESP` uses over-representation analysis to identify enriched GO terms and returns two `data.frames`; of enriched GO terms (nodes) and GO term relationships (edges). We first need to get the GO annotations and specifically 'Biological Process' (BP).

1. Generate clean GO Biological Process gene sets with gene symbols

```
# install.packages("msigdb") # uncomment to install
library(msigdb)

# Using msigdb ensures standardized, up-to-date gene sets
go_bp <- msigdb(species = "Homo sapiens", collection = "C5", subcollection = "GO:BP")

# Let's look at the format of go_bp
names(go_bp)
```

8.5 [1] “gene_symbol” “ncbi_gene” “ensembl_gene”

8.6 [4] “db_gene_symbol” “db_ncbi_gene” “db_ensembl_gene”

8.7 [7] “source_gene” “gs_id” “gs_name”

8.8 [10] “gs_collection” “gs_subcollection” “gs_collection_name”

8.9 [13] “gs_description” “gs_source_species” “gs_pmid”

8.10 [16] “gs_geoid” “gs_exact_source” “gs_url”

8.11 [19] “db_version” “db_target_species”

```
head(go_bp[,c("gene_symbol", "gs_id")])
```

```
8.12 # A tibble: 6 x 2
```

```
8.13 gene__symbol gs__id
```

```
8.14
```

```
8.15 1 AASDHPPT M29088
```

```
8.16 2 ALDH1L1 M29088
```

```
8.17 3 ALDH1L2 M29088
```

```
8.18 4 MTHFD1 M29088
```

```
8.19 5 MTHFD1L M29088
```

```
8.20 6 MTHFD2L M29088
```

```
dim(go_bp)
```

```
8.21 [1] 617648 20
```

```
# Convert to named list format  
hGOBP.gmt <- split(go_bp$gene_symbol, go_bp$gs_name)  
hGOBP.gmt[1:2]
```


8.22 \$GOBP_10_FORMYLTETRAHYDROFOLATE_METABOLIC_PROCESS

8.23 [1] “AASDHPPT” “ALDH1L1” “ALDH1L2” “MTHFD1” “MTHFD1L”
“MTHFD2L”

8.24

8.25 \$GOBP_2_OXOGLUTARATE_METABOLIC_PROCESS

8.26 [1] “AADAT” “ADHFE1” “COL6A1” “D2HGDH” “DLD” “DLST”
“GOT1” “GOT2”

8.27 [9] “GPT2” “IDH1” “IDH2” “KGD4” “KYAT3” “L2HGDH” “OGDH”
“OGDHL”

8.28 [17] “PHYH” “TAT”

```
# Clean up term names (remove GOBP_ prefix and replace underscores)
names(hGOBP.gmt) <- gsub("^GOBP_", "", names(hGOBP.gmt))
names(hGOBP.gmt) <- gsub("_", " ", names(hGOBP.gmt))
hGOBP.gmt[1:2]
```

8.29 \$10 FORMYLTETRAHYDROFOLATE METABOLIC PROCESS

8.30 [1] “AASDHPPT” “ALDH1L1” “ALDH1L2” “MTHFD1” “MTHFD1L”
“MTHFD2L”

8.31

8.32 \$2 OXOGLUTARATE METABOLIC PROCESS

8.33 [1] “AADAT” “ADHFE1” “COL6A1” “D2HGDH” “DLD” “DLST”
“GOT1” “GOT2”

8.34 [9] “GPT2” “IDH1” “IDH2” “KGD4” “KYAT3” “L2HGDH” “OGDH”
“OGDHL”

8.35 [17] “PHYH” “TAT”

Create GO ID (go_input argument) for runGORESP: assigns GO IDs to GO terms

```
go_mapping <- go_bp[!duplicated(go_bp$gs_name), c("gs_name", "gs_id")]
```

```
# Let's look at the format of go_mapping  
head(go_mapping)
```

8.36 # A tibble: 6 x 2

8.37 gs_name gs_id

8.38

8.39 1 GOBP_10_FORMYLTETRAHYDROFOLATE__METABOLIC__PROCE
M29088

8.40 2 GOBP_2FE_2S__CLUSTER__ASSEMBLY M23654

8.41 3 GOBP_2_OXOGLUTARATE__METABOLIC__PROCESS M15857

8.42 4 GOBP_3_UTR_MEDIATED__MRNA__DESTABILIZATION
M24263

8.43 5 GOBP_3_UTR_MEDIATED__MRNA__STABILIZATION M11060

8.44 6 GOBP_4FE_4S__CLUSTER__ASSEMBLY M45874

```
names(go_mapping) <- c("term", "GOID")

# Cleanup GO terms
go_mapping$term <- gsub("^GOBP_", "", go_mapping$term)
go_mapping$term <- gsub("_", " ", go_mapping$term)

head(go_mapping)
```

8.45 `# A tibble: 6 x 2`

8.46 `term GOID`

8.47

8.48 `1 10 FORMYLTETRAHYDROFOLATE METABOLIC PROCESS
M29088`

8.49 `2 2FE 2S CLUSTER ASSEMBLY M23654`

8.50 `3 2 OXOGLUTARATE METABOLIC PROCESS M15857`

8.51 `4 3 UTR MEDIATED MRNA DESTABILIZATION M24263`

8.52 `5 3 UTR MEDIATED MRNA STABILIZATION M11060`

8.53 `6 4FE 4S CLUSTER ASSEMBLY M45874`

2. `enricher` uses over-representation analysis to identify enriched GO terms. Prepare data for `enricher`: Create `TERM2GENE` data frame (term = GO term, gene = gene symbol) for the GO library

```
term2gene <- go_bp[, c("gs_name", "gene_symbol")]  
head(term2gene)
```

8.54 # A tibble: 6 x 2

8.55 gs__name gene__symbol

8.56

8.57 1 GOBP_10_FORMYLTETRAHYDROFOLATE_METABOLIC_PROCE
AASDHPPT

8.58 2 GOBP_10_FORMYLTETRAHYDROFOLATE_METABOLIC_PROCE
ALDH1L1

8.59 3 GOBP_10_FORMYLTETRAHYDROFOLATE_METABOLIC_PROCE
ALDH1L2

8.60 4 GOBP_10_FORMYLTETRAHYDROFOLATE_METABOLIC_PROCE
MTHFD1

8.61 5 GOBP_10_FORMYLTETRAHYDROFOLATE_METABOLIC_PROCE
MTHFD1L

8.62 6 GOBP_10_FORMYLTETRAHYDROFOLATE_METABOLIC_PROCE
MTHFD2L

```
names(term2gene) <- c("term", "gene")

# Cleanup GO terms
term2gene$term <- gsub("^GOBP_", "", term2gene$term)
term2gene$term <- gsub("_", " ", term2gene$term)

head(term2gene)
```

8.63 `# A tibble: 6 x 2`

8.64 `term gene`

8.65

8.66 `1 10 FORMYLTETRAHYDROFOLATE METABOLIC PROCESS
AASDHPPT`

8.67 `2 10 FORMYLTETRAHYDROFOLATE METABOLIC PROCESS
ALDH1L1`

8.68 `3 10 FORMYLTETRAHYDROFOLATE METABOLIC PROCESS
ALDH1L2`

8.69 `4 10 FORMYLTETRAHYDROFOLATE METABOLIC PROCESS
MTHFD1`

8.70 `5 10 FORMYLTETRAHYDROFOLATE METABOLIC PROCESS
MTHFD1L`

8.71 `6 10 FORMYLTETRAHYDROFOLATE METABOLIC PROCESS
MTHFD2L`

3. Run `enricher` analysis:

```
# Optimal parameters for robust analysis
enricher_result <- enricher(
  gene = sigOE_genes,
  TERM2GENE = term2gene,
  pvalueCutoff = 1,
  qvalueCutoff = 0.0001,      # Stringent FDR threshold
  minGSSize = 20,             # Robust minimum size
  maxGSSize = 300
)
```

4. Prepare data for `runGORESP` Create `scoreMat`: data frame with gene, score, and index columns

```
# Filter to only include genes that have GO annotations
all_genes <- unique(term2gene$gene)
```

```
sigOE_genes <- sigOE_genes[sigOE_genes %in% all_genes]

# Now create scoreMat with the filtered gene lists
scoreMat <- data.frame(
  gene = all_genes,
  score = runif(length(all_genes)),      # Random scores
  index = ifelse(all_genes %in% sigOE_genes, 1, 0), # 1 = gene of interest
  stringsAsFactors = FALSE
)

head(scoreMat)
```

8.72 gene score index

8.73 1 AASDHPPT 0.5082527 1

8.74 2 ALDH1L1 0.1969996 1

8.75 3 ALDH1L2 0.1870385 1

8.76 4 MTHFD1 0.1142226 1

8.77 5 MTHFD1L 0.7391241 1

8.78 6 MTHFD2L 0.8296142 0

```
dim(scoreMat)
```

8.79 [1] 18000 3

```
length(all_genes)
```

8.80 [1] 18000

```
length(sigOE_genes)
```

8.81 [1] 5037

```
table(scoreMat$index)
```

8.82

8.83 0 1

8.84 12963 5037

5. Run runGORESP analysis:

```
goresp_result <- runGORESP(
  scoreMat = scoreMat,
  bp_input = hGOBP.gmt,           # Gene sets as named list
  go_input = go_mapping,          # GO term to ID mapping
  fdrThresh = 0.0001,             # FDR threshold (same as enricher)
  minSetSize = 20,                # Same as enricher minGSSize
  maxSetSize = 300                # Same as enricher maxGSSize
)

names(goresp_result$edgeMat)
```

8.85 [1] “source” “target” “overlapCoeff” “width” “label”

```
names(goresp_result$enrichInfo)
```


8.86 [1] "filename" "GOID" "term"

8.87 [4] "nGenes" "nQuery" "nOverlap"

8.88 [7] "querySetFraction" "geneSetFraction" "foldEnrichment"

8.89 [10] "P" "FDR" "overlapGenes"

8.90 [13] "maxOverlapGeneScore" "cluster" "id"

8.91 [16] "size" "formattedLabel"

```
head(goresp_result$enrichInfo[,c(2,3,4,5,10)])
```

8.92 GOID term nGenes nQuery P

8.93 1 M16023 RNA LOCALIZATION 204 4998 4.42e-15

8.94 2 M16816 REGULATION OF RNA SPLICING 189 5150 1.29e-14

8.95 3 M24840 VESICLE MEDIATED TRANSPORT IN SYNAPSE 260
5117 2.32e-13

8.96 4 M15413 NUCLEAR EXPORT 172 5112 1.40e-12

8.97 5 M15304 STEM CELL DIFFERENTIATION 265 5003 1.85e-11

8.98 6 M23968 ESTABLISHMENT OF RNA LOCALIZATION 165 5115
1.57e-11

```
dim(goresp_result$edgeMat)
```

8.99 [1] 59 5

```
dim(goresp_result$enrichInfo)
```

8.100 [1] 57 17

6. Display top results from both methods

```
cat("\n=== Top enricher results ===\n")
```

8.101

8.102 === Top enricher results ===

```
print(head(enricher_result[, c("Description", "Count", "pvalue", "qvalue")))
```


8.103	Description			
8.104	RNA LOCALIZATION	RNA LOCALIZATION		
8.105	REGULATION OF RNA SPLICING	REGULATION OF RNA SPLICING		
8.106	VESICLE MEDIATED TRANSPORT IN SYNAPSE	VESICLE MEDIATED TRANSPORT IN SYNAPSE		
8.107	NUCLEAR EXPORT	NUCLEAR EXPORT		
8.108	ESTABLISHMENT OF RNA LOCALIZATION	ESTABLISHMENT OF RNA LOCALIZATION		
8.109	STEM CELL DIFFERENTIATION	STEM CELL DIFFERENTIATION		
8.110	Count	pvalue	qvalue	
8.111	RNA LOCALIZATION	110	4.417655e-15	1.053727e-11
8.112	REGULATION OF RNA SPLICING	103	1.293937e-14	1.543190e-11
8.113	VESICLE MEDIATED TRANSPORT IN SYNAPSE	128	2.315881e-13	1.841329e-10
8.114	NUCLEAR EXPORT	92	1.404827e-12	8.377206e-10
8.115	ESTABLISHMENT OF RNA LOCALIZATION	87	1.573398e-11	7.345982e-09
8.116	STEM CELL DIFFERENTIATION	125	1.847842e-11	7.345982e-09

```
cat("\n=== Top runGORESP results ===\n")
```

8.117

8.118 === Top runGORESP results ===

```
print(head(goresp_result$enrichInfo[, c("term", "nOverlap", "P", "FDR")]))
```

8.119 term nOverlap P FDR

8.120 1 RNA LOCALIZATION 110 4.42e-15 1.48e-11

8.121 2 REGULATION OF RNA SPLICING 103 1.29e-14 2.17e-11

8.122 3 VESICLE MEDIATED TRANSPORT IN SYNAPSE 128 2.32e-13
2.59e-10

8.123 4 NUCLEAR EXPORT 92 1.40e-12 1.18e-09

8.124 5 STEM CELL DIFFERENTIATION 125 1.85e-11 1.03e-08

8.125 6 ESTABLISHMENT OF RNA LOCALIZATION 87 1.57e-11 1.03e-
08

```
print(dim(enricher_result))
```

8.126 [1] 65 12

```
print(dim(goresp_result$enrichInfo))
```

8.127 [1] 57 17

7. Let's check the overlap between the enriched terms found using `runGORESP` and those found using `enricher` as they used the same GO term libraries:

```
overlap_terms <- intersect(enricher_result$ID, goresp_result$enrichInfo$term)
cat("Overlapping terms:", length(overlap_terms), "\n")
```

8.128 Overlapping terms: 57

```
cat("Overlap percentage:", round(100 * length(overlap_terms) /
                                max(nrow(enricher_result), nrow(goesp_result$enrichInfo)), 1), "%\n")
```

8.129 Overlap percentage: 87.7 %

Percent overlap is close to 90% that is very good.

Key differences between methods - Both use hypergeometric testing with Benjamini-Hochberg FDR correction - `enricher`: Standard clusterProfiler workflow - `runGORESP`: Adds gene set clustering and network visualization features - Expected difference: 5-15% due to minor implementation differences - Core significant results are nearly identical

We can visualize the results of the `runGORESP` analysis using the `visNetwork` package. This package allows us to create interactive network visualizations.

We use the `visSetup` function to prepare the data for network visualization. We then run the `runNetwork` function to generate the interactive network plot.

```
vis = visSetup(goesp_result$enrichInfo, goresp_result$edgeMat)
GOenrichment::runNetwork(vis$nodes, vis$edges, main = "MOV10 OE GO Enrichment Network")
```

This network analysis is based on Cytoscape, an open source bioinformatics software platform for visualizing molecular interaction networks from Gary Bader at the University of Toronto.

Exercise points = +4 1. Perform a GO enrichment analysis using the `runGORESP` function from the `GOenrichment` package using `res_tableKD` with a `padj` cutoff of 0.05. Use a significance FDR threshold of 0.0001. Save the results in a variable called `kresp`.

2. Visualize the enriched GO terms using the `runNetwork` function from the `GOenrichment` package. Save the visualization setup in a variable called `kvis`.

8.130 Gene Set Enrichment Analysis (GSEA) - Optional

Note: This section is optional. GSEA uses all ranked genes rather than just significant ones. We'll focus on ORA in this course, but GSEA is shown here for reference.

8.130.1 GSEA Using 'clusterProfiler' 'gseGO'

Prepare fold changes, sort by expression, and run GSEA with `gseGO()` from `clusterProfiler`

```
## Extract the foldchanges
foldchanges <- res_tableOE_tb$log2FoldChange

## Name each fold change with the gene name
names(foldchanges) <- res_tableOE_tb$gene

## Sort fold changes in decreasing order
foldchanges <- sort(foldchanges, decreasing = TRUE)
```

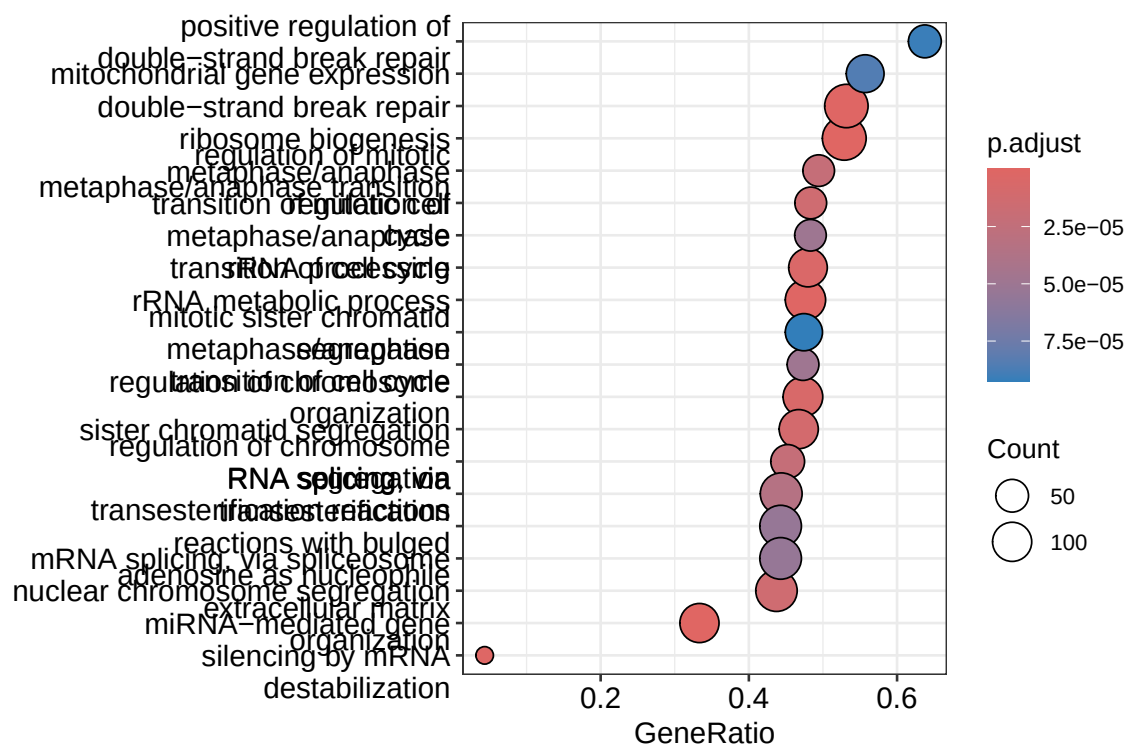
We can use GSEA to test whether genes in specific biological processes tend to be up- or down-regulated together in our experiment:

```
gseaGO <- clusterProfiler::gseGO(
  geneList = foldchanges,
  ont = "BP",
  keyType = "SYMBOL",
  minGSSize = 20,
  maxGSSize = 300,
  eps = 0,
  pAdjustMethod = "BH",
  pvalueCutoff = 0.01,
  verbose = TRUE,
  OrgDb = org.Hs.eg.db,
  by = "fgsea"          # fast gene set enrichment analysis
)
```

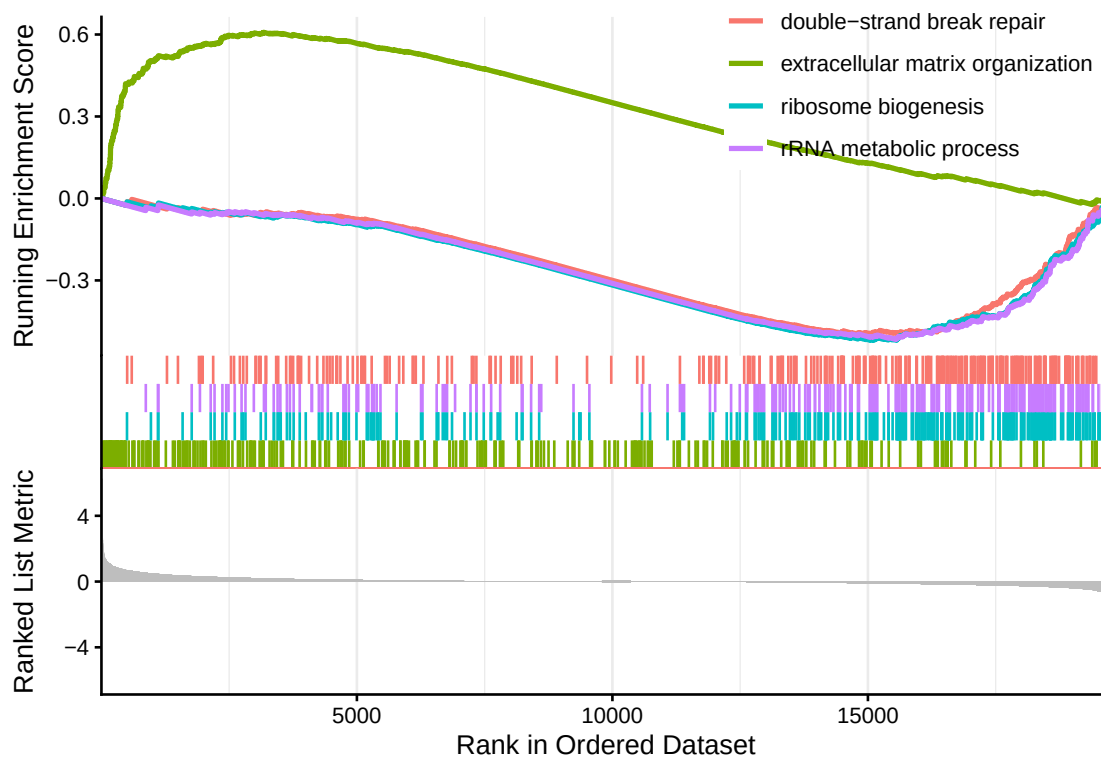
8.130.2 Visualizing GSEA Results

Visualize GSEA results with `dotplot()` for significant categories and `gseaplot2()` for gene set ranks.

```
dotplot(gseaGO, showCategory = 20)
```



```
gseaplot2(gseaG0, geneSetID = 1:4)
```



The emapplot clusters the 50 most significant (by padj) GO terms to visualize relationships between terms.

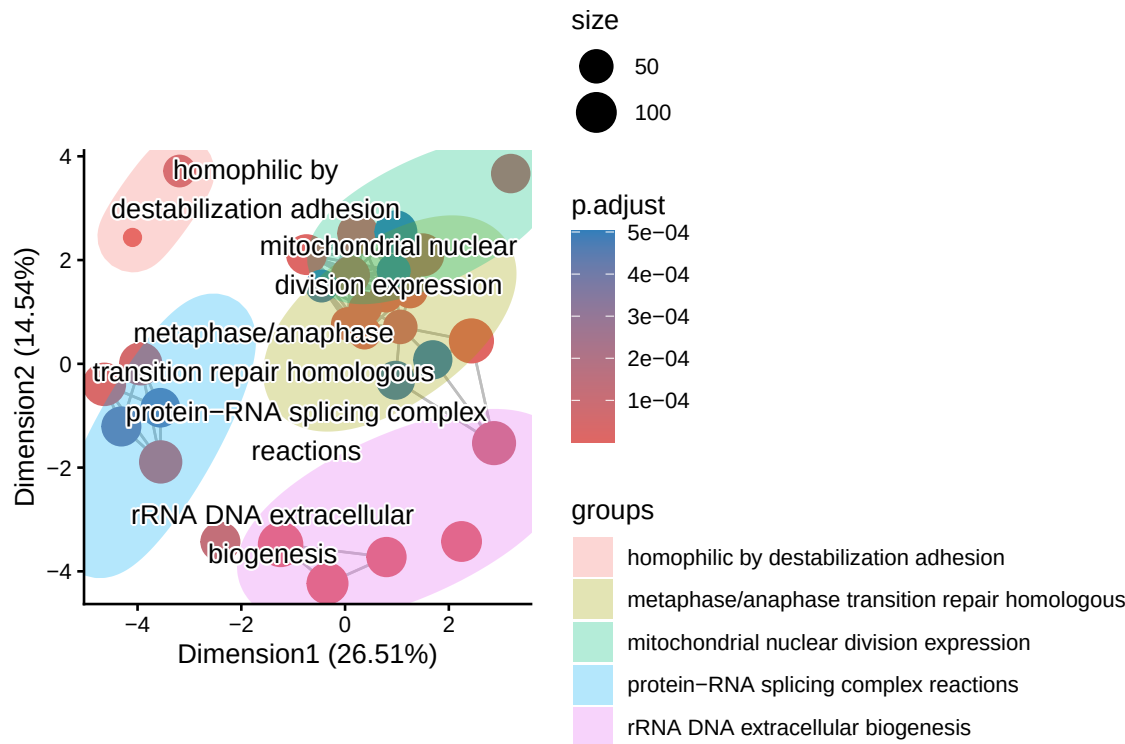
```
gseaGO_filtered <- gseaGO
gseaGO_filtered@result <- gseaGO@result[1:50, ] # Top 50 by p.adjust

# Create pairwise similarity
pwt <- pairwise_termsim(
  gseaGO_filtered,
  method = "JC",
  semData = NULL
)

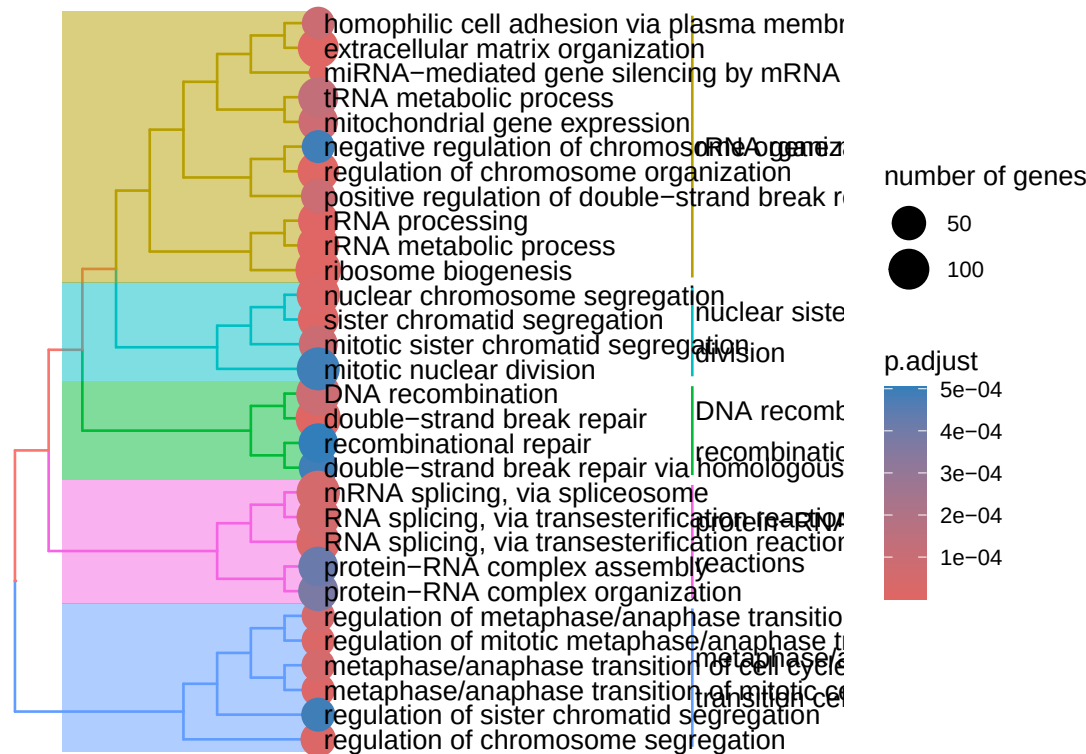
emapplot(pwt, showCategory = 50,
  node_label = "category")
```



```
ssplot(pwt)
```



treeplot(pwt)



We are going to compare ORA `enricher` to running the ORA `runGORESP`.

Let's first load the `G0enrichment` package:

```
# Uncomment the following if you haven't yet installed G0enrichment.  
# devtools::install_github("gurinina/G0enrichment")  
library(G0enrichment)
```

8.130.3 Other tools and resources

- GeneMANIA. GeneMANIA finds other genes that are related to a set of input genes, using a very large set of functional association data curated from the literature.
- ReviGO. Revigo is an online GO enrichment tool that allows you to copy-paste your significant gene list and your background gene list. The output is a visualization of enriched GO terms in a hierarchical tree.
- AmiGO. AmiGO is the current official web-based set of tools for searching and browsing the Gene Ontology database.
- etc.